



UNIVERSIDAD
DE GRANADA

TRABAJO FIN DE GRADO
GRADO EN INGENIERÍA INFORMÁTICA

Evaluación de técnicas de preprocesamiento con Deep Learning para clasificación de imágenes histológicas en predicción de cáncer de mama

Autor

Juan Carlos Mora Díaz

Directores

Alberto Guillén Perales

Luis Javier Herrera Maldonado

Mentora

Nerea Hernández Carpintero



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Junio de 2026

Resumen

Este Trabajo de Fin de Grado aborda el problema de clasificación de lesiones mamarias histopatológicas a partir del dataset BRACS, un conjunto de imágenes con tinción H&E anotadas por especialistas y organizado en 3 y 7 clases. La motivación del trabajo surge de la elevada complejidad visual del problema y de la necesidad de estudiar hasta qué punto los modelos actuales de *Deep Learning* pueden ofrecer un apoyo fiable al diagnóstico en este contexto.

El objetivo principal del proyecto ha sido desarrollar y evaluar un *pipeline* de clasificación histopatológica, estudiando distintas familias de modelos, distintas estrategias de reducción del conjunto de entrenamiento, reglas de agregación desde el nivel de *patch* hasta nivel de ROI y mecanismos de análisis de incertidumbre. Para ello, se ha partido de un baseline inicial a nivel de *patch* comparando CNNs, *Vision Transformers* y modelos fundacionales, y se ha continuado durante todo el trabajo con los *embeddings* extraídos por los modelos más competitivos. Sobre estos *embeddings* se han analizado métodos de limpieza de *patches*, evaluación a nivel de ROI, abstención en casos dudosos, análisis no supervisado del espacio latente, prototipos claros y experimentos exploratorios de selección de características.

Los resultados obtenidos muestran que los modelos fundacionales son claramente superiores al resto de modelos evaluados, destacando H-Optimus1 y Virchow2. Asimismo, se observa que la agregación a nivel de ROI mediante media de probabilidades constituye la estrategia más sólida, y que los mecanismos de abstención permiten identificar los casos más dudosos. Los análisis complementarios realizados apuntan además a que una parte importante de las confusiones del sistema no depende del clasificador, sino de la ambigüedad presente en el propio dataset. En conjunto, el trabajo confirma el potencial de los modelos fundacionales para este problema y aporta una evaluación amplia, razonada y reproducible del comportamiento del sistema en un escenario histopatológico complejo.

Palabras clave: cáncer de mama, histopatología, CNNs, *Deep Learning*, H&E, BRACS, *pipeline*, *baseline*, *patch*, *dataset*, modelos fundacionales, *Vision Transformers* (ViTs), clasificación multiclase, regiones de interés (ROIs), *embeddings*, abstención, incertidumbre.

Abstract

This Bachelor's Thesis addresses the problem of classifying breast histopathological lesions using the BRACS dataset, a collection of H&E-stained images annotated by specialists and organized into 3 and 7 classes. The motivation for this work arises from the high visual complexity of the problem and from the need to study to what extent current *Deep Learning* models can provide reliable support for diagnosis in this context.

The main objective of the project has been to develop and evaluate a histopathology classification *pipeline*, studying different model families, different strategies for reducing the training set, aggregation rules from the *patch* level to the ROI level, and uncertainty analysis mechanisms. To this end, the work started from an initial *baseline* at the *patch* level, comparing CNNs, *Vision Transformers*, and foundation models, and continued throughout the project using the *embeddings* extracted by the most competitive models. On top of these *embeddings*, several analyses were carried out, including *patch*-cleaning methods, ROI-level evaluation, abstention on doubtful cases, unsupervised latent-space analysis, clear-prototype, and exploratory feature-selection experiments.

The obtained results show that foundation models clearly outperform the rest of the evaluated models, with H-Optimus1 and Virchow2 standing out in particular. In addition, ROI-level aggregation based on mean probabilities proved to be the most robust strategy, while abstention mechanisms made it possible to identify the most doubtful cases. The complementary analyses further suggest that an important part of the system's confusion does not depend on the classifier itself, but rather on the ambiguity present in the dataset. Overall, this work confirms the potential of foundation models for this problem and provides a broad, reasoned, and reproducible evaluation of system behaviour in a complex histopathological scenario.

Keywords: breast cancer, histopathology, CNNs, *Deep Learning*, H&E, BRACS, *pipeline*, *baseline*, *patch*, *dataset*, foundation models, *Vision Transformers* (ViTs), multiclass classification, regions of interest (ROIs), *embeddings*, abstention, uncertainty.

Índice general

Resumen	1
Abstract	2
1. Introducción	11
1.1. El Cáncer	11
1.1.1. El cáncer de mama	12
1.2. Patología digital e Imágenes Histológicas	16
1.2.1. Del microscopio a la pantalla: Whole Slide Images (WSI)	16
1.2.2. Química del color: H&E	16
1.3. Motivación: IA en el diagnóstico médico	17
1.3.1. El sesgo humano y la variabilidad inter-observador	17
1.3.2. Evolución: De los sistemas expertos al Deep Learning	18
1.3.3. Distintos tipos de Deep Learning	19
1.4. Objetivos del proyecto	19
1.5. Estructura de la memoria	20
2. Fundamentos y Estado del Arte	22
2.1. Inteligencia artificial en Visión por computador	22
2.1.1. Aprendizaje supervisado	23
2.1.2. Aprendizaje profundo (<i>Deep Learning</i>)	29
2.2. Arquitecturas de Deep Learning	35
2.2.1. Redes Convolucionales (CNNs)	35
2.2.2. Vision Transformers (ViTs)	43
2.2.3. Modelos Fundacionales (Foundation Models)	47
2.3. Modelos fundacionales y trabajos recientes en patología digital	50
2.3.1. El modelo UNI	50
2.3.2. El modelo Virchow	52
2.3.3. El modelo fundacional basado en datos públicos: Phikon	53
2.3.4. Benchmark clínico de modelos fundacionales: PathBench	55
3. Dataset BRACS	58
3.1. Baseline oficial y dificultad del dataset	61
Trabajo Fin de Grado	3

4. Metodología y Planificación	63
4.1. Enfoque metodológico	63
4.2. Tecnologías y herramientas utilizadas	63
4.3. Planificación y seguimiento del proyecto	65
4.3.1. Seguimiento y coordinación	67
4.3.2. Dedicación temporal estimada	67
4.4. Desviaciones, dificultades y reflexión sobre la planificación	67
4.5. Presupuesto estimado del proyecto	68
5. Experimentación y Resultados	70
5.1. Baseline inicial	70
5.1.1. Descripción del protocolo experimental	70
5.1.2. Resultados	74
5.1.3. Ranking	82
5.2. Limpieza de patches mediante Información Mutua	83
5.2.1. Motivación y fundamento metodológico	83
5.2.2. Decisión metodológica: limpieza exclusiva del conjunto de entre- namiento	84
5.2.3. Pipeline	84
5.2.4. Pruebas y ajuste de parámetros para la limpieza con H-Optimus1 .	86
5.2.5. Pruebas y ajuste de parámetros para la limpieza con Virchow2 . .	91
5.2.6. Discusión conjunta y siguientes alternativas metodológicas	96
5.3. Limpieza de patches con métodos de undersampling estándar	97
5.3.1. Métodos evaluados: RandomUnderSampler y NeighbourhoodClea- ningRule	98
5.3.2. Protocolo experimental	99
5.3.3. Resultados con H-Optimus1	100
5.3.4. Resultados con Virchow2	102
5.3.5. Comparación global	104
5.4. Evaluación a nivel de ROIs a partir de predicciones sobre patches	108
5.4.1. Metodología de agregación patch-ROI	109
5.4.2. Resultados globales de las reglas de votación	111
5.4.3. Comparación final entre métodos a nivel de ROI	112
5.4.4. Discusión	116
5.5. Clasificación a nivel de ROI con abstención y revisión de casos dudosos .	118
5.5.1. Regla de decisión con abstención	118
5.5.2. Selección del umbral de abstención	119
5.5.3. Resultados de la abstención	120
5.5.4. Análisis de casos dudosos y claros	122
6. Trabajo Extra	127
6.1. Análisis no supervisado de la estructura del dataset	127
6.1.1. Motivación de la fase no supervisada	127
6.1.2. Metodología	128

6.1.3. Resultados cuantitativos	129
6.1.4. Análisis visual de la estructura latente	130
6.1.5. Análisis fino con $k = 5$	132
6.1.6. Conclusiones	134
6.2. Proyección clínica en 3 clases	134
6.3. Análisis de ROIs dudosas mediante prototipos claros	137
6.4. Selección de características sobre embeddings	141
7. Conclusiones y Trabajos futuros	143
7.1. Conclusiones	143
7.2. Trabajos futuros	145
Bibliografía	146
A. Repositorio del proyecto	149
A.1. Enlace al repositorio	149
A.2. Estructura general del repositorio	149
A.3. Organización del código experimental	150
A.4. Control de versiones y trazabilidad	151

Índice de figuras

1.1. Tumor benigno y maligno	11
1.2. Tasas de incidencia por sexo (1975-2020) [2]	12
1.3. Número estimado de nuevos casos de cáncer en mujeres a nivel mundial (2022) [2]	13
1.4. Número estimado de muertes por cáncer en mujeres a nivel mundial (2022) [2]	13
1.5. Clasificación del dataset BRACS	15
1.6. Ejemplo de tinción H&E [12]	18
2.1. IA, ML y DL [15]	23
2.2. Comportamiento del Descenso de Gradiente según el LR (η)	27
2.3. Bias-Variance Trade-Off	29
2.4. Modelo de una neurona artificial/perceptrón [16]	30
2.5. Arquitectura típica de un Perceptrón Multicapa (MLP) [17]	31
2.6. Ejemplo de predicción de Albert Einstein	31
2.7. Funciones de activación más comunes en <i>Deep Learning</i> [18]	33
2.8. Arquitectura general de una CNN [19]	36
2.9. <i>Max Pooling</i> y <i>Average Pooling</i> [20]	39
2.10. Bloque de una CNN residual general [21]	41
2.11. Fine-Tuning	43
2.12. Arquitectura general del Vision Transformer (ViT) [23]	44
2.13. Self-Attention utilizando el producto escalar con las matrices Q, K y V. Arquitectura del Multi-Head Attention procesando múltiples proyecciones en paralelo [22]	47
2.14. Arquitectura y pipeline de entrenamiento del modelo UNI [24]	51
2.15. Arquitectura y pipeline de entrenamiento del modelo Virchow [25]	53
2.16. Comparativa de modelos fundacionales en tareas de patología computacional [26]	54
2.17. Resumen del benchmark PathBench [27]	56
3.1. <i>WSI</i> del dataset BRACS junto con distintas <i>ROIs</i>	61
4.1. Cronograma de desarrollo del TFG por fases e hitos de seguimiento	65

5.1. Ejemplo de seguimiento experimental mediante MLflow.	74
5.2. Evolución de la pérdida en entrenamiento y validación para ResNet18 . .	75
5.3. Evolución del F1 macro en entrenamiento y validación para ResNet18 . .	76
5.4. Curvas de <i>train</i> y <i>val</i> para los <i>ViTs</i> en el <i>baseline</i>	78
5.5. Matriz de confusión en validación para Phikon en la semilla 29	79
5.6. Evolución del F1 macro en entrenamiento y validación para Virchow2 en una de las semillas evaluadas	81
5.7. Representación del ranking final del baseline inicial según F1 macro sobre validación.	82
5.8. Sensibilidad del porcentaje de patches eliminados frente al parámetro α en H-Optimus1	88
5.9. Sensibilidad del porcentaje de patches eliminados frente al parámetro α en Virchow2	93
5.10. Comparación del F1 macro y del coste computacional entre el baseline original y los conjuntos limpios	96
5.11. Comparación global del rendimiento medio en validación para los métodos evaluados sobre patches	106
5.12. Comparación del coste de construcción del subconjunto y del tiempo de reentrenamiento por semilla	107
5.13. Matrices de confusión a nivel de ROI para H-Optimus1	114
5.14. Matrices de confusión a nivel de ROI para Virchow2	115
5.15. Compromiso entre <i>coverage</i> y <i>F1</i> macro sobre las ROIs aceptadas al variar el umbral de abstención τ	120
5.16. Gráfico de barras de los resultados con abstención ($\tau = 0,10$).	121
5.17. Confusión benigna entre N y PB	122
5.18. Confusión entre FEA y ADH	123
5.19. Caso de éxito y alta confianza en IC	125
6.1. Proyección PCA 2D de los embeddings de entrenamiento de Virchow2. . .	130
6.2. Proyección PCA 2D de los embeddings de entrenamiento de H-Optimus1. .	131
6.3. Agrupamiento K-Means con $k = 3$ sobre los embeddings de entrenamiento de Virchow2.	131
6.4. Agrupamiento K-Means con $k = 3$ sobre los embeddings de entrenamiento de H-Optimus1.	132
6.5. Mapa de calor de clases reales para Virchow2 con $k = 5$	132
6.6. Mapa de calor de clases reales dentro de cada cluster para H-Optimus1 con $k = 5$	133
6.7. Matrices de confusión a nivel de ROI en 3 clases de las dos configuraciones más competitivas.	136
6.8. ROI dudosa de FEA con prototipo más cercano FEA.	138
6.9. ROI dudosa de ADH próxima a FEA.	139
6.10. ROI dudosa de PB próxima a N.	140
A.1. Vista general del repositorio del TFG en GitHub.	149

A.2. Carpeta de scripts experimentales del proyecto.	150
A.3. Historial de commits del repositorio durante el desarrollo del TFG.	151

Índice de tablas

3.1. Clases histológicas del dataset BRACS y su agrupación clínica.	59
3.2. Resumen de la composición del dataset BRACS.	59
3.3. Distribución de WSIs por clase y subconjunto en BRACS.	60
3.4. Distribución de ROIs por clase y subconjunto en BRACS.	60
3.5. Resultados del baseline publicado por Brancati <i>et al.</i> (2022) sobre el dataset BRACS [4].	62
4.1. Principales bibliotecas, plataformas y herramientas utilizadas en el proyecto.	64
4.2. Planificación por fases de trabajo	66
4.3. Principales mecanismos de seguimiento y coordinación del proyecto	67
4.4. Presupuesto estimado del proyecto.	68
5.1. Resultados de ResNet18 en el baseline	74
5.2. Resultados de ResNet50 en el baseline	75
5.3. Resultados de DeiT-Small en el baseline	77
5.4. Resultados de ViT-B/16 en el baseline	77
5.5. Resultados de Phikon en el baseline	79
5.6. Resultados de Virchow2 en el baseline	80
5.7. Resultado de H-Optimus1 en el baseline	81
5.8. Ranking de modelos del baseline inicial	82
5.9. Estudio empírico del parámetro α sobre un subconjunto aleatorio de 500 patches	86
5.10. Estudio empírico del parámetro α sobre un subconjunto aleatorio de 5000 patches	88
5.11. Resultados de la limpieza completa del conjunto de entrenamiento de H-Optimus1	89
5.12. Resultados de H-Optimus1 reentrenado sobre el conjunto limpio con $\alpha = 0,00025$	90
5.13. Comparación entre el baseline original de H-Optimus1 y el reentrenamiento sobre el conjunto limpio	90
5.14. Estudio empírico del parámetro α sobre un subconjunto aleatorio de 500 patches para Virchow2	92

5.15. Estudio empírico del parámetro α sobre un subconjunto aleatorio de 5000 patches para Virchow2	93
5.16. Resultados de la limpieza completa del conjunto de entrenamiento de Virchow2	94
5.17. Resultados de Virchow2 reentrenado sobre el conjunto limpio con $\alpha = 0,0002$	94
5.18. Resultados de Virchow2 reentrenado sobre el conjunto limpio con $\alpha = 0,00025$	95
5.19. Comparación entre el baseline original de Virchow2 y el reentrenamiento sobre conjuntos limpios	95
5.20. Recalibración de NCR sobre el conjunto completo de entrenamiento . . .	100
5.21. Resultados de H-Optimus1 reentrenado sobre el conjunto reducido mediante RandomUnderSampler	101
5.22. Resultados de H-Optimus1 reentrenado sobre el conjunto filtrado mediante NCR en su configuración inicial ($k = 3$)	101
5.23. Resultados de H-Optimus1 reentrenado sobre el conjunto filtrado mediante NCR recalibrado ($k = 45$)	102
5.24. Resultados de Virchow2 reentrenado sobre el conjunto reducido mediante RandomUnderSampler	103
5.25. Resultados de Virchow2 reentrenado sobre el conjunto filtrado mediante NCR en su configuración inicial ($k = 3$)	103
5.26. Resultados de Virchow2 reentrenado sobre el conjunto filtrado mediante NCR recalibrado ($k = 45$)	104
5.27. Ranking global de métodos de selección de patches en H-Optimus1, ordenado por <i>F1 macro</i> medio en validación	105
5.28. Ranking global de métodos de selección de patches en Virchow2, ordenado por <i>F1 macro</i> medio en validación	105
5.29. Ranking final de métodos de selección de patches.	106
5.30. Comparación computacional de las estrategias evaluadas sobre patches . .	107
5.31. Comparación de reglas de votación a nivel de test-ROI en H-Optimus1 . .	111
5.32. Comparación de reglas de votación a nivel de test-ROI en Virchow2. . .	112
5.33. Comparación final a nivel de ROI en H-Optimus1.	112
5.34. Comparación final a nivel de ROI en Virchow2.	113
5.35. Comparación final entre métodos a nivel de ROI usando mean_proba . . .	113
5.36. Resultados globales a nivel de ROI con abstención ($\tau = 0,10$)	121
6.1. Resultados globales del análisis no supervisado con K-Means sobre embeddings de entrenamiento.	129
6.2. Resultados a nivel de ROI tras la proyección clínica de 7 clases a 3 clases.	135
6.3. Resultados de selección de características sobre embeddings de Virchow2 a nivel de ROI.	141

Capítulo 1

Introducción

1.1. El Cáncer

El *cáncer* engloba un grupo de enfermedades por las que algunas células anormales crecen, se dividen o se multiplican descontroladamente en el cuerpo humano.

El cáncer puede comenzar en cualquier parte del cuerpo, estando formado por billones de células. En condiciones normales, las células humanas se forman y se multiplican (división celular) para formar otras nuevas que el cuerpo necesita. Cuando las células “envejecen” o se dañan, mueren y las nuevas las desplazan.

A veces, estas células anormales o dañadas se forman y se multiplican, cuando no debería ser así. Estas células pueden formar **tumores**, que son bultos de tejido, y que pueden diferenciarse en:

- **Benignos:** Permanecen en su lugar de origen. Suelen crecer lentamente y no invaden tejidos adyacentes. Tras ser eliminados, rara vez vuelven a crecer. Las células que los forman se parecen más a las normales.
- **Maligos (cáncer):** Pueden desprenderse y formar tumores secundarios por todo el cuerpo (metástasis) a través del sistema linfático o sanguíneo. Crecen de manera rápida y descontrolada, destruyendo e invadiendo tejidos cercanos a ellos. A diferencia de los benignos, suelen tener alta tasa de recurrencia (incluso una vez eliminados). Estructuralmente, las células son más anormales y están más desordenadas.

El cáncer es un problema de enorme complejidad diagnóstica. En numerosos contextos clínicos, la dificultad no está únicamente en detectar el tumor, sino en distinguir con precisión entre tumores con distinta malignidad. Esta necesidad de distinción fi-

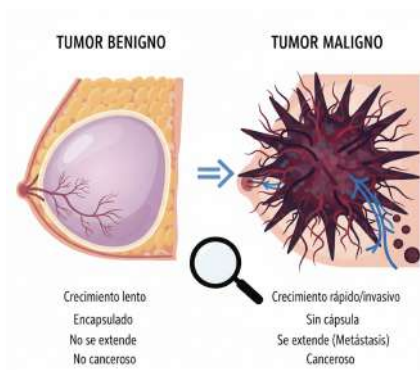


Figura 1.1: Tumor benigno y maligno

na resulta especialmente relevante en anatomía patológica, donde pequeñas diferencias morfológicas pueden condicionar decisiones clínicas muy importantes. Además, no todos los tumores tienen el mismo comportamiento biológico, lo que implica tratamientos muy diferentes entre distintos tipos de cáncer. Estas diferencias a veces vienen determinadas por biomarcadores específicos a nivel genómico o proteómico (las imágenes pueden apoyar pero a veces hacen falta más pruebas), lo que complica aún más el diagnóstico y tratamiento.

1.1.1. El cáncer de mama

El **cáncer de mama** es una proliferación maligna de las células epiteliales que revisten los conductos o lobulillos de la mama. Anatómicamente, la glándula mamaria está formada por lóbulos y lobulillos (productores de leche) conectados por ductos (transportadores) hacia el pezón. Su crecimiento puede darse por *invasión local* (de estructuras vecinas), *diseminación linfática* (ganglios axilares) o *diseminación hematológica* (vasos sanguíneos hacia huesos, pulmón, hígado y piel)[1].

Impacto y estadísticas globales

Como se puede observar en la Figura 1.2, existe un paralelismo epidemiológico entre ambos sexos: el cáncer de mama en las mujeres ha mantenido una posición dominante a lo largo del tiempo, semejante a la del cáncer de próstata en los hombres. Ambas patologías representan los tumores de mayor incidencia en sus respectivos grupos, siendo los principales desafíos de salud pública a nivel global.

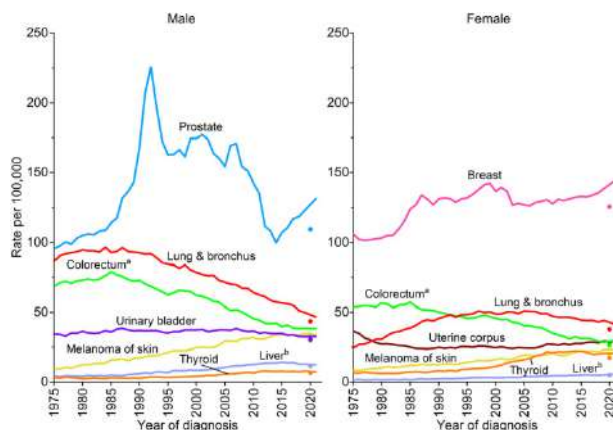


Figura 1.2: Tasas de incidencia por sexo (1975-2020) [2]

Según los datos más recientes del Observatorio Global del Cáncer (GCO) de la Organización Mundial de la Salud (IARC) en el año 2022, el cáncer de mama ocupa el *primer lugar en incidencia entre la población femenina mundial*, superando los **2.3 millones**

de nuevos casos estimados. Esta cifra es significativamente superior a la de otros tipos de cáncer como el de pulmón o el colorrectal (Figura 1.3).

No obstante, el dato más alarmante no es solo su frecuencia, sino su letalidad, siendo la *principal causa de muerte por cáncer* en mujeres, con aproximadamente **670.000 fallecimientos** registrados en 2022, como se refleja en la Figura 1.4.

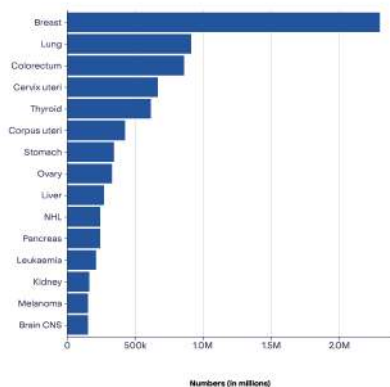


Figura 1.3: Número estimado de nuevos casos de cáncer en mujeres a nivel mundial (2022) [2]

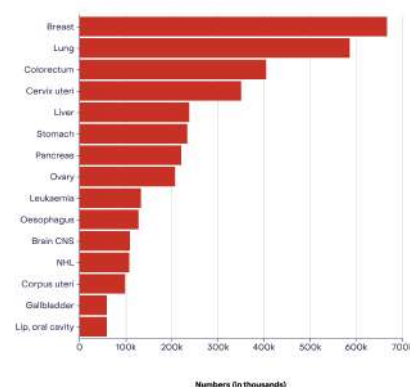


Figura 1.4: Número estimado de muertes por cáncer en mujeres a nivel mundial (2022) [2]

Estas cifras no solo muestran la relevancia médica del problema, sino también la necesidad de herramientas computacionales que ayuden a mejorar la precisión diagnóstica, reducir la variabilidad entre especialistas y agilizar el análisis de muestras histológicas.

Detección y Diagnóstico

La detección del cáncer de mama puede producirse por dos vías:

- Cribado poblacional (mamografías preventivas)
- Aparición de síntomas anómalos

Según la práctica médica habitual, una paciente puede sospechar de una anomalía al palpar un *engrosamiento* en la mama, o al observar cambios inusuales en la morfología del pezón, como descamación, retracción o secreción de sangre.

Ante estos casos, o ante una imagen sospechosa en una mamografía o ecografía, es necesario confirmar el diagnóstico. Como algunas de estas técnicas tienen limitaciones y pueden arrojar falsos positivos, el diagnóstico definitivo o el estándar siempre requiere el análisis celular mediante una biopsia [3].

La **biopsia mamaria** es el procedimiento médico mediante el cual se extrae una muestra de tejido para su análisis histopatológico.

Aunque existen diversas técnicas (como la aspiración con aguja fina o la biopsia quirúrgica abierta), en el contexto del diagnóstico oncológico y para la generación de imágenes histológicas, la técnica más relevante es la ***Biopsia con Aguja Gruesa***:

1. Se utiliza una aguja hueca de mayor calibre para extraer pequeños cilindros de tejido.
2. A diferencia de otras técnicas que solo extraen células sueltas, esta biopsia preserva la *arquitectura histológica* del tejido, permitiendo evaluar no solo las células tumorales, sino su interacción con el estroma circundante.
3. La muestra extraída es la que posteriormente se fija, se tiñe y se digitaliza para generar las WSI.

Una vez confirmado el diagnóstico mediante el informe de patología, se determina el estado y el subtipo del tumor.

En el contexto de este TFG, y siguiendo el esquema del dataset utilizado **BRACS** (BReAst Carcinoma Subtyping) [4], las lesiones se clasifican en **7 subclases histológicas**, agrupadas en tres grandes categorías [5].

Tejido Normal y Lesiones Benignas

Este grupo representa tipos de tejido que no suponen un riesgo oncológico inmediato.

- **Normal (N):** Tejido mamario sano, caracterizado por conductos y lobulillos formados por una doble capa celular (epitelial y mioepitelial), rodeada de estroma y tejido adiposo.
- **Patológico Benigno (PB):** Incluye lesiones como fibroadenomas o quistes. Aunque hay alteraciones en la mama (bultos), las células mantienen una apariencia normal y no hay riesgo de malignidad.
- **Hiperplasia Ductal Usual (UDH):** Se produce un aumento en el número de células dentro del conducto (*hiperplasia*). La clave visual es que las células son heterogéneas (diferentes entre sí). Al no ser clonales, se consideran benignas.

Lesiones Atípicas

Este grupo representa lesiones proliferativas de alto riesgo (precancerosas). Son el **grupo más complicado de discriminar** debido a su enorme similitud visual con el cáncer temprano.

- **Atipia Epitelial Plana (FEA):** Los conductos aparecen dilatados y revestidos por una o varias capas de células que empiezan a ser monótonas (clones), perdiendo su orientación natural.

- **Hiperplasia Ductal Atípica (ADH):** *Es la clase más crítica y difícil de detectar.* Morfológicamente, presenta las mismas características que un carcinoma de bajo grado (células rígidas, espacios cribiformes geométricos), pero está limitado a un tamaño muy pequeño (< 2 mm) [4].

Lesiones Malignas

Representan el cáncer, y se diferencian dos por su capacidad de invasión.

- **Carcinoma Ductal In Situ (DCIS):** Es una proliferación neoplásica maligna que llena el conducto mamario. Aunque las células son cancerosas (con núcleos grandes y atípicos), la **membrana basal** que rodea el conducto permanece intacta. Esto significa que el tumor está envuelto y aún no tiene capacidad física de provocar *metástasis*.
- **Carcinoma Invasivo (IC):** Es el final del proceso canceroso. Las células tumorales han roto la membrana basal e invaden el estroma circundante y el tejido graso. Visualmente, se pierde la estructura de conducto y aparecen nidos de células tumorales dispersos de forma desordenada. Es la lesión de peor pronóstico.

Esta jerarquía diagnóstica pone de manifiesto que el problema abordado en este trabajo no consiste solo en separar tejido sano de tejido tumoral, sino en diferenciar subtipos histológicos con características morfológicas casi idénticas. En particular, las lesiones atípicas son las que ocupan la zona más compleja.

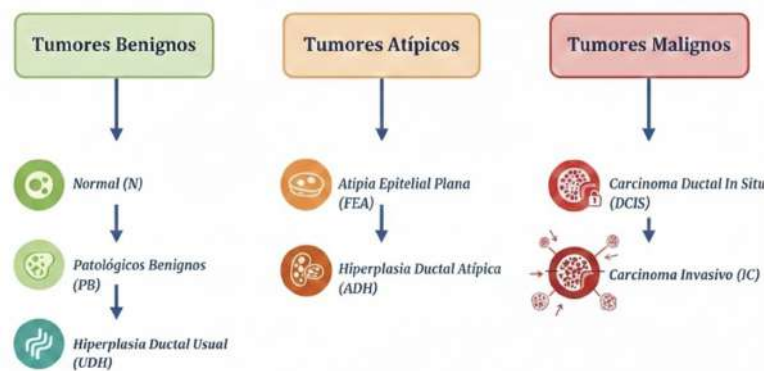


Figura 1.5: Clasificación del dataset BRACS

1.2. Patología digital e Imágenes Histológicas

1.2.1. Del microscopio a la pantalla: Whole Slide Images (WSI)

La histopatología ha experimentado un cambio brusco en la última década.

Tradicionalmente estaba limitada al entorno analógico, donde el diagnóstico dependía exclusivamente de la observación directa a través de un microscopio óptico.

Sin embargo, la disciplina ha evolucionado hacia la **Patología Digital**. Este nuevo entorno aglutina la adquisición, gestión e interpretación de la información tisular en formato electrónico, permitiendo la aplicación de herramientas tecnológicas para un mejor análisis cuantitativo y mayor precisión [6].

El elemento central de esta evolución es la *Imagen de Diapositiva Completa* o *Whole Slide Image* (WSI). Estas imágenes se generan mediante escáneres de alta resolución y consta de dos fases críticas principales:

1. **Adquisición por teselas:** El escáner captura secuencialmente miles de pequeñas imágenes individuales (teselas o tiras) de la muestra mientras recorre el portaobjetos.
2. **Stitching (Cosido digital):** Mediante algoritmos software sofisticados, estas teselas se fusionan para crear una única imagen panorámica coherente y de altísima resolución [7].

El resultado es un archivo enorme (Gigapíxeles). Para gestionar este volumen de datos y permitir una visualización fluida, las WSI emplean formatos de almacenamiento con compresión eficiente como *JPEG2000* y se estructuran internamente en forma de **pirámide multirresolución**. Esta arquitectura permite al sistema cargar y visualizar únicamente las teselas necesarias para el nivel de zoom actual, optimizando el ancho de banda y la memoria, de forma similar a como funcionan los mapas satelitales modernos.

No obstante, esta riqueza de información también introduce un importante reto computacional, ya que las WSI contienen una cantidad masiva de píxeles y estructuras a múltiples escalas, lo que obliga a diseñar estrategias de procesamiento eficientes para poder explotarlas con modelos de aprendizaje automático. Esta dificultad será especialmente relevante en la metodología de este trabajo, donde será necesario transformar dichas imágenes en subimágenes más pequeñas y extrapolar el conocimiento a la imagen completa.

1.2.2. Química del color: H&E

La digitalización (WSI) permite la visualización remota del tejido, pero la imagen digital depende de la preparación física de la muestra. El tejido humano en su estado natural es prácticamente **transparente** a la luz del microscopio, sin tener el contraste

necesario para distinguir las estructuras celulares. Por ello, la tinción química es un paso obligatorio.

Aunque existen múltiples técnicas, la tinción con **Hematoxilina y Eosina (H&E)** ha sido la principal para el diagnóstico en histopatología, siendo la técnica utilizada en la generación del dataset utilizado en este proyecto, BRACS [8].

La tinción H&E se basa en una reacción ácido-base que genera un contraste bicromático (tonos rosas y morados) [9] [10] [11]:

- **Hematoxilina (componente nuclear):** No es un tinte por sí mismo hasta que se oxida en *hemateína*. Se utiliza generalmente junto con un **mordiente** (sales de aluminio) que le da una carga positiva. Esto le permite unirse por afinidad electrostática a los componentes **aniónicos** (cargados negativamente) de la célula, principalmente los ácidos nucleicos (ADN/ARN) del núcleo. Tras un proceso de “azulado”, tiñe los núcleos de un color **azul oscuro o violeta** intenso.
- **Eosina (componente citoplasmático):** Actúa como contratinción. Es un colorante **ácido** (carga negativa) que tiene afinidad por los componentes **cationicos** (cargados positivamente) del tejido, como las proteínas del citoplasma, el colágeno y las fibras musculares. Genera tonalidades en **rosa y rojo**, permitiendo distinguir el estroma y el cuerpo celular del núcleo.

A pesar de ser un protocolo estandarizado, la apariencia final de la H&E es bastante sensible a factores externos como el tiempo de inmersión, la concentración química de los reactivos (que se oxidan con el tiempo) o el fabricante del escáner.

Esta variabilidad provoca que dos imágenes del mismo tejido, procesadas en laboratorios diferentes, puedan tener distribuciones de color distintas. Esta inconsistencia cromática es uno de los principales problemas para los sistemas de Deep Learning, justificando la necesidad de aplicar técnicas de preprocesamiento para la normalización del color.

Por tanto, la variabilidad cromática no debe entenderse como un simple detalle visual, sino como una fuente real de cambio de dominio entre muestras.

1.3. Motivación: IA en el diagnóstico médico

La integración de herramientas computacionales en el flujo de trabajo clínico ha dado lugar a los sistemas de **Diagnóstico Asistido por Ordenador (CAD)**. El objetivo de estos sistemas no es sustituir al especialista, sino actuar como una herramienta de ayuda que ayude a mitigar las limitaciones y sesgos de los humanos.

1.3.1. El sesgo humano y la variabilidad inter-observador

El análisis histopatológico tradicional es una tarea frecuentemente sujeta a la subjetividad. Diversos estudios han cuantificado esta discrepancia utilizando el **índice Kappa**

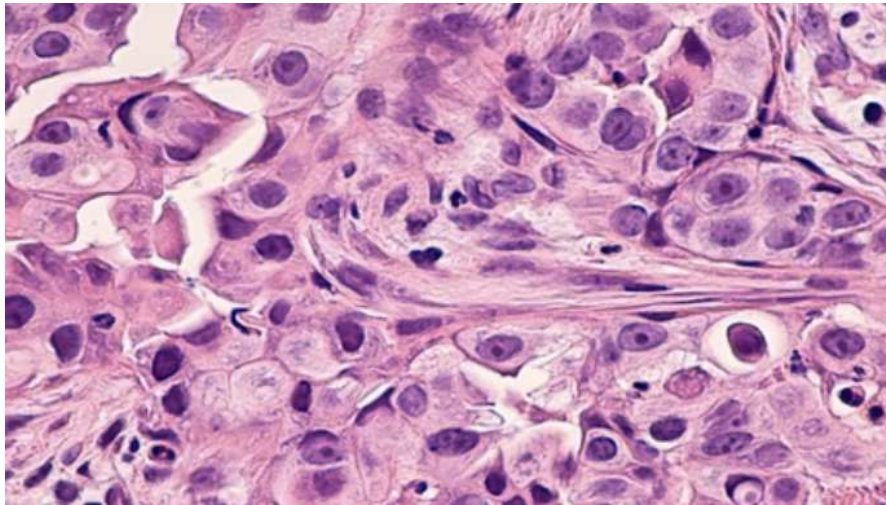


Figura 1.6: Ejemplo de tinción H&E [12]

de Cohen (κ), una métrica estadística que evalúa la coincidencia entre dos evaluadores eliminando la probabilidad de acierto por azar (donde $\kappa = 1$ implica acuerdo total y $\kappa = 0$ acuerdo nulo).

Estudios fundamentales como el de Veta, Van Diest y Williams [13] analizaron la concordancia entre patólogos expertos en tareas críticas como el conteo de la agresividad tumoral. Los resultados revelaron valores de Kappa que demuestran que incluso entre especialistas existe una importante **variabilidad inter-observador**. Esta falta de concordancia justifica aún más la necesidad de sistemas que ofrezcan una ayuda estandarizada y determinista.

1.3.2. Evolución: De los sistemas expertos al Deep Learning

Históricamente, los primeros sistemas CAD se basaban en la ingeniería de características manual (*hand-crafted features*). Los ingenieros debían programar explícitamente reglas *if-else-if* para dar diagnósticos. Este enfoque era rígido y dependía casi siempre del conocimiento del especialista [14].

Sin embargo, la última década ha presenciado un cambio de paradigma radical con la revolución del **Deep Learning**. Tal y como recogen numerosos estudios e investigaciones, el cambio hacia algoritmos de aprendizaje “end-to-end”¹ ha permitido que los sistemas aprendan automáticamente las características relevantes directamente de los datos, sin intervención ni sesgo humano para definir reglas [14].

¹Se denomina aprendizaje *end-to-end* al enfoque en el que un mismo modelo aprende de forma conjunta todo el proceso, desde la entrada bruta hasta la salida final, sin necesidad de definir manualmente las características intermedias.

No obstante, es importante destacar que, aunque los modelos aprendan directamente de los datos, eso no implica que todas las arquitecturas respondan igual ante los retos propios de la histopatología. La naturaleza de las WSI, la variabilidad de tinción y la similitud entre lesiones intermedias hacen necesario analizar con detalle qué familias de modelos se adaptan mejor al problema que estemos enfrentando.

1.3.3. Distintos tipos de Deep Learning

A pesar de los éxitos del *Deep Learning*, el procesamiento de imágenes histológicas presenta una dificultad persistente: la extrema **variabilidad visual**. Factores como el grosor del corte del tejido o los reactivos de tinción confunden a los modelos tradicionales basados en Redes Neuronales Convolucionales (CNNs), limitando a veces su fiabilidad en las predicciones.

Para mitigar esto, la solución ya no pasa únicamente por aplicar algoritmos de **normalización de color** y aumento de datos, sino por valorar la propia arquitectura del modelo. La reciente introducción de los ***Vision Transformers* (ViTs)** y los **modelos fundacionales preentrenados en patología** busca precisamente romper esta limitación, prometiendo sistemas más sólidos frente a la variabilidad que las CNNs clásicas no podían ofrecer.

En este escenario, sigue abierta la cuestión de determinar hasta qué punto las mejores arquitecturas se traducen en ventajas reales cuando se aplican a un problema concreto, reducido y altamente variable. Este trabajo se sitúa precisamente en ese punto, planteando una comparación sistemática entre modelos convolucionales, Vision Transformers y modelos fundacionales, junto con el posterior análisis de distintas estrategias de pre-procesamiento sobre su capacidad de generalización.

1.4. Objetivos del proyecto

El objetivo general de este Trabajo Fin de Grado es:

- **OBJ1:** Desarrollar y evaluar un sistema de diagnóstico asistido por ordenador basado en *Deep Learning* para la clasificación automática de lesiones mamarias, analizando tanto el rendimiento de modelos fundacionales como el efecto de distintas estrategias de reducción, agregación e interpretación de la incertidumbre.

Este objetivo general se descompone en los siguientes **objetivos específicos principales**, que son el núcleo del presente proyecto:

- **OBJ1.1. Estudiar** el estado del arte en clasificación histopatológica de mama con técnicas de Deep Learning, prestando especial atención a la evolución desde arquitecturas CNN hasta *Vision Transformers* y modelos fundacionales aplicados a patología digital.

- **OBJ1.2. Preparar** un flujo de trabajo completo para el procesamiento del dataset BRACS, incluyendo la gestión de imágenes histopatológicas, la organización de regiones de interés y la generación de patches para el entrenamiento y evaluación de los modelos.
- **OBJ1.3. Implementar y evaluar** los mejores modelos para la clasificación de lesiones mamarias, utilizando sus embeddings como base para construir clasificadores.
- **OBJ1.4. Analizar** el impacto de distintas estrategias de reducción del conjunto de patches de entrenamiento, con el fin de comprobar si es posible mantener o mejorar el rendimiento final.
- **OBJ1.5. Diseñar y comparar** distintas reglas de agregación desde el nivel de *patch* a ROI, evaluando cuál de ellas representa mejor la decisión final del sistema en un contexto más próximo al uso real.
- **OBJ1.6. Incorporar** mecanismos de análisis de incertidumbre estudiando tanto reglas de abstención como casos dudosos, con el propósito de identificar situaciones en las que el modelo requiera revisión y no deba emitir una decisión automática cerrada.

A medida que se avanzó en el desarrollo del proyecto y una vez cubierto el núcleo principal del trabajo, se abordaron una serie de **objetivos complementarios**, orientados a profundizar en la interpretación de los resultados y a ampliar el alcance del estudio con análisis adicionales:

- **OBJ1.7. Profundizar** en la interpretación del problema mediante análisis complementarios del espacio latente y de la estructura del dataset, con el objetivo de determinar hasta qué punto las confusiones observadas responden a limitaciones del modelo o a una ambigüedad intrínseca de los datos.
- **OBJ1.8. Comparar** rigurosamente los resultados obtenidos mediante métricas adecuadas de clasificación multiclase (*accuracy*, *precision*, *recall*, *F1-score*), matrices de confusión y análisis cualitativos, identificando las configuraciones más sólidas y extrayendo conclusiones sobre la viabilidad del enfoque propuesto.

1.5. Estructura de la memoria

En el *capítulo 1* se han presentado el contexto y motivación de este trabajo, así como los principales objetivos que nos hemos propuesto para abordar la problemática asociada al diagnóstico asistido por ordenador en histopatología de cáncer de mama. El resto de la memoria se ha estructurado como sigue:

- En el *capítulo 2* se hace un análisis del estado del arte en relación a las principales tecnologías e investigaciones de Inteligencia Artificial aplicadas en el dominio de nuestro problema.
- En el *capítulo 3* se presenta el dataset BRACS utilizado en este trabajo, detallando su composición, las clases histológicas definidas y la distribución de datos para entrenamiento, validación y prueba.
- En el *capítulo 4* se expone la metodología y la planificación seguidas, reflejando la forma de trabajar durante todo el proyecto.
- En el *capítulo 5* se muestran los experimentos realizados y los resultados obtenidos.
- En el *capítulo 6* se hace un análisis del trabajo extra realizado, discutiendo los resultados obtenidos y las limitaciones de mejora.
- En el *capítulo 7* se presentan las conclusiones derivadas del trabajo realizado, así como las líneas de investigación futura para extender y mejorar el sistema desarrollado.
- Finalmente, se incluyen las referencias bibliográficas utilizadas en este trabajo y el anexo con el repositorio del proyecto.

Capítulo 2

Fundamentos y Estado del Arte

2.1. Inteligencia artificial en Visión por computador

A pesar de la diversidad de definiciones, podemos definir a la **Inteligencia Artificial** como la disciplina dentro de la informática cuyo objetivo es dotar a las máquinas para imitar el razonamiento humano. Estas tecnologías permiten que las máquinas aprendan por sí solas a partir de la experiencia, se adapten a nuevos problemas y realicen tareas humanas como la toma de decisiones, reconocimiento por voz o la visión por computador.

El **Aprendizaje Automático** (*Machine Learning*) es un campo dentro de la IA donde se entrena a las máquinas para que aprendan a través de los datos y hagan predicciones sin necesidad de ser programados explícitamente. Dentro del *Machine Learning* existen distintos tipos de aprendizaje:

- Supervisado: El conjunto de entrenamiento con el que se entrena incluye las soluciones deseadas (etiquetas).
- No supervisado: El conjunto de entrenamiento NO incluye las soluciones, es decir, la máquina aprende 'sin un profesor'.
- Por refuerzo: La máquina aprende a base de prueba y error, recibiendo recompensas o penalizaciones y aprendiendo de estas.

En este proyecto, nos centraremos en el **Aprendizaje Supervisado**.

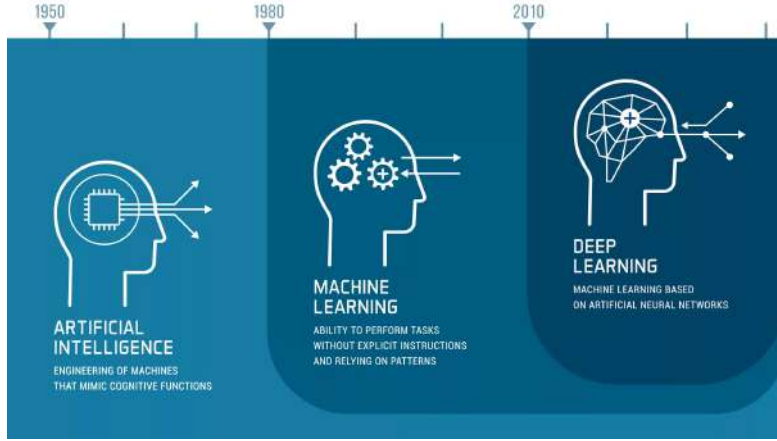


Figura 2.1: IA, ML y DL [15]

2.1.1. Aprendizaje supervisado

Como hemos indicado antes, el Aprendizaje Supervisado es una rama del *Machine Learning* en la que los entrenamientos se hacen con datos que incluyen las etiquetas, es decir, las soluciones correctas. Podríamos explicarlo como cuando le enseñamos a un niño a resolver sumas de matemáticas enseñándole múltiples cuentas junto a sus resultados.

Formalmente, el Aprendizaje Supervisado toma un conjunto de datos de entrada/salida (E/S) denotado como $\mathcal{D}$, compuesto por N pares de observaciones empíricas:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N \quad (2.1)$$

donde:

- $\mathbf{x}_i \in \mathcal{X} \subset \mathbb{R}^n$ representa el vector de características (*features*) del i -ésimo patrón de entrada, compuesto por n atributos: $\mathbf{x}_i = (x_{i,1}, x_{i,2}, \dots, x_{i,n})$.
- $y_i \in \mathcal{Y}$ es la etiqueta o valor verdadero asociado a dicho patrón.

El propósito de un algoritmo de aprendizaje supervisado es deducir una función de mapeo f , parametrizada por un conjunto de pesos θ , de forma que, dado un vector de entrada $\mathbf{x}_i$, sea capaz de calcular una aproximación $\hat{y}_i$ lo más cercana posible a la salida real y_i :

$$\hat{y}_i = f(\mathbf{x}_i; \theta) \quad , \quad \hat{y}_i \approx y_i \quad (2.2)$$

Existen dos tipos de Aprendizaje Supervisado:

- **Regresión / Aproximación funcional:** Se define cuando la variable objetivo es continua, es decir, el modelo intenta predecir un valor numérico real. Un ejemplo sería predecir el porcentaje exacto de celularidad tumoral en una biopsia.

- **Clasificación:** Ocurre cuando la variable objetivo pertenece a un conjunto discreto, categórico y finito de clases, es decir, $y_i \in \mathcal{C} = \{c_1, c_2, \dots, c_K\}$. El objetivo aquí es establecer fronteras de decisión en el espacio $\mathbb{R}^n$ para asignar un nuevo patrón a la categoría correcta. Este es precisamente el tipo de problema de este proyecto, tratándose de un problema de **clasificación multiclase**, donde $K = 7$ (o 3), correspondiente a las siete subclases del dataset BRACS.

Sea cual sea el tipo, el algoritmo de Aprendizaje Automático siempre se va a plantear como un **algoritmo de optimización** para minimizar la función de pérdida del modelo.

Función de pérdida

La función de pérdida (función de error, *loss function*, función de coste o función objetivo) es la medida matemática que cuantifica la diferencia entre los resultados predichos por el modelo y los valores reales (etiquetas) del conjunto de datos. Esta función irá guiando al modelo en el proceso de aprendizaje indicando qué tan bien o mal está prediciendo. Para cada tipo de aprendizaje, existen distintas funciones de pérdida:

- **Error Cuadrático Medio (*Mean Squared Error*, MSE):** Es la métrica por excelencia en los problemas de regresión. El MSE calcula el promedio de las diferencias al cuadrado entre los valores predichos y los valores reales para un conjunto de N muestras. Matemáticamente se define como:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2.3)$$

El uso del cuadrado en esta fórmula tiene dos implicaciones matemáticas. Por un lado, garantiza que la función sea estrictamente convexa, lo que asegura la existencia de un único mínimo global. Por otro lado, al elevar la diferencia al cuadrado, el modelo penaliza de forma cuadrática (no lineal) los errores grandes. Esto obliga a la red a ajustar los pesos más agresivamente para corregir las predicciones más alejadas del valor real. Sin embargo, esta misma propiedad hace que sea mucho más sensible a valores atípicos (*outliers*), ya que un solo error puede dominar el valor total de la pérdida.

- **Entropía Cruzada Binaria (*Binary Cross-Entropy*, BCE):** Se utiliza en problemas de clasificación donde $K = 2$, es decir, donde un evento solo puede pertenecer a la clase positiva (1) o negativa (0). Matemáticamente:

$$BCE = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2.4)$$

Si la clase real es $y_i = 1$, el segundo término se anula y la pérdida depende solo de $-\log(\hat{y}_i)$. Gracias a las propiedades del logaritmo, si la red predice una

probabilidad cercana a 0 para la clase 1 (es decir, está muy segura de un error), la pérdida tiende a infinito, generando un gradiente enorme que corrige drásticamente los pesos.

■ **Entropía Cruzada (*Cross-Entropy*, CE)**

Es la generalización natural de la BCE para problemas con múltiples clases ($K > 2$). Es el estándar para tareas de clasificación. Se define como:

$$CE = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^K y_{i,j} \log(\hat{y}_{i,j}) \quad (2.5)$$

Donde $y_{i,j}$ es una variable indicadora (1 si el ejemplo i pertenece a la clase j , 0 en caso contrario -en formato *One-Hot Encoding*-), y $\hat{y}_{i,j}$ es la probabilidad predicha por la red (generalmente mediante una capa de activación *Softmax*). Al igual que la BCE, castiga severamente al modelo si asigna una probabilidad baja a la clase correcta.

- **Focal Loss:** Una de las mayores limitaciones de la Entropía Cruzada es su sensibilidad a conjuntos de datos fuertemente desbalanceados. Es decir, en nuestro problema, si las clases sanas o benignas son mayoritarias frente a las malignas, la red optimizará su coste simplemente clasificando los ejemplos fáciles, ignorando las patologías raras. Para mitigar esto y no tener que balancear el dataset obligatoriamente, una función de pérdida más avanzada es la *Focal Loss*, que matemáticamente se formula así:

$$FL = -\sum_{i=1}^N \alpha_t (1 - \hat{p}_t)^\gamma \log(\hat{p}_t) \quad (2.6)$$

Donde $\hat{p}_t$ es la probabilidad predicha para la clase real. Esta función está diseñada para tareas de clasificación donde existe un desequilibrio de clases severo. Se introduce un factor de modulación $(1 - \hat{p}_t)^\gamma$. Si un ejemplo es “fácil” y la red lo clasifica correctamente con alta confianza ($\hat{p}_t \approx 1$), el factor de modulación tiende a 0, reduciendo la contribución de ese ejemplo a la pérdida total. El hiperparámetro $\gamma \geq 0$ ajusta la fuerza de esta reducción, mientras que α_t actúa como un factor de balanceo por clase. Esto obliga al modelo a concentrar su capacidad de aprendizaje en los ejemplos difíciles o minoritarios.

En definitiva, en problemas como el tratado en este proyecto, donde existe un cierto desequilibrio entre clases y una elevada similitud entre categorías intermedias, la elección de la función de pérdida puede influir directamente en la capacidad del modelo para distinguir correctamente las lesiones más complejas.

Descenso de Gradiente y Learning Rate

Una vez definida la métrica que cuantifica el error del modelo, nos surge la pregunta fundamental del aprendizaje: ¿cómo modificamos los pesos de la red para que el error se vaya reduciendo al mínimo?

La respuesta la encontramos en los algoritmos de optimización, siendo el **Descenso de Gradiente** el motor matemático que impulsa el aprendizaje.

El Descenso de Gradiente es un algoritmo de optimización iterativo basado en un principio fundamental del cálculo: el gradiente de una función en un punto dado apunta siempre en la dirección de máximo crecimiento de esa función.

Si imaginamos la función de coste como una montaña con numerosos picos y al modelo como un escalador, el gradiente le indica al escalador hacia dónde sube la pendiente más pronunciada para ir en dirección opuesta a esa pendiente. Es decir, como nuestro objetivo es minimizar el error (bajar al valle), el algoritmo nos dice hacia dónde debemos dar el paso para ir en **dirección opuesta al gradiente**.

Formalmente, sea θ el vector que contiene todos los parámetros (pesos y sesgos) del modelo. En cada iteración t , el vector de gradientes $\nabla J(\theta_t)$ se calcula evaluando las derivadas parciales de la función de coste respecto a cada uno de los parámetros:

$$\nabla J(\theta_t) = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_m} \right]^T \quad (2.7)$$

Una vez obtenido este vector direccional, los pesos de la red se actualizan mediante la siguiente ecuación iterativa:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t) \quad (2.8)$$

Donde θ_{t+1} hace referencia a los nuevos pesos y θ_t a los anteriores, y donde η es un hiperparámetro crítico conocido como **Tasa de Aprendizaje** o *Learning Rate*. Este determina el tamaño del “paso” que da el algoritmo en cada iteración. La elección de η es una de las más importantes que determina si nuestro modelo converge mejor o peor:

- Si η es **demasiado pequeño**, el descenso será muy lento, incrementando el coste computacional y teniendo el riesgo de que el modelo quede atrapado en un *mínimo local* o en una zona de meseta.
- Si η es **demasiado grande**, el algoritmo dará pasos gigantes, pudiendo sobrepasar el mínimo global que buscamos o incluso rebotar en las paredes del “valle” de error y causar la divergencia matemática del modelo (el error tiende a infinito).

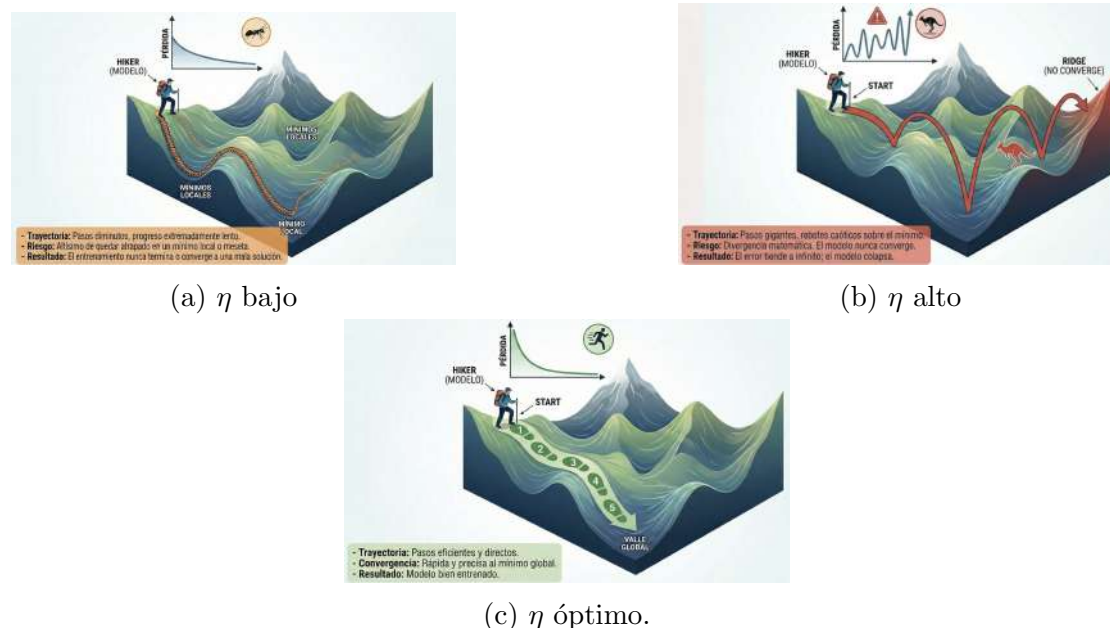


Figura 2.2: Comportamiento del Descenso de Gradiente según el LR (η)

El cálculo del gradiente $\nabla J(\theta_t)$ sobre la función de coste depende del número de muestras utilizadas. En función de la cantidad de datos que procesa el algoritmo en cada paso, existen tres variantes principales de descenso del gradiente:

- **Batch Gradient Descent (BGD):** Calcula el gradiente utilizando todo el conjunto de entrenamiento (N muestras) en cada iteración. Aunque garantiza un camino estable hacia el mínimo, es computacionalmente muy costoso, es decir, inviable para conjuntos de datos masivos, ya que requeriría cargar toda la base de datos en memoria para dar un solo “paso”.
- **Stochastic Gradient Descent (SGD):** Es el contrario al anterior. Toma una única muestra aleatoria ($N = 1$) para calcular el gradiente y actualizar los pesos. Es extremadamente rápido y consume poca memoria, pero la dirección del gradiente es muy ruidosa y comete más errores, lo que hace que la convergencia sea muy inestable.
- **Mini-batch Gradient Descent:** Es la mejor solución y el estándar en *Deep Learning*. Se selecciona un subconjunto aleatorio de muestras *batch size* (típicamente entre 16 y 256) para evaluar el gradiente. Esta variante aprovecha la vectorización y el cálculo en paralelo de las GPUs, logrando una convergencia más rápida y suave que el SGD puro, a la vez que introduce el ruido suficiente para ayudar al modelo a escapar de mínimos locales.

Generalización y Sobreajuste

Aunque los distintos algoritmos de optimización garantizan la convergencia hacia un mínimo en la función de pérdida, esto no significa que el modelo funcione mejor, ya que no queremos que memorice los datos, sino que busquemos siempre la mejor **generalización** posible.

La práctica exige la división del conjunto de datos con el que vamos a trabajar en 3 subconjuntos:

- **Entrenamiento (60-80 %):** Es el más grande de todos. Es el conjunto con el que se entrena el modelo, es decir, con el que se ajustan los pesos.
- **Validación (10-20 %):** Se utiliza para evaluar el rendimiento del modelo durante la fase de entrenamiento e ir ajustando hiperparámetros (ej. *Learning Rate*).
- **Test (10-20 %):** Se utiliza después de que el modelo haya sido entrenado y validado. Da una evaluación imparcial del rendimiento final del modelo con datos completamente desconocidos, es decir, pone a prueba al modelo de cara a cómo funcionará en el mundo real con datos nuevos. Nunca debe usarse para entrenar o ajustar parámetros, ya que esto provocaría lo que se conoce como *Data Snoo-ping*, contaminando los datos y provocando que las métricas de rendimiento sean optimistas pero no reales y el modelo no generalice bien ante datos nuevos.

Respecto a la generalización del modelo con estos tres conjuntos de datos, pueden ocurrir dos fenómenos fatales:

- **Sobreajuste (*Overfitting*):** Ocurre cuando el modelo “memoriza” los datos de entrenamiento y el ruido, funcionando perfectamente con ellos pero mal con datos nuevos, es decir, en *valid* y *test*.
- **Subajuste (*Underfitting*):** Sucede cuando el modelo es demasiado simple para captar la estructura fundamental de los datos, funcionando mal tanto en el entrenamiento como en nuevos datos.

Estos dos fenómenos no son fallos aleatorios del entrenamiento, sino que muestran una propiedad estadística fundamental conocida como el *Bias-Variance Trade-off* (*Dilema Sesgo-Varianza*).

Matemáticamente, el error de generalización de cualquier modelo se puede descomponer en tres componentes: el error irreducible (ruido inherente al propio problema), el sesgo al cuadrado y la varianza. Para que un modelo sea exitoso, debemos comprender y equilibrar los dos últimos:

- **Sesgo (*bias*):** Es el error introducido para aproximar un problema del mundo real (que suele ser muy complejo y no lineal) mediante un modelo demasiado simple. Un modelo con un sesgo alto hace suposiciones rígidas sobre los datos y es incapaz de capturar los patrones subyacentes, lo que lleva al *Underfitting*.

- **Varianza (*variance*):** Cuantifica la sensibilidad del modelo ante pequeños cambios en el conjunto de datos de entrenamiento. Un modelo con alta varianza cambiará drásticamente sus predicciones si se le entrena con un subconjunto de datos algo distinto. Al tener una capacidad de representación excesiva, el modelo termina “memorizando” el ruido, provocando el *Overfitting*.

El *trade-off* surge porque es imposible minimizar ambas simultáneamente sin aumentar la cantidad de datos. Al incrementar la complejidad de la red (por ejemplo, añadiendo más capas neuronales o parámetros), el sesgo disminuye porque el modelo es más flexible; sin embargo, esta misma flexibilidad dispara la varianza.

Por tanto, el objetivo principal aquí es encontrar el **punto óptimo** en la curva de validación: aquel donde el modelo tiene la suficiente capacidad para aprender la tarea (bajo bias) pero está lo suficientemente regularizado como para no memorizar el ruido (baja varianza).

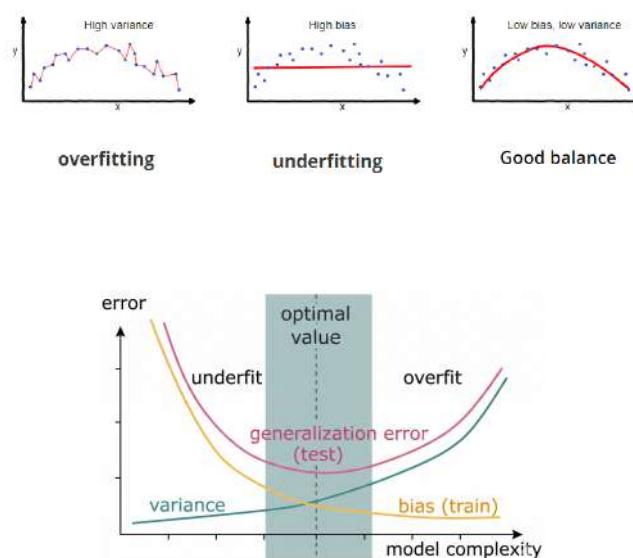


Figura 2.3: Bias-Variance Trade-Off

2.1.2. Aprendizaje profundo (*Deep Learning*)

Mientras que el Aprendizaje Supervisado define el “qué” (minimizar un error generalizando a partir de datos), el *Deep Learning* define el “cómo”. Este subcampo se fundamenta en las Redes Neuronales Artificiales (ANNs), modelos computacionales inspirados en la biología.

La base de cualquier arquitectura profunda es la neurona artificial o **Perceptrón**. Matemáticamente, una neurona es una unidad de procesamiento que realiza una combinación lineal sobre sus datos de entrada.

Dada una instancia de entrada representada por el vector $\mathbf{x} = [x_1, x_2, \dots, x_n]$, la neurona calcula una salida z aplicando la siguiente ecuación:

$$z = \sum_{i=1}^n w_i x_i + b \quad (2.9)$$

Donde los parámetros aprendibles del modelo son:

- **Pesos ($\mathbf{w}$):** Un vector de parámetros $\mathbf{w} = [w_1, w_2, \dots, w_n]$, donde cada w_i determina la importancia de cada característica de entrada x_i . Durante el entrenamiento, si una característica es crucial, la red aumentará su peso y, si es irrelevante, lo aproximará a cero.
- **Sesgo ($Bias$, b):** Es un valor escalar independiente de la entrada. Se añade para desplazar la frontera de decisión originada por los pesos, permitiendo que el modelo se ajuste correctamente y no sea rígido, es decir, que no tenga que pasar si o si por el origen $(0,0)$.

El resultado z (*logit*) representa la suma ponderada neta de la neurona (en la figura 2.4 $y = z$).

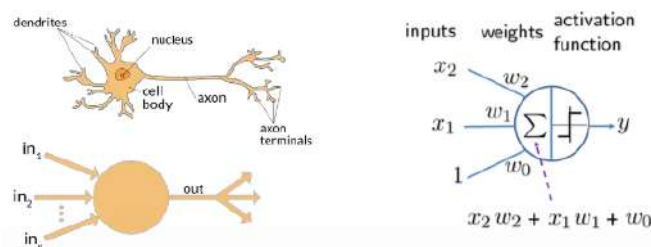


Figura 2.4: Modelo de una neurona artificial/perceptrón [16]

El verdadero potencial del *Deep Learning* consiste en juntar miles de estas neuronas en estructuras interconectadas, dando lugar al **Perceptrón Multicapa** (*Multilayer Perceptron*, MLP).

La arquitectura estándar de un MLP se organiza en tres tipos de capas estructuradas de forma secuencial (véase Figura 2.5):

1. **Capa de Entrada (*Input Layer*):** Es el punto de recepción de los datos. Su tamaño coincide exactamente con la dimensionalidad del vector de características $\mathbf{x}$. En esta capa no hay procesamiento ni pesos, solo transmisión de valores.
2. **Capas Ocultas (*Hidden Layers*):** Capas que se encargan del cálculo de la red. Se denominan *ocultas* porque sus salidas no se observan directamente en el conjunto de entrenamiento. En un MLP estándar, todas las neuronas de una capa l están conectadas con todas las neuronas de la capa anterior $l - 1$ (conexiones densas). Es aquí donde entra el concepto de “Profundidad” (*Deep*): añadir más

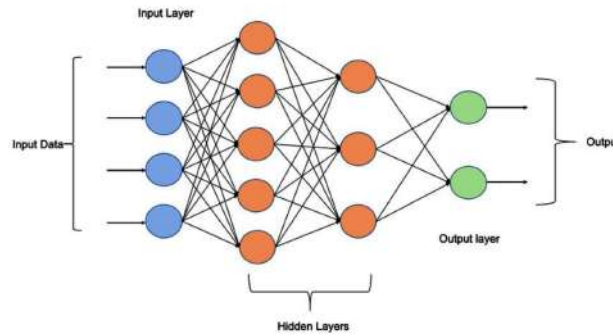


Figura 2.5: Arquitectura típica de un Perceptrón Multicapa (MLP) [17]

capas ocultas permite a la red aprender una jerarquía de características, desde patrones simples en las primeras capas hasta características más complejas en las últimas.

3. **Capa de Salida (*Output Layer*):** Es la última capa de la red y depende estrictamente del problema a resolver. Tiene tantas neuronas de salida como logit nos interese en nuestro problema. Por ejemplo, en este proyecto esta capa estaría formada por 7 neuronas, cada una emitiendo el *logit* asociado a una clase del dataset BRACS.

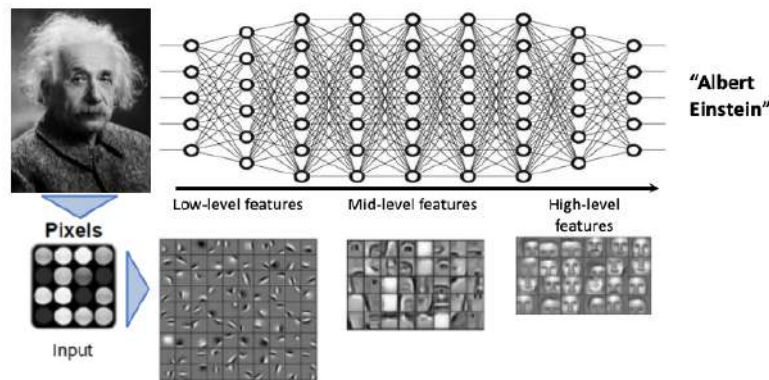


Figura 2.6: Ejemplo de predicción de Albert Einstein

Funciones de Activación

Si la suma ponderada $z = \mathbf{w}^T \mathbf{x} + b$ se transmitiera directamente a la siguiente capa, sin importar cuántas capas ocultas tuviera el Perceptrón Multicapa, matemáticamente toda la red colapsaría en una regresión lineal simple. Esto haría imposible que el modelo aprendiese las fronteras de decisión complejas necesarias para distinguir.

Para solucionar esto, la salida de cada neurona se pasa a través de una **Función de Activación** no lineal, denotada como $\sigma(z)$.

Históricamente, las primeras redes neuronales utilizaron la función **Sigmoide**, debido a su interpretación biológica (una neurona se activa o no se activa) y probabilística, ya que comprime cualquier valor real en el rango $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.10)$$

No obstante, se observó que la Sigmoide presentaba un problema crítico en redes profundas: el **Desvanecimiento del Gradiente** (*Vanishing Gradient*). Cuando los valores de entrada son muy altos o muy bajos, la curva se vuelve plana (su derivada tiende a cero). Al aplicar la regla de la cadena en el *Backpropagation*, estos gradientes cercanos a cero se multiplican capa tras capa, provocando que las primeras capas de la red dejen de aprender o lo hagan muy lentamente, limitando la capacidad de la red para mejorar.

Para intentar poner solución a este problema, se introdujo la **Tangente Hiperbólica** (**Tanh**), que comprime los valores en el rango $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (2.11)$$

Al estar centrada en cero, el algoritmo de optimización llegaba a convergir más rápidamente que con la Sigmoide. Sin embargo, en los extremos, la función sigue saturándose y el gradiente continúa desvaneciéndose en arquitecturas muy profundas.

La verdadera revolución que permitió entrenar arquitecturas con un número enorme de capas fue la función de activación que se usa en la mayoría de problemas de *Deep Learning*: la **ReLU** (*Rectified Linear Unit*), cuya fórmula matemática es muy simple:

$$f(z) = \max(0, z) \quad (2.12)$$

Esta función deja pasar los valores positivos (derivada constante igual a 1) y anula los negativos (derivada igual a 0). Esto tiene tres ventajas:

- Resuelve el problema del desvanecimiento del gradiente para valores positivos.
- Su coste computacional es casi nulo en comparación con el cálculo de exponenciales.
- Al apagar ciertas neuronas por completo, obliga al modelo a aprender representaciones más eficientes y lo hace más robusto.

El único problema de ReLU es el problema de las *neuronas muertas* (*Dying ReLU*): si un gran gradiente negativo hace que los pesos se ajusten de forma que la entrada a la ReLU sea siempre negativa, esa neurona jamás volverá a activarse ni a aprender. Para ello, existen variantes como **Leaky ReLU**, que introduce una pequeña pendiente (e.g., 0,01) para valores negativos, permitiendo que el gradiente siga fluyendo.

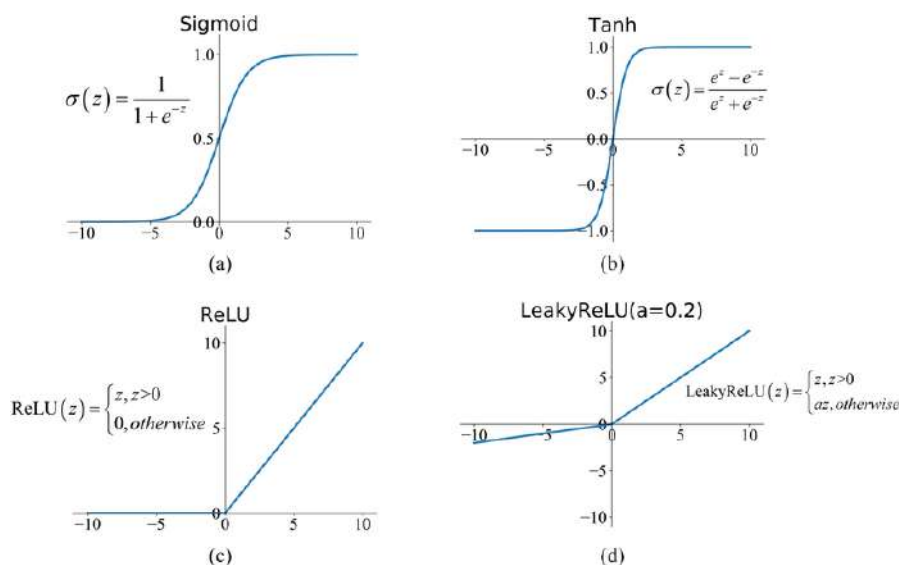


Figura 2.7: Funciones de activación más comunes en *Deep Learning* [18]

Mientras que ReLU domina en las **capas ocultas**, la capa de salida requiere un tratamiento especial dependiendo de la tarea. Para problemas de clasificación multiclase, la última capa debe aplicar la función **Softmax**.

Esta función actúa como una generalización de la sigmoide para vectores. Esta toma el vector de salidas de la red (los *logits* z) y los transforma en una distribución de probabilidad normalizada, asegurando que todos los valores estén entre 0 y 1, y que su suma total sea exactamente 1:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (2.13)$$

Donde K es el número total de clases. Esta salida probabilística es la que posteriormente evalúa la función de pérdida durante el entrenamiento.

Backpropagation

Hemos explicado cómo la red realiza predicciones (con multiplicaciones de pesos y funciones de activación) y cómo medimos su fallo (mediante la función de pérdida). Además, sabemos que el Descenso de Gradiente es el encargado de actualizar los pesos usando derivadas. Sin embargo, ¿cómo calculamos las derivadas de la función de pérdida con respecto a millones de pesos individuales distribuidos en decenas de capas?

El algoritmo fundamental que permite esto, y que hace que las redes neuronales profundas aprendan es el **Backpropagation** (*Retropropagación del Error*). En términos sencillos, es el método que calcula cuánto contribuyó cada peso de la red al error final de una predicción, para luego ajustarlos y mejorar la red.

Si tenemos una red simple donde la entrada x pasa por un peso w para obtener z , luego por una activación para obtener a , y finalmente se evalúa en una pérdida L , la función completa es $L(a(z(w, x)))$.

El objetivo es encontrar el gradiente del error respecto a un peso específico de las primeras capas: $\frac{\partial L}{\partial w}$. El algoritmo de *Backpropagation* aplica de forma iterativa y eficiente la **Regla de la Cadena** del cálculo diferencial:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (2.14)$$

Este cálculo se realiza recursivamente de atrás hacia delante (de la capa de salida hasta la capa de entrada). Al reutilizar las derivadas parciales ya calculadas en las capas superiores, el algoritmo evita recálculos redundantes, teniendo un problema lineal respecto al número de conexiones de la red.

Para dejar claro cuándo se activa y se desactiva el *Backpropagation* lo vamos a explicar diferenciando las dos fases:

- **Fase de Entrenamiento:** Es el único momento en el que el *Backpropagation* se **activa**. El modelo realiza una pasada hacia adelante (*Forward Pass*) para predecir, almacena en memoria las activaciones, calcula la pérdida y luego realiza la pasada hacia atrás (*Backward Pass*) para calcular los gradientes y actualizar los pesos.
- **Fase de Inferencia:** Una vez el modelo está entrenado y se despliega para diagnosticar nuevas imágenes, el *Backpropagation* se **desactiva** por completo. La red solo ejecuta el *Forward Pass*, lo que reduce el coste computacional al no necesitar almacenar valores para calcular derivadas.

Métricas de evaluación

Una vez entendido cómo aprende una red neuronal, resulta necesario definir **cómo se evalúa el rendimiento**. En problemas de clasificación, el proceso de entrenamiento permite ajustar los pesos del modelo para reducir el error, pero es mediante distintas **métricas de evaluación** como se cuantifica hasta qué punto esas predicciones son correctas y útiles.

Existen numerosas métricas de interés, entre ellas la **accuracy**, la **precision**, el **recall**, el **F1-score** o las **matrices de confusión**. Cada una aporta una perspectiva distinta del comportamiento del clasificador, por lo que su elección depende del tipo de problema y de la naturaleza de los datos.

En este trabajo se prestará especial atención a dos métricas principales: la **accuracy** y el **F1-score**. La **accuracy** o exactitud, mide la proporción total de muestras correctamente clasificadas respecto al número total de ejemplos evaluados. Se trata de una métrica sencilla e intuitiva, ya que refleja el porcentaje global de aciertos del sistema. Sin embargo, puede resultar algo engañosa cuando existe cierto desbalanceo entre clases, ya que un valor alto de **accuracy** puede estar ocultando un mal comportamiento sobre categorías minoritarias.

Por su parte, el **F1-score** combina en una única medida la *precision* y el *recall*, proporcionando una valoración más equilibrada del rendimiento del clasificador. En problemas multiclase como el abordado en este trabajo, resulta especialmente útil emplear el **F1-score macro**, ya que calcula la media del rendimiento entre clases otorgando el mismo peso a todas ellas, independientemente de su frecuencia. Esto permite analizar de forma más justa el comportamiento del sistema cuando algunas categorías presentan menos muestras o una mayor dificultad de discriminación.

En consecuencia, a lo largo del proyecto se utilizará el **F1-score macro** como medida principal, pero se prestará especial atención al **accuracy**, ya que sigue siendo una métrica muy robusta aunque haya clases especialmente minoritarias.

2.2. Arquitecturas de Deep Learning

2.2.1. Redes Convolucionales (CNNs)

Una Red Neuronal Convolutiva o CNN (*Convolutional Neural Network*) es una red neuronal con al menos una convolución en una de sus capas. Está diseñada para problemas donde la información se muestra como una matriz (ej. imagen).

A diferencia del Perceptrón Multicapa, que conecta todas las neuronas con todos los píxeles (conexiones densas), las CNNs reducen drásticamente el número de parámetros y mejoran la generalización por:

- **Conectividad Local:** Cada neurona no procesa la imagen entera, sino únicamente una pequeña región espacial contigua.
- **Pesos Compartidos:** Los mismos pesos se aplican en diferentes partes de la imagen, consiguiendo que la red sea *invariante a la traslación* (es capaz de detectar un objeto sin depender de si está en el centro o en una esquina de la imagen).

Para lograr transformar los píxeles de una imagen bruta en una predicción final, una CNN se compone casi siempre de una secuencia de tres tipos principales de capas:

1. **Capas Convolucionales:** Son el extractor de características de la red. Mediante el uso de filtros aprendibles, estas capas escanean la imagen buscando patrones específicos. Las primeras capas detectan cosas más genéricas (bordes y colores), mientras que las más profundas combinan esta información para detectar estructuras complejas.
2. **Capas de Agrupación (*Pooling Layers*):** Se intercalan entre las capas convolucionales. Su función es submuestrear la imagen (reducir sus dimensiones espaciales), eliminando información redundante, disminuyendo el coste computacional y haciendo que la red sea más robusta.
3. **Capas Totalmente Conectadas (*Fully Connected Layers*):** Se sitúan siempre al final de la arquitectura. Una vez que las capas anteriores han extraído las

características de la imagen, estos mapas se “aplanan” en un vector unidimensional y se pasan por un Perceptrón Multicapa clásico para realizar la clasificación final probabilística.

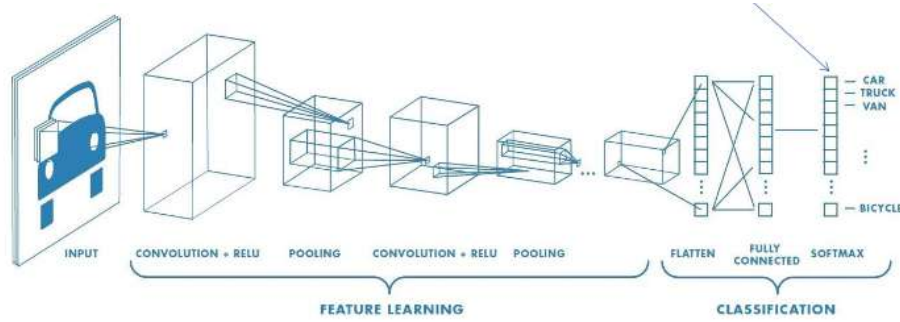


Figura 2.8: Arquitectura general de una CNN [19]

Capas convolucionales

Las bases de estas capas vienen del filtrado de imágenes (*Image Filtering*). La operación consiste en reemplazar el valor de cada píxel por una combinación lineal de los valores de su vecindad local. Para ello, existe un **filtro o kernel** (K), que es una pequeña matriz de pesos que actúa como una ventana deslizante sobre la imagen original (I).

Para un filtro de dimensiones $(2k+1) \times (2k+1)$, la operación donde se multiplican los pesos del filtro por los píxeles superpuestos de la imagen se denomina matemáticamente **Correlación Cruzada** ($\otimes$). El valor resultante en la posición (i, j) se define como:

$$(I \otimes K)(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k K(u, v) I(i + u, j + v) \quad (2.15)$$

Sin embargo, en el procesamiento de imágenes, la **Convolución** ($*$) es una operación igual, pero el filtro K es volteado tanto horizontal como verticalmente antes de realizar el producto punto a punto. Matemáticamente, esto se traduce en una resta en los índices espaciales de la imagen:

$$(I * K)(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k K(u, v) I(i - u, j - v) \quad (2.16)$$

En la práctica, casi todas las librerías modernas (como *PyTorch* o *TensorFlow*) implementan sus capas convolucionales bajo el nombre de **Conv2d**, cuando en realidad la operación que ejecutan es la **Correlación Cruzada**, no la Convolución como tal.

Esto se debe a que en una red neuronal artificial, los valores del filtro K no son fijos, sino que **son aprendidos automáticamente** (es lo que realmente aprende la red).

Dado que la red va a optimizar los pesos para extraer la mejor característica posible (un borde, una textura, etc.), aprender el filtro del derecho o del revés genera el mismo mapa de representación final. Por tanto, no rotar el filtro elimina operaciones innecesarias en memoria, ahorrando valiosos ciclos computacionales durante la generación de los **Mapas de Características** (*Feature Maps*, salidas resultantes de aplicar un filtro de convolución a una imagen de entrada o a una capa anterior).

Al aplicar el kernel deslizante sobre una imagen, surge un problema fundamental en los límites de esta. Por ejemplo, si el centro de un filtro de 3×3 se sitúa exactamente sobre el píxel de la esquina superior izquierda de una imagen, parte del filtro quedará fuera de la matriz, sin valores con los que multiplicarse.

Si la operación se realiza únicamente a aquellas posiciones donde el filtro cabe completamente dentro de la imagen, el mapa de características resultante será más pequeño que la imagen original, lo que en redes profundas se traduce en que tras unas pocas capas, la imagen desaparecería por completo.

Para solucionar esto, se introduce el concepto de **Padding**, que consiste en añadir píxeles alrededor de la imagen original antes de aplicar el filtro. En la práctica, aunque dentro del padding existen varios tipos (*reflect*, *replicate*,...), existen dos modalidades principales:

- **Valid Padding (Sin relleno):** No se añade ningún borde a la imagen. El filtro solo se aplica donde intersecta totalmente con los píxeles reales. Como resultado, la dimensión de salida se reduce. Si la imagen es de $N \times N$ y el filtro es de $F \times F$, la salida será de tamaño $(N - F + 1) \times (N - F + 1)$.
- **Same Padding (ej. Zero Padding):** Es el método más utilizado en arquitecturas modernas. Consiste en envolver la imagen con capas de píxeles del mismo valor (lo más habitual es cero, aunque pueden ser otros valores de relleno) en una cantidad exacta para que **el mapa de características de salida mantenga exactamente las mismas dimensiones espaciales que la imagen de entrada**. Para un filtro impar de tamaño F , se debe añadir un borde de tamaño $P = \frac{F-1}{2}$ en todos los lados.

El uso del *Zero Padding* no solo previene la reducción geométrica de los datos en redes profundas, sino que además evita que la información situada en los bordes de la imagen se ignore rápidamente en las primeras etapas de procesamiento.

Por otro lado, un concepto base a tener en cuenta es el **Stride** (paso o avance), hiperparámetro que define la cantidad exacta de píxeles que el filtro se desplaza sobre la matriz de entrada en cada iteración.

Normalmente, la ventana deslizante avanza de forma continua píxel a píxel (*stride* = 1), produciendo un mapa de características denso donde se evalúan y solapan todas las posiciones posibles.

Sin embargo, si se incrementa este valor (por ejemplo, aplicando un *stride* = 2), el filtro comienza a dar saltos sobre la imagen, ignorando las posiciones intermedias. Esto consigue reducir (a la mitad, en el caso de usar 2) las dimensiones espaciales del mapa

de características resultante. De este modo, el *stride* permite a los diseñadores de la red integrar un mecanismo de submuestreo o compresión espacial directamente dentro de la propia operación de convolución, reduciendo el coste computacional general del modelo.

Capas de Agrupación (*Pooling*)

Tras la extracción de características mediante el filtrado lineal, los mapas de características resultantes suelen tener una dimensionalidad bastante alta. Llevar esta gran cantidad de datos a través de toda la red aumentaría exponencialmente el coste computacional y el riesgo de *overfitting*, ya que el modelo tendría demasiados parámetros para memorizar los datos de entrenamiento.

Para mitigar este problema, las CNNs intercalan **Capas de Agrupación (*Pooling*)** entre las capas convolucionales. El objetivo de estas capas es realizar un submuestreo espacial (*downsampling*) para reducir las dimensiones de la imagen, manteniendo intacta la dimensión de los canales (la profundidad).

El *Pooling* opera mediante una ventana espacial de tamaño definido (e.g., 2×2) que se desliza sobre la imagen con un determinado *stride*. Sin embargo, **no posee pesos aprendibles**, sino que simplemente aplica una función matemática sobre los valores de esa ventana.

Los dos enfoques más utilizados son:

- **Max Pooling (por máximo):** Es la variante más usada. Consiste en seleccionar y conservar únicamente el **valor máximo** de todos los píxeles presentes en la ventana. La intuición es conservar la activación más prominente de la región.
- **Average Pooling (por promedio):** En lugar del valor máximo, calcula la **media aritmética** de todos los valores de la ventana. Hoy en día su uso principal se reserva para el final de la red en forma de *Global Average Pooling* (GAP), donde promedia un mapa de características completo para convertirlo en un único valor, preparando los datos para la clasificación sin necesidad de aplanarlos violentamente.

Además de la reducción del coste computacional, esta operación hace que la red tenga **invarianza local a pequeñas traslaciones**. Dado que se toma el valor calculado de una pequeña vecindad, si la imagen se desplaza ligeramente en la imagen original, su representación en el mapa de características agrupado permanecerá igual, lo que otorga una enorme robustez.

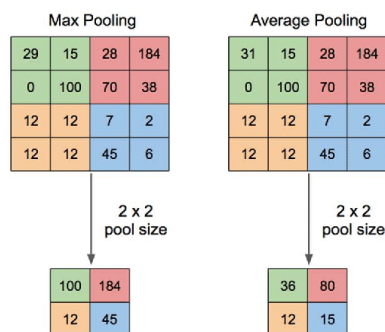


Figura 2.9: *Max Pooling* y *Average Pooling* [20]

Capas Totalmente Conectadas (*Fully Connected*)

Hasta este punto, tras los múltiples bloques de Convolución, Activación (ReLU) y *Pooling*, la red neuronal ha transformado la imagen de entrada en un conjunto de mapas de características bastante abstractos. Mientras que las primeras capas detectaban patrones de bajo nivel, estas últimas capas profundas mantienen información semántica de alto nivel.

No obstante, en este punto, la información todavía se encuentra estructurada como un tensor tridimensional (anchura $\times$ altura $\times$ canales). En tareas de clasificación categórica como es este proyecto, es necesario transformar este tensor espacial en un vector unidimensional de probabilidades.

Para ello, las arquitecturas CNN clásicas concluyen con una o varias **Fully Connected Layers** o capas densas, que vienen a ser las mismas que forman un Perceptrón Multicapa (MLP), es decir, todas las neuronas de una capa están conectadas con **todas** las neuronas de la capa anterior. La transición hacia estas capas consta de los siguientes pasos:

1. **Aplanamiento (*Flattening*):** En algunas CNNs tradicionales (como VGG o LeNet, más tarde hablaremos de ellas), el tensor resultante de la última capa de *Pooling* se desenrolla en un vector 1D muy largo. Cada elemento de este vector representa una característica detectada en alguna región de la imagen.
2. **Capas Densas:** Este vector se introduce en las capas *Fully Connected*, donde cada neurona está conectada con **todos** los elementos del vector anterior. Estas capas tienen como objetivo aprender las combinaciones no lineales de las características espaciales que determinan a qué clase pertenece la entrada.
3. **Capa de Salida y *Softmax*:** La arquitectura termina en una última capa densa que contiene exactamente tantas neuronas como clases tenga el problema. A estas salidas (*logits*) se les aplica la función de activación ***Softmax***, obteniendo así una distribución de probabilidad que indica la confianza del modelo para cada clase posible.

Sin embargo, el uso del *flattening* seguido de capas densas presenta un problema: concentra más del 80 % de los parámetros totales de la red en la fase final, incrementando drásticamente el riesgo de *overfitting*.

Para solventar esto, además de las distintas técnicas de regularización de las que ya hablaremos, las arquitecturas modernas (como las ResNet) sustituyen el *flattening* por una operación de **Global Average Pooling (GAP)**, que calcula la media espacial de cada mapa de características completo, reduciendo una matriz bidimensional entera a un único número escalar. Es decir, si la última capa convolucional extrae 512 mapas de características, el GAP genera un vector exacto de 512 elementos. Este vector se pasa directamente a la capa de salida final (*Softmax*), eliminando la necesidad de capas ocultas y creando modelos mucho más ligeros, rápidos y menos propensos a memorizar ruido.

Evolución y Redes Residuales (ResNets)

A lo largo de la historia, han surgido numerosos tipos de arquitecturas CNN, cada una con sus propias innovaciones y mejoras.

- **LeNet (1998):** Fue la primera CNN exitosa, diseñada por Yann LeCun para reconocer dígitos escritos a mano (dataset MNIST). Tenía una arquitectura muy simple con solo 2 capas convolucionales y 2 capas densas. Después de esta arquitectura, el campo de las CNNs se estancó durante más de una década debido a la escasez de capacidad computacional y de grandes conjuntos de datos para entrenar.
- **AlexNet (2012):** Marcó un antes y un después en la historia de las CNNs. Esta fue la primera red profunda en ganar el concurso ImageNet, reduciendo el error de clasificación a la mitad respecto al segundo clasificado. Introdujo varias innovaciones como el uso masivo de GPUs para entrenamiento, la función de activación ReLU y técnicas de regularización como el *Data Augmentation* y el *Dropout*.
- **VGG-16 (2014):** Esta arquitectura se basó en la idea de usar bloques de capas convolucionales pequeñas (3×3) apiladas, lo que permitió aumentar la profundidad de la red sin incrementar el número de parámetros de forma descontrolada. Sin embargo, seguía utilizando muchas capas densas al final, lo que hacía que el modelo fuera pesado y propenso a sobreajustar (casi 138M de parámetros).

A medida que se le intentaba añadir profundidad a cada arquitectura de red, aparecían problemas como el *Gradiente Desvanecido* (*Vanishing Gradient*) (al atenuarse durante el backpropagation), el *Gradiente Explosivo* (*Exploding Gradient*) (gradientes inestables) y la degradación (cuanto más profundas eran, mayor error de entrenamiento).

A raíz de esto, en 2015, Kaiming He y su equipo introdujeron la arquitectura **ResNet** (**Residual Network**), que revolucionó el campo del *Deep Learning* al introducir el concepto de *Residual Learning* (Aprendizaje Residual):

En lugar de que cada conjunto de capas aprendiera directamente una transformación deseada $H(x)$, el aprendizaje residual consistía en que el bloque aprendiera una función residual $F(x)$ definida como:

$$F(x) = H(x) - x$$

De este modo, la salida del bloque pasa a definirse como:

$$y = F(x) + x$$

donde x representa la entrada al bloque y $F(x)$ la transformación aprendida por las capas convolucionales internas. La suma directa de la entrada con la salida del bloque se implementa mediante una conexión de salto (*skip connection* o *shortcut connection*).

Esta formulación aporta una ventaja fundamental desde el punto de vista de la optimización. Si la transformación óptima fuese cercana a la identidad, el bloque únicamente tendría que aprender $F(x) = 0$, lo cual resulta significativamente más sencillo que aproximar directamente $H(x) = x$. Esto facilita el entrenamiento de redes considerablemente más profundas sin que aparezca el fenómeno de degradación observado en arquitecturas previas.

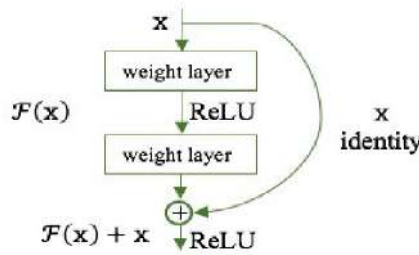


Figura 2.10: Bloque de una CNN residual general [21]

Un **bloque residual** típico está compuesto por varias capas convolucionales (normalmente dos o tres), seguidas de capas de *Batch Normalization* y funciones de activación ReLU. La entrada del bloque se suma a la salida de estas capas antes de aplicar la activación final.

Formalmente, si denotamos por x_l la entrada del bloque l , la salida queda definida como:

$$x_{l+1} = x_l + F(x_l, W_l)$$

donde W_l representa los parámetros aprendidos en dicho bloque.

Desde el punto de vista del *backpropagation*, esta estructura permite que el gradiente fluya directamente a través de la conexión identidad, solucionando en parte el problema del *vanishing gradient*. Como consecuencia, se pueden entrenar redes con muchas más capas.

Existen principalmente dos variantes de bloques residuales:

- **Basic Block:** Utilizado en arquitecturas como ResNet-18 y ResNet-34. Está compuesto por dos convoluciones 3×3 . Es más simple y ligero, pero ofrece menor capacidad representacional.
- **Bottleneck Block:** Utilizado en ResNet-50, ResNet-101 y ResNet-152. Está formado por una secuencia $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$. La primera capa reduce dimensionalidad, la segunda realiza la convolución espacial y la tercera restaura la dimensión original. Esta estructura reduce el número de parámetros y el coste computacional manteniendo una alta capacidad de representación.

El bloque *Bottleneck* resulta especialmente eficiente en términos de relación precisión-coste computacional, lo que ha convertido a **ResNet-50** en una de las variantes más utilizadas.

Además de esta, las arquitecturas ResNet más comunes son:

- ResNet-18 (≈ 11 M parámetros)
- ResNet-34 (≈ 21 M parámetros)
- ResNet-50 (≈ 25 M parámetros)
- ResNet-101 (≈ 44 M parámetros)
- ResNet-152 (≈ 60 M parámetros)

A diferencia de VGG-16 (≈ 138 M parámetros), ResNet consigue mayor profundidad con un número considerablemente menor de parámetros, lo que mejora la eficiencia en memoria y el rendimiento general.

En definitiva, las redes residuales superan a arquitecturas previas como AlexNet o VGG-16 por:

- Mejor flujo del gradiente gracias a las conexiones identidad.
- Eliminación del problema de degradación en redes profundas.
- Mayor eficiencia paramétrica.
- Mejor estabilidad en el entrenamiento.

Transfer Learning y Fine-Tuning

A pesar de la eficiencia y potencia tanto de las arquitecturas citadas como de las que hablaremos después, inicializar todos los parámetros de una de estas redes desde cero presenta un problema obvio: la necesidad de una cantidad masiva de datos para evitar el **overfitting**.

Para solventar esto, una de las técnicas más utilizadas es el **Transfer Learning** (*Aprendizaje por Transferencia*), que consiste en aprovechar los pesos de una red preentrenada en un gran conjunto de datos (como ImageNet) y adaptarlos a una nueva tarea con un conjunto de datos más pequeño.

Para adaptar una red preentrenada, se aplica un proceso conocido como **Fine-Tuning** (*Ajuste Fino*), que consta de los siguientes pasos:

1. **Sustitución de la cabecera (*Head*):** Se elimina la última capa totalmente conectada de la red original (esa que predecía miles de clases) y se reemplaza por una nueva capa inicializada aleatoriamente con tantas neuronas como clases tenga el nuevo problema.
2. **Congelación (*Freezing*):** Aunque es opcional, casi siempre se “congelan” (se desactiva el cálculo de gradientes) las primeras capas convolucionales para no destruir los detectores de estructuras básicas (bordes y colores) que ya funcionan bien.
3. **Reentrenamiento:** Se entrena la red (completa o solo las últimas capas) con el dataset del problema actual. El objetivo no es que la red aprenda a ver desde cero, sino que ajuste sus pesos de alto nivel para entender la semántica del nuevo problema.

Con el paso de los años, el éxito del *Transfer Learning* ha demostrado que preentrenar modelos masivos era la clave para solucionar casi cualquier problema de visión por computador, por lo que será la base de la metodología de este proyecto.

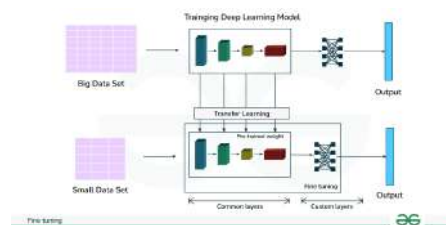


Figura 2.11: Fine-Tuning

2.2.2. Vision Transformers (ViTs)

A pesar del dominio de las Redes Neuronales Convolucionales en el campo de la visión por computador durante la última década, su diseño presenta una limitación estructural: el **campo receptivo local**. Con esto nos referimos a que al procesar la imagen mediante una ventana deslizante, las CNNs son excelentes capturando patrones adyacentes (bordes, texturas), pero no tienen la suficiente capacidad para modelar relaciones a larga distancia. En problemas con imágenes complejas (como este), relacionar una estructura en un extremo de la imagen con otro objeto situado a miles de píxeles de

distancia requiere el apilamiento de muchas capas de *Pooling*, perdiendo resolución por el camino.

La revolución que puso solución en parte a este problema vino del Procesamiento de Lenguaje Natural (NLP). En 2017, la arquitectura **Transformer** [22] supuso un antes y un después en la traducción automática de textos al prescindir de arquitecturas secuenciales y utilizar exclusivamente **mecanismos de atención**, que permiten al modelo relacionar cualquier parte del texto con otra, sin importar su distancia. Así, los transformers fueron la base de los modelos de lenguaje masivos (LLMs) como *GPT-3*.

La conversión desde el NLP hacia la visión por computador (de textos a imágenes) se produjo en 2020, cuando un equipo de investigadores de Google Brain presentó el **Vision Transformer (ViT)** en su artículo “*An Image is Worth 16x16 Words*” [23].

La premisa de este trabajo fue: en lugar de utilizar convoluciones, **una imagen puede ser tratada exactamente igual que un texto**. Del mismo modo que un Transformer de NLP procesa una secuencia de palabras (o *tokens*) para entender una frase, un ViT procesa una secuencia de trozos de imagen para entender e inferir de la imagen completa.

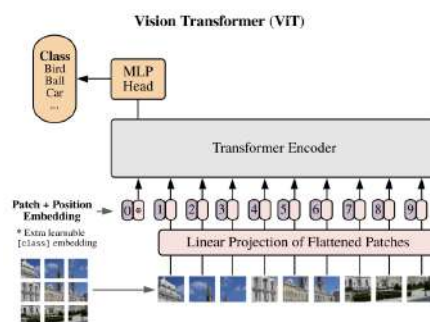


Figura 2.12: Arquitectura general del Vision Transformer (ViT) [23]

Secuenciación de la imagen: Parches

El principal problema de arquitectura al aplicar un Transformer estándar a problemas de visión por computador es el formato de los datos de entrada. Mientras que los modelos de NLP reciben secuencias de una dimensión (tokens o palabras), una imagen es un tensor tridimensional (anchura $\times$ altura $\times$ canales).

Para adaptar la imagen a la arquitectura del Transformer, el ViT se acoge al concepto de **Parches**. La imagen se divide en pequeñas regiones cuadradas. Si definimos una imagen $H \times W \times C$ (donde H es la altura, W la anchura y C el número de canales, típicamente 3 para imágenes RGB) y parches de resolución fija ($P \times P$), la imagen se fragmenta en una secuencia de N parches, donde:

$$N = \frac{H \times W}{P^2} \quad (2.17)$$

Por ejemplo, si introducimos una imagen 224×224 y configuramos un parche 16×16 píxeles, la imagen quedará dividida exactamente en $N = 196$ parches.

Sin embargo, el Transformer no puede procesar directamente estos parches. Para convertir cada parche en un vector unidimensional (un token compatible con el Transformer), se aplica una operación de **Flattening** (aplanamiento) a cada parche, transformándose en un vector unidimensional de tamaño $P^2 \cdot C$ (en nuestro ejemplo, $16 \times 16 \times 3 = 768$ elementos).

Finalmente, para que todos los vectores tengan una dimensionalidad constante y manejable dentro de las capas ocultas del modelo (una dimensión D), cada parche aplanado se multiplica por una matriz de pesos aprendible $\mathbf{E}$. A esta operación se le denomina **Proyección Lineal** o *Patch Embedding*:

$$x_p \mathbf{E} \in \mathbb{R}^{N \times D} \quad (2.18)$$

El resultado de esta transformación es una secuencia de N vectores de dimensión D . A partir de este punto, se dice que la red ya no “sabe” que está procesando una imagen, simplemente está operando sobre una secuencia de vectores idéntica a la que generaría una frase de texto en un traductor automático.

Codificación Posicional

A diferencia de las CNNs, que procesan los píxeles de una imagen en su orden natural, los ViTs tratan los parches como una secuencia de tokens, lo que implica que el modelo no “sabe” nada sobre la posición espacial de cada parche dentro de la imagen original.

Para solucionar esto y conseguir que sepa información geométrica, aparece el mecanismo de **Codificación Posicional** (*Positional Encoding*). Este mecanismo consiste en añadir a cada vector de parche un nuevo vector de la misma dimensión D que codifica su posición dentro de la cuadrícula 2D original. El modelo en el entrenamiento irá ajustando estas posiciones, es decir, estas codificaciones pueden ser aprendidas (el modelo aprende un vector específico para cada posición) o fijas (utilizando funciones trigonométricas para generar patrones únicos para cada posición).

Además, para que el modelo pueda procesar la imagen completa, se añade un token especial al inicio de la secuencia, conocido como **[CLS]** (de *Classification*). Este token no representa ningún parche real, sino que es un vector aprendible que se inicializa junto a los parches de la imagen y va a través de todas las capas del Transformer interactuando con ellas.

A la salida del último bloque de la red, se asume que este token [CLS] ha aprendido la información global de toda la imagen, ya que ha tenido la oportunidad de relacionarse con todos los parches. Por tanto, se extrae el estado final de este token y se introduce en una capa de clasificación (normalmente una capa densa seguida de *Softmax*) para obtener la predicción final.

Aunque no entraremos mucho en detalles matemáticos, la secuencia completa de entrada al primer bloque del Transformer (z_0) se formularía de la siguiente manera:

$$z_0 = [x_{\text{class}}; x_p^1 \mathbf{E}; x_p^2 \mathbf{E}; \dots; x_p^N \mathbf{E}] + \mathbf{E}_{\text{pos}} \quad (2.19)$$

Donde x_{class} representa el *token* [CLS], los términos $x_p^i \mathbf{E}$ son los parches de la imagen tras la proyección lineal, y $\mathbf{E}_{\text{pos}}$ representa la matriz de las codificaciones posicionales aprendibles que se suman a la secuencia completa.

Mecanismo de Auto-Atención (*Self-Attention*)

Este es el núcleo principal del Transformer, es la forma en la que “entiende” las relaciones entre los parches de la imagen.

El funcionamiento del mecanismo de *Self-Attention* lo podemos entender como un médico mirando una biopsia. Este no da un diagnóstico en base a unos pocos píxeles, sino que su instinto lo lleva a prestar atención a muchas cosas como: ¿qué tiene alrededor?, ¿está dentro de un conducto?,...

Así, en su primera capa, **permite que cada parche de la imagen “mire” a todos los demás parches al mismo tiempo**, independientemente de la distancia entre ellos. Esto dota a la red de una comprensión global y semántica de la imagen original desde el primer momento.

Para lograr esta interacción global, por cada parche de la imagen, la red genera tres vectores:

- **Query (Q, Consulta):** “¿Qué estoy buscando?” Es el vector que representa la consulta o pregunta que cada parche se hace sobre los demás para determinar su relevancia.
- **Key (K, Clave):** “¿Qué información tengo?” Es el vector que representa lo que el parche “contiene” o “es”.
- **Value (V, Valor):** “¿Qué información visual real tengo?” Contiene las características que el parche aportará si resulta ser relevante para otro parche.

El objetivo es medir la compatibilidad entre lo que un parche busca y lo que los demás ofrecen. Si la *Query* del parche 1 encaja perfectamente con la *Key* del parche 25, significa que están altamente relacionados.

El cálculo de esta atención, se calcula con el **producto escalar** de la consulta (Q) con las claves (K) de todos los demás parches. Si el producto escalar es alto, la relación entre los dos parches es fuerte.

Para evitar que los valores de atención se vuelvan demasiado grandes, se normalizan dividiendo por la raíz cuadrada de la dimensión de las claves ($\sqrt{d_k}$). Seguidamente, se aplica una función **Softmax** para convertir estos valores en una distribución de probabilidad que suman 1.

Finalmente, estos pesos se multiplican por la matriz Valores (V). matemáticamente, el mecanismo de atención se define como:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.20)$$

El resultado de esta operación es un nuevo conjunto de vectores para cada parche, pero ahora no solo contiene su información original, sino que contiene información global de todos los parches de la imagen con los que guarda alguna relación.

En la práctica, relacionar los parches usando un único conjunto de matrices Q, K, V resulta limitante, ya que una misma parte de la imagen puede tener múltiples tipos de relaciones simultáneas.

Para solucionarlo, el ViT emplea el **Multi-Head Attention (MHA)**. Este mecanismo consiste en dividir los vectores en h “cabezas” (*heads*) independientes y calcular la ecuación de atención en paralelo para cada una de ellas. Las salidas de todas las cabezas se concatenan y se proyectan de nuevo a la dimensión original. Esto permite al modelo prestar atención a diferentes características representacionales de la imagen de forma simultánea.

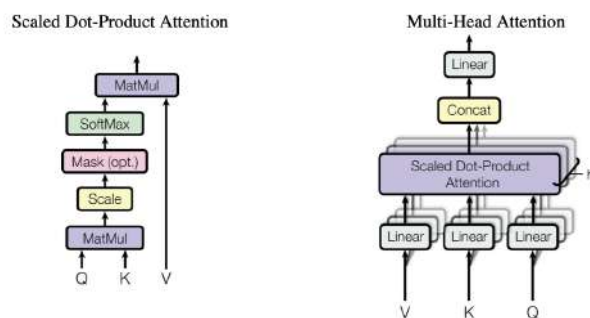


Figura 2.13: Self-Attention utilizando el producto escalar con las matrices Q, K y V . Arquitectura del Multi-Head Attention procesando múltiples proyecciones en paralelo [22]

2.2.3. Modelos Fundacionales (Foundation Models)

Como hemos venido explicando a lo largo de la anterior sección, los ViTs consiguieron solucionar el problema del campo receptivo local de las CNNs. No obstante, esto tiene un precio no siempre al alcance: la necesidad de una cantidad masiva de datos para entrenar desde cero estos modelos.

En el dominio de este proyecto, este límite de datos es aún más pronunciado. El aprendizaje supervisado clásico requiere que cada imagen de entrenamiento esté anotada meticulosamente por un especialista. Dado el coste (temporal y humano), crear conjuntos de datos médicos etiquetados del tamaño de ImageNet es logísticamente imposible. Ante esto, era de máxima necesidad un cambio de enfoque para poder aprovechar los millones de muestras histológicas **no etiquetadas** que los hospitales proporcionan cada año.

La solución a este nuevo problema llegó con la consolidación de los **Modelos Fundacionales (Foundation Models)**, modelos masivos preentrenados en grandes cantidades de datos (no etiquetados).

El cambio de paradigma se basa en la separación de la fase de entrenamiento en dos etapas que, a priori, podríamos pensar que son iguales a las anteriormente explicadas, pero que en realidad son completamente distintas:

1. **Preentrenamiento masivo y generalista:** El modelo procesa millones de imágenes **sin importar a qué clase pertenecen**, dedicando toda su capacidad a entender la estructura del mundo visual (en nuestro caso, la morfología de los tejidos humanos, la densidad celular o la arquitectura de los conductos).
2. **Adaptación (*Fine-Tuning* o *Feature Extraction*):** Una vez que el modelo ya “sabe ver”, se reutiliza su conocimiento para resolver un problema muy específico y con pocos datos etiquetados.

Sin embargo, lo lógico sería preguntarnos: si el modelo fundacional se entrena con millones de imágenes que **no tienen etiquetas médicas**, ¿cómo sabe la red qué debe aprender o cómo corregir sus errores durante el *backpropagation*? La respuesta reside en el motor matemático de estos modelos: el Aprendizaje Auto-Supervisado.

Aprendizaje Auto-Supervisado (SSL)

Como hemos visto, en el aprendizaje supervisado tradicional (como el que utilizan las CNNs clásicas), la red ajusta sus parámetros intentando minimizar el error entre su predicción y la etiqueta real proporcionada por un experto. Sin embargo, al tener conjuntos de datos sin etiquetas, se necesita una nueva forma para calcular ese error y propagar los gradientes.

El **Aprendizaje Auto-Supervisado** (*Self-Supervised Learning*, SSL) consiste en que el sistema genere sus propias señales de supervisión a partir de datos crudos, es decir, es transformar un problema no supervisado en uno “supervisado” internamente mediante tareas *pretext* (o tareas de preentrenamiento). En los Modelos Fundacionales visuales, destacan principalmente dos enfoques:

- **Modelado de Imágenes Enmascaradas (*Masked Image Modeling*, MIM):** Esta técnica es la adaptación visual de la forma en la que se entrenan los modelos de lenguaje (como BERT). Durante el preentrenamiento, la imagen se divide en parches, pero **se oculta o se enmascara aleatoriamente un porcentaje muy alto de ellos** (hasta el 75 % de la imagen). El objetivo de la red es analizar los parches que han quedado visibles y, utilizando el mecanismo de *Self-Attention* descrito anteriormente, reconstruir los píxeles de los parches que han sido ocultos. Para lograr realizar esta tarea de reconstrucción sobre muchas imágenes, el modelo se ve matemáticamente obligado a desarrollar una profunda comprensión del fundamento de la imagen.

- **Aprendizaje Contrastivo y de Destilación (DINO, MoCo):** Esta técnica se centra en la invariancia de las representaciones. En algoritmos como **DINO** (*Self-Distillation with No Labels*), el modelo toma una única imagen y genera dos o más “vistas” diferentes mediante transformaciones agresivas (como recortes aleatorios a distintas escalas, rotaciones o alteraciones en la tonalidad). A continuación, ambas vistas se pasan por la red. El objetivo matemático es forzar a que las representaciones vectoriales (*embeddings*) resultantes de ambas vistas sean lo más parecidas posible entre sí, mientras se maximiza la distancia respecto a las representaciones de otras imágenes distintas. Gracias a esto, la red aprende a ignorar alteraciones y se centra en reconocer qué representa la imagen (independientemente del zoom o el color, por ejemplo).

Al iterar estos procesos computacionalmente masivos sobre millones de imágenes durante **semanas o meses** de entrenamiento en supercomputadores, los pesos al final convergen. Al finalizar, el Modelo Fundacional posee un “diccionario visual” de altísima calidad. Aunque la red todavía no sabe por ejemplo qué es el “cáncer” ni conoce las distintas clases, es capaz de extraer las características más ricas, complejas y representativas de cualquier biopsia que se le presente.

Modelos Fundacionales en Histopatología

El éxito del preentrenamiento auto-supervisado en imágenes impulsó a la comunidad médica a aplicar estas técnicas en grandes bases de datos clínicas. Hasta hace muy poco, los modelos en patología computacional se inicializaban con pesos de ImageNet, lo cual era poco óptimo dado que las imágenes naturales (perros, coches) no se parecen en nada a una imagen histológica.

En los últimos dos años (2023-2024), instituciones líderes han liberado **Modelos Fundacionales Específicos para Patología (PFMs - *Pathology Foundation Models*)**. Estos modelos, basados en arquitecturas Vision Transformer (ViT), han sido preentrenados de forma auto-supervisada con millones de recortes histológicos procedentes de cientos de miles de pacientes. Entre los más destacados se encuentran:

- **Phikon:** Desarrollado por Owkin, es un modelo ViT preentrenado mediante el algoritmo *iBOT* (que combina la destilación de DINO con el enmascaramiento MIM) sobre el conjunto de datos TCGA (*The Cancer Genome Atlas*), utilizando más de 40 millones de parches de múltiples tipos de cáncer.
- **UNI:** Desarrollado por el laboratorio de Faisal Mahmood (Harvard), es un modelo fundacional masivo (ViT-Large) preentrenado con más de 100.000 histologías completas (WSIs) del Mass General Brigham.
- **Virchow:** Creado por Paige y el Memorial Sloan Kettering Cancer Center, es un modelo entrenado con 1.5 millones de WSIs usando aprendizaje contrastivo. Su enorme escala le permite capturar variaciones en la tinción y la morfología que modelos más pequeños ignoran.

La forma de aprovechar estos gigantes en un conjunto de datos pequeño como el de nuestro proyecto (BRACS), es mediante la **Extracción de Características** (*Feature Extraction*).

Dadas las dimensiones de estos modelos (cientos de millones de parámetros), intentar realizar un *Fine-Tuning* completo (actualizando todos los pesos de la red) con las pocas imágenes de BRACS nos llevaría a un sobreajuste inmediato.

Es por ello que la metodología que aplicaremos en este trabajo será:

1. **Congelar todos los pesos del Modelo Fundacional:** Las imágenes del dataset BRACS se pasarán a través del modelo únicamente en modo de inferencia (hacia adelante). El modelo fundacional actuará como un “traductor perfecto”, recibiendo una imagen de píxeles y devolviendo a su salida un **vector de características o *embedding*** (por ejemplo, un vector unidimensional de 768 o 1024 valores numéricos que resume toda la semántica del tumor).
2. **Construcción de un clasificador ligero:** Una vez obtenidos estos *embeddings* para todas las imágenes de entrenamiento, se construirá y entrenará un clasificador muy ligero (como un MLP pequeño o un clasificador lineal) que tomará estos vectores como entrada para predecir las 7 clases de lesiones mamarias.

De este modo, se combina lo mejor de ambos: la inmensa comprensión anatómica de un modelo fundacional preentrenado y la agilidad de un clasificador ligero adaptado específicamente a la tarea clínica de nuestro proyecto.

2.3. Modelos fundacionales y trabajos recientes en patología digital

2.3.1. El modelo UNI

En los últimos años, tal y como venimos comentando a lo largo de este proyecto, han crecido numerosos modelos fundacionales en histopatología que buscan superar las limitaciones de las arquitecturas tradicionales mediante el aprendizaje auto-supervisado a gran escala. Uno de los trabajos más destacados en esta línea es el modelo **UNI**, propuesto por Chen *et al.* (2024) [24].

UNI es un modelo generalista basado en la arquitectura Vision Transformer (ViT-Large), diseñado como un encoder visual para tareas de anatomía patológica. A diferencia de estudios previos limitados por el tamaño de los datos, este modelo fue preentrenado sobre un conjunto masivo denominado *Mass-100K*, compuesto por más de 100 millones de recortes histológicos extraídos de más de 100.000 imágenes completas (WSIs), abarcando 20 tipos de órganos distintos.

El entrenamiento se llevó a cabo utilizando el algoritmo auto-supervisado **DINOv2**, un algoritmo que combina estrategias de auto-destilación y modelado de imágenes enmascaradas, permitiendo aprender representaciones visuales robustas.

Para evaluar su capacidad de generalización, los autores diseñaron un largo benchmark compuesto por **34 tareas clínicas** que incluyen clasificación a nivel de región de interés (ROI), segmentación, recuperación de imágenes y clasificación a nivel de diapositiva completa (WSI). UNI demostró superar de forma consistente a modelos previos en la mayoría de las tareas evaluadas.

Uno de los resultados más relevantes fue en tareas de clasificación de cáncer a gran escala (benchmarks OT-43 y OT-108 en el paper oficial), donde el modelo fue capaz de clasificar hasta **108 tipos distintos de cáncer** con mejoras en métricas como el AU-ROC (área bajo la curva) y la exactitud frente a otros modelos preentrenados. Además, UNI mostró una destacada capacidad para generalizar a enfermedades raras y subtipos inusuales.

Otra propiedad clave que se logró con este modelo fue la potencia en escenarios con pocas etiquetas, mediante el uso de prototipos de clase, así como su independencia de la resolución de entrada, lo que lo hace especialmente adecuado para entornos clínicos reales donde no se establece unas dimensiones de entrada fijas.

Estos resultados evidencian que el escalado de datos junto a técnicas de aprendizaje auto-supervisado permite obtener representaciones altamente transferibles, posicionando a UNI como uno de los modelos fundacionales más avanzados hasta la fecha en histopatología computacional.

En la Figura 2.14 se muestra una visión general del modelo UNI, incluyendo su proceso de preentrenamiento y las distintas tareas de evaluación consideradas.

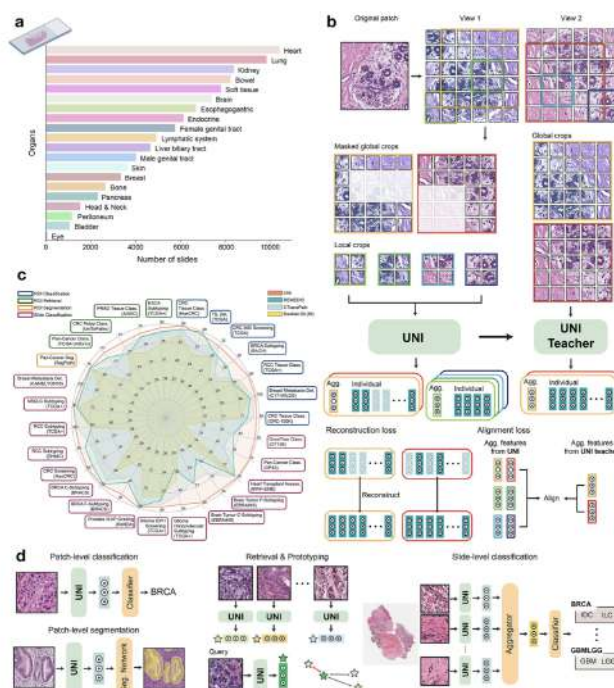


Figura 2.14: Arquitectura y pipeline de entrenamiento del modelo UNI [24]

2.3.2. El modelo Virchow

Otro de los modelos fundacionales más importantes es **Virchow**, propuesto por Vorontsov *et al.* (2023) [25]. Este modelo representa uno de los mayores esfuerzos en cuanto a escalado de datos y arquitectura en el ámbito de la patología computacional.

Virchow es un modelo basado en Vision Transformers con alrededor de 632 millones de parámetros, entrenado utilizando también el algoritmo DINOv2. Su principal novedad está en el uso de un conjunto de datos de una escala sin precedentes, compuesto por aproximadamente **1.5 millones de imágenes completas (WSIs)** de alrededor de 100.000 pacientes, abarcando 17 tipos de tejidos distintos.

A diferencia de trabajos previos, este modelo se entrenó sobre datos reales clínicos a gran escala, incluyendo tanto tejido sano como canceroso, lo que le permitió capturar diversos patrones histológicos. El entrenamiento se realizó de nuevo mediante un enfoque auto-supervisado basado en múltiples vistas, que permitió aprender representaciones robustas a partir de diferentes regiones.

Para evaluar su rendimiento, los autores diseñaron numerosos benchmarks centrados en tareas clave, incluyendo la detección de cáncer a nivel de WSI y la predicción de biomarcadores. En la tarea de detección de cáncer, Virchow alcanzó un **AUC de 0.949** en 17 tipos de cáncer, así como un **AUC de 0.937** en 7 tipos de cáncer raros, demostrando una potente capacidad de generalización incluso en escenarios con baja representación de datos.

Además, el modelo mostró un rendimiento bueno tanto en benchmarks a nivel de parche como a nivel de diapositiva completa, superando de nuevo a modelos previos. Es especialmente destacable su robustez frente a datos fuera de distribución (diferentes a los usados en el entrenamiento), así como su capacidad para mantener un alto rendimiento en datos de distintos centros clínicos.

En la Figura 2.15 se presenta el esquema general del modelo Virchow, incluyendo el proceso de extracción de parches, aprendizaje de representaciones y su aplicación a tareas clínicas.

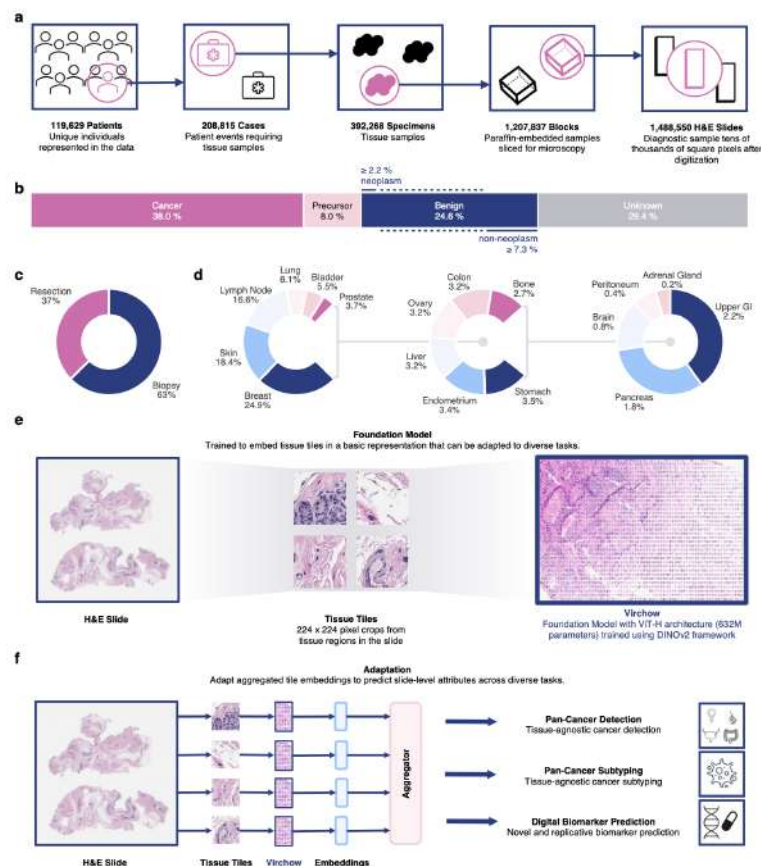


Figura 2.15: Arquitectura y pipeline de entrenamiento del modelo Virchow [25]

2.3.3. El modelo fundacional basado en datos públicos: Phikon

Dentro de los distintos modelos fundacionales desarrollados sobre datos públicos, destaca **Phikon**, propuesto por Owkin [26], que representa uno de los primeros intentos de construir un modelo generalista en histopatología utilizando exclusivamente datos abiertos al público.

Phikon es un modelo basado en Vision Transformers (ViT-Base) con aproximadamente 86 millones de parámetros, preentrenado sobre el conjunto de datos TCGA (*The Cancer Genome Atlas*), empleando el algoritmo auto-supervisado **iBOT**. Este algoritmo, al igual que DINOv2, combina aprendizaje contrastivo con técnicas de modelado de imágenes enmascaradas.

El modelo fue entrenado sobre aproximadamente 43 millones de parches histológicos extraídos de unas 6.000 imágenes completas (WSIs), abarcando múltiples tipos de cáncer y tejidos. Este enfoque basado en datos públicos lo diferencia de modelos como UNI o Virchow, que utilizan grandes colecciones de datos privados de más calidad.

Phikon fue evaluado en múltiples tareas, incluyendo clasificación de tejidos, detección de metástasis, y predicción de biomarcadores a nivel de WSI. Los resultados mostraron que el modelo alcanzaba un rendimiento competitivo frente a otros modelos fundacionales más grandes, demostrando que se puede obtener representaciones de alta calidad incluso con conjuntos de datos más limitados o genéricos.

Sin embargo, estudios posteriores han evidenciado que el escalado masivo de datos no siempre garantiza mejoras consistentes en todas las tareas. En particular, en tareas específicas como la predicción de biomarcadores, modelos más pequeños y especializados basados en iBOT pueden incluso superar a modelos fundacionales de mayor tamaño. Este hallazgo sugiere que, si bien los modelos fundacionales generalistas ofrecen una gran capacidad de transferencia, la especialización sigue siendo un factor clave en determinadas aplicaciones clínicas.

En conjunto, Phikon representa un punto intermedio entre modelos tradicionales y modelos fundacionales de gran escala, demostrando que el aprendizaje auto-supervisado sobre datos públicos también puede ser competitivo, a la vez que pone de manifiesto las limitaciones actuales del escalado de datos en el ámbito de la histopatología computacional.

En la Figura 2.16 se muestra una comparación del rendimiento de distintos modelos fundacionales en tareas de predicción de biomarcadores, evidenciando que modelos más pequeños como Phikon pueden igualar o incluso superar a modelos de mayor escala en ciertos escenarios.

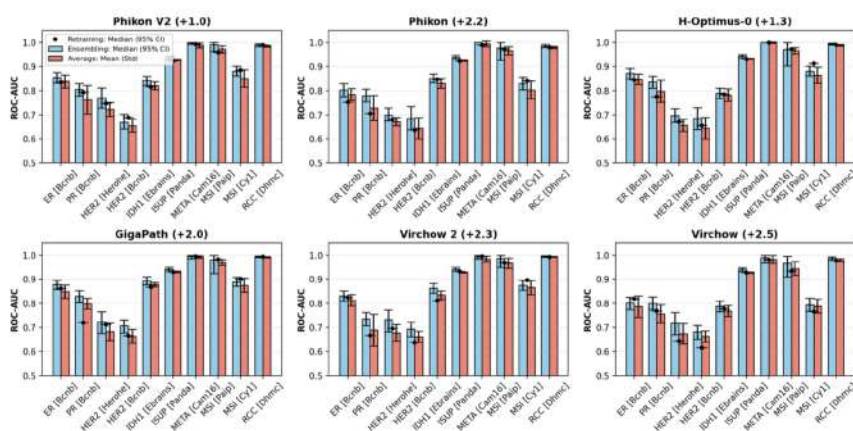


Figura 2.16: Comparativa de modelos fundacionales en tareas de patología computacional [26]

2.3.4. Benchmark clínico de modelos fundacionales: PathBench

A medida que los modelos fundacionales han ido haciéndose sitio en patología computacional, ha surgido la necesidad de evaluarlos en escenarios clínicos reales. En este contexto, recientemente se ha propuesto **PathBench** [27], el primer benchmark diseñado específicamente para comparar modelos fundacionales en múltiples tipos de cáncer en condiciones realistas.

Este trabajo surge como respuesta a varias limitaciones presentes en estudios previos, tales como la falta de estandarización en las evaluaciones, la posible contaminación de datos debido al uso de datasets públicos presentes en el preentrenamiento, y la evaluación restringida a tareas específicas. Para abordar estos problemas, PathBench propone un marco de evaluación basado exclusivamente en datos clínicos privados, garantizando la ausencia de *data leakage* y proporcionando una evaluación más representativa del entorno hospitalario real.

El benchmark evalúa un total de **19 modelos fundacionales** en **64 tareas clínicas** distribuidas en **5 tipos de cáncer**, habiendo específicamente **12 tareas en cáncer de mama**. Estas tareas abarcan desde diagnóstico histológico hasta predicción de biomarcadores, subtipado molecular y análisis de supervivencia.

A nivel global, los resultados muestran que modelos como **Virchow2**, **H-Optimus-1** y **UNI2** se sitúan entre los de mejor rendimiento. En particular, Virchow2 y H-Optimus-1 destacan en tareas de subtipado histológico, mientras que H-Optimus-1 muestra un comportamiento especialmente sólido en subtipado molecular. Por su parte, UNI2 obtiene los mejores resultados en tareas de predicción de supervivencia.

Los resultados evidencian que **no existe un modelo dominante en todas las tareas**. Sin embargo, se observan tendencias claras: **Virchow2** destaca en tareas de diagnóstico y subtipado histológico, **H-Optimus-1** presenta un rendimiento competitivo en tareas más complejas como el subtipado molecular, y **UNI2** sobresale en escenarios de pronóstico.

Estos resultados ponen de manifiesto que el rendimiento de los modelos fundacionales depende de la tarea y que, a pesar de los avances recientes, el cáncer de mama continúa representando un desafío significativo en patología computacional.

En este contexto, el presente trabajo se inspira directamente en este benchmark para poner a prueba esos mejores modelos y diseñar una comparación sobre un dataset más reducido, con el objetivo de analizar el comportamiento de distintas arquitecturas en un entorno controlado y de alta dificultad diagnóstica.

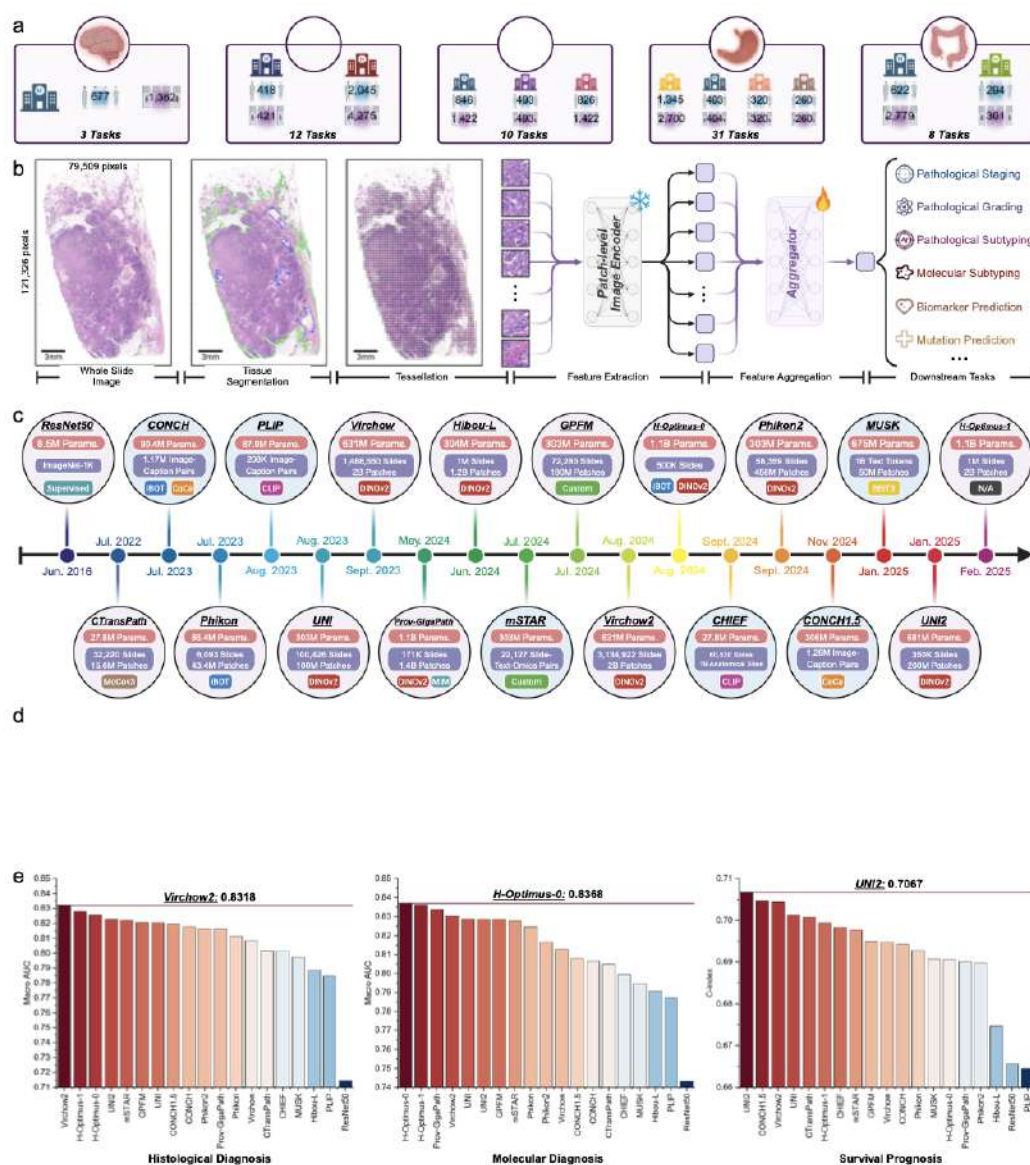


Figura 2.17: Resumen del benchmark PathBench [27]

En conjunto, estos trabajos reflejan una clara evolución desde arquitecturas convolucionales tradicionales hacia modelos basados en transformers y aprendizaje auto-supervisado a gran escala. Sin embargo, también ponen de manifiesto que el rendimiento no depende únicamente del tamaño del modelo o del volumen de datos, sino de la adaptación al problema específico.

En este contexto, surge una cuestión clave: determinar qué tipo de modelo resulta más adecuado cuando se enfrenta a un escenario real, con datos limitados y alta complejidad intra-clase.

Este trabajo se sitúa precisamente en ese punto, planteando una evaluación sistemática y comparativa entre redes convolucionales, Vision Transformers y modelos fundacionales, con el objetivo de analizar su capacidad de generalización y su comportamiento frente a la variabilidad inherente de las imágenes histológicas.

Capítulo 3

Dataset BRACS

El conjunto de datos utilizado en este trabajo es **BRACS** (*Breast Carcinoma Subtyping*), un dataset público diseñado específicamente para el análisis de imágenes histopatológicas de cáncer de mama.

A diferencia de otros datasets más simplificados, BRACS presenta una dificultad adicional al incluir no solo tejido sano y lesiones claramente malignas, sino también lesiones intermedias o atípicas, que constituyen uno de los mayores retos en el diagnóstico histopatológico. Esto lo convierte en un escenario especialmente adecuado para evaluar la capacidad de generalización de distintos modelos de Deep Learning.

El dataset fue desarrollado en colaboración entre *IRCCS Fondazione Pascale* (National Cancer Institute of Naples), el *Institute for High Performance Computing and Networking* del Consejo Nacional de Investigación de Italia (ICAR-CNR) y *IBM Research-Zurich*. Esta colaboración garantiza tanto la calidad clínica de las anotaciones como el rigor técnico en su construcción.

Desde el punto de vista de los datos, BRACS está compuesto por imágenes histológicas obtenidas mediante tinción Hematoxilina y Eosina (H&E), digitalizadas utilizando un escáner *Aperio AT2* con una resolución de 0.25 micras por píxel (lo que equivale a un aumento de 40x). Estas imágenes se presentan en formato de *Whole Slide Images* (WSI), alcanzando dimensiones de cientos de miles de píxeles en cada eje.

Dada esta enorme resolución, el dataset no se limita únicamente a las imágenes completas, sino que incluye también regiones de interés (*Regions of Interest*, ROIs) extraídas y anotadas. Estas ROIs permiten trabajar a una escala más manejable, manteniendo al mismo tiempo la información diagnóstica relevante.

En definitiva, se ha elegido BRACS como dataset de este proyecto porque combina datos clínicos reales, anotaciones expertas y una estructura que nos permite abordar problemas tanto de clasificación a nivel de imagen completa como a nivel de región.

El dataset se divide en un total de **7 subclases histológicas**, que reflejan distintas etapas del carcinoma mamario. Estas clases no solo representan categorías independientes, sino que siguen una progresión desde tejido sano hasta carcinoma invasivo.

Tal y como se observa en la Tabla 3.1, estas siete clases pueden agruparse de forma natural en tres grandes categorías de lesiones: **benignas**, **atípicas** y **malignas**.

Tabla 3.1: Clases histológicas del dataset BRACS y su agrupación clínica.

Sigla	Clase	Grupo clínico
N	Normal	Benigno
PB	Patológico Benigno	Benigno
UDH	Hiperplasia Ductal Usual	Benigno
FEA	Atipia Epitelial Plana	Atípico
ADH	Hiperplasia Ductal Atípica	Atípico
DCIS	Carcinoma Ductal In Situ	Maligno
IC	Carcinoma Invasivo	Maligno

Desde el punto de vista computacional, esta estructura introduce un reto de gran dificultad. Mientras que las clases benignas y malignas presentan diferencias relativamente claras, las lesiones atípicas (especialmente ADH y FEA) comparten características tanto con tejido benigno como con carcinoma temprano, dificultando la discriminación incluso para especialistas humanos.

Esto se traduce en un problema de clasificación altamente complejo, donde las fronteras entre clases no son nítidas. Por este motivo, el dataset BRACS resulta especialmente adecuado para evaluar modelos capaces de capturar tanto patrones locales como contexto global.

El dataset BRACS está compuesto por un total de **547 imágenes completas (WSIs)** correspondientes a **189 pacientes distintos**.

El dataset incluye además un subconjunto de **regiones de interés (ROIs)**, que han sido cuidadosamente seleccionadas y anotadas por expertos. En concreto, se proporcionan **4539 ROIs** extraídas de **387 WSIs**, correspondientes a **151 pacientes**.

Tabla 3.2: Resumen de la composición del dataset BRACS.

Elemento	Cantidad
WSIs totales	547
Pacientes (WSI)	189
ROIs totales	4539
WSIs con ROIs	387
WSIs sin ROIs	160
Pacientes (ROIs)	151

Como se recoge en la Tabla 3.2, existe una diferencia significativa entre el número de WSIs totales y aquellas que contienen ROIs anotadas. Esto se debe a que no todas las imágenes completas fueron utilizadas para la extracción de regiones de interés, introduciendo una ligera heterogeneidad en el conjunto de datos.

Esta dualidad entre WSIs y ROIs permite abordar el problema desde dos perspectivas distintas. Por un lado, las WSIs representan un escenario más cercano a la práctica clínica real, donde el modelo debe analizar imágenes completas de gran tamaño. Por otro lado, las ROIs facilitan el análisis al reducir la complejidad computacional, siendo el enfoque más habitual en tareas de clasificación supervisada.

Los creadores de este dataset proporcionan también su partición oficial en tres subconjuntos: **entrenamiento (train)**, **validación (valid)** y **evaluación (test)**.

Tabla 3.3: Distribución de WSIs por clase y subconjunto en BRACS.

Clase	Train	Val	Test
N	27	10	7
PB	120	11	16
UDH	56	9	9
FEA	24	6	11
ADH	28	8	12
DCIS	40	9	12
IC	100	12	20
TOTAL	395	65	87

Tabla 3.4: Distribución de ROIs por clase y subconjunto en BRACS.

Clase	Train	Val	Test
N	357	46	81
PB	714	43	79
UDH	389	46	82
FEA	624	49	83
ADH	387	41	79
DCIS	665	40	85
IC	521	47	81
TOTAL	3657	312	570

Tal y como se observa en las Tablas 3.3 y 3.4, el dataset presenta una distribución relativamente equilibrada entre las clases benignas y malignas. Sin embargo, las clases correspondientes a lesiones atípicas contienen un menor número de muestras, lo que introduce un cierto grado de desbalanceo.

Este aspecto resulta especialmente relevante, ya que estas clases intermedias son precisamente las más difíciles de discriminar, tanto desde el punto de vista clínico como

computacional. Como consecuencia, los modelos tienden a mostrar un peor rendimiento en estas categorías, lo que justifica el uso de métricas más robustas como el **F1-score** frente a la simple *accuracy*.

En el contexto de este proyecto, se respetará esta división, utilizando el conjunto de entrenamiento para el aprendizaje de los modelos, el conjunto de validación para el ajuste de hiperparámetros y el conjunto de test exclusivamente para la evaluación final del rendimiento.

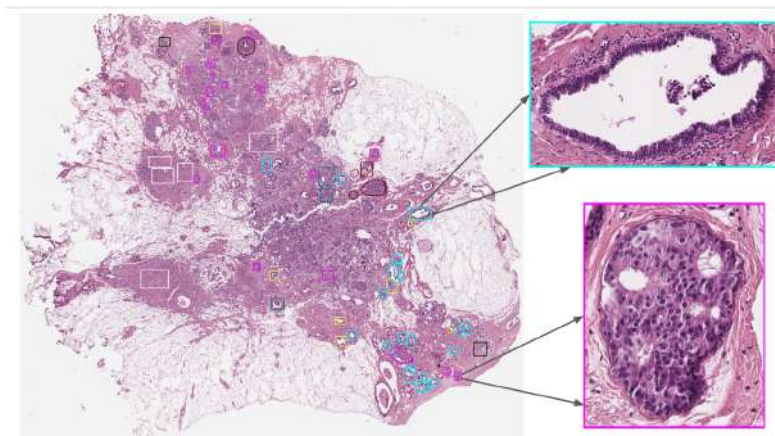


Figura 3.1: WSI del dataset BRACS junto con distintas ROIs

Como se puede observar en la Figura 3.1, mientras que la WSI contiene la totalidad de la muestra histológica, las ROIs representan áreas específicas seleccionadas por expertos donde se concentran patrones relevantes para el diagnóstico.

En definitiva, el dataset BRACS presenta una combinación de características que lo convierten en un banco de pruebas especialmente exigente: un número limitado de muestras, una alta variabilidad intra-clase y la presencia de categorías intermedias difíciles de distinguir. Estas propiedades lo sitúan como un escenario ideal para analizar la capacidad de generalización de modelos de *Deep Learning*.

3.1. Baseline oficial y dificultad del dataset

Para contextualizar adecuadamente la complejidad del dataset, resulta útil revisar los resultados de referencia publicados por los propios autores de BRACS [28]. Estos experimentos constituyen el primer *baseline* oficial asociado al dataset y permiten entender, desde el propio diseño original del conjunto de datos, qué tipos de lesiones resultan más sencillas o más difíciles de reconocer.

En su estudio, Brancati *et al.* evaluaron distintas arquitecturas basadas en Redes Neuronales Convolucionales (CNNs) sobre dos escenarios: una clasificación de imágenes completas (WSI) agrupadas en **3 clases** clínicas generales (Benigno, Atípico y Maligno), y una clasificación a nivel de **ROI** en las **7 subclases** diagnósticas específicas del dataset.

Los resultados globales obtenidos se recogen en la Tabla 3.5. Como puede observarse, el rendimiento del modelo fue razonable en el escenario de 3 clases, con valores próximos al 70 % en las principales métricas. Sin embargo, al pasar al problema fino de 7 clases a nivel de ROI, el rendimiento descendió de forma notable, situándose en torno al 55 % de *accuracy* y 54 % de *F1-score*. Esta caída ya anticipaba que BRACS no constituye únicamente un problema de clasificación convencional, sino un escenario especialmente exigente en las categorías intermedias.

Tabla 3.5: Resultados del baseline publicado por Brancati *et al.* (2022) sobre el dataset BRACS [4].

Métrica de evaluación	3 clases (WSI)	7 clases (ROI)
<i>Accuracy</i>	70.3 %	55.9 %
<i>Precision</i>	71.2 %	54.1 %
<i>Recall</i>	69.9 %	56.0 %
<i>F1-score</i>	69.8 %	54.3 %

Más allá de las métricas globales, los resultados por clase publicados en el artículo muestran con claridad dónde se concentra la dificultad del dataset. Las clases más extremas y morfológicamente más diferenciables, como el **Carcinoma Invasivo (IC)** o el tejido **Normal (N)**, alcanzaron valores de F1-score claramente superiores. En cambio, las lesiones intermedias o de transición presentaron un comportamiento mucho más problemático: la **Hiperplasia Ductal Usual (UDH)** obtuvo un F1-score de 34,3 %, mientras que la **Hiperplasia Ductal Atípica (ADH)** descendió hasta 24,0 %.

Esta distribución del error es especialmente relevante para el presente TFG, ya que pone de manifiesto desde el inicio que la dificultad principal de BRACS no radica tanto en distinguir los extremos benigno y maligno, sino en separar correctamente las lesiones atípicas y las categorías de transición. En otras palabras, el propio *baseline* oficial ya sugería que el problema exige modelos capaces de capturar contexto global y relaciones morfológicas finas, algo que justifica el interés por estudiar arquitecturas más modernas y representaciones más ricas a lo largo de este trabajo.

Capítulo 4

Metodología y Planificación

4.1. Enfoque metodológico

El desarrollo de este proyecto no siguió una planificación rígida y cerrada desde el inicio, sino que fue una metodología de trabajo **iterativa, experimental y guiada por resultados**. A medida que se obtuvieron resultados y evidencias en cada fase, se fueron tomando las decisiones metodológicas más oportunas, definiendo así los experimentos llevados a cabo y el alcance de todos los bloques del proyecto.

Esta forma de trabajo resultó especialmente adecuada para un proyecto experimental como este, en el que muchas decisiones dependen directamente de los resultados obtenidos en la fase anterior. Frente a enfoques más rígidos, esta aproximación permitió reajustar objetivos, incorporar análisis complementarios y reconducir experimentos cuando las evidencias así lo aconsejaban. No obstante, también implicó una mayor necesidad de seguimiento, documentación continua y control de versiones para evitar perder trazabilidad en la evolución del proyecto.

4.2. Tecnologías y herramientas utilizadas

El desarrollo del proyecto se apoyó en un conjunto de tecnologías orientadas tanto a la implementación experimental como a la trazabilidad y documentación del trabajo.

El lenguaje principal empleado fue **Python**, debido a su ecosistema consolidado en *Machine Learning*, visión por computador y análisis de datos. Su flexibilidad y la disponibilidad de bibliotecas especializadas permitieron integrar en un mismo entorno la preparación de datos, el entrenamiento de modelos, la evaluación de resultados y la generación de figuras y tablas.

La Tabla 4.1 resume las principales bibliotecas y herramientas de software utilizadas, junto con su función dentro del flujo experimental:

Tabla 4.1: Principales bibliotecas, plataformas y herramientas utilizadas en el proyecto.

Tecnología / biblioteca	Uso principal
Python	Lenguaje principal del proyecto y base de todo el flujo experimental.
PyTorch	Implementación y entrenamiento de modelos de <i>Deep Learning</i> , así como uso de cabezas lineales sobre <i>embeddings</i> .
Hugging Face	Acceso a modelos fundacionales preentrenados y a sus pesos, facilitando la extracción reproducible de <i>embeddings</i> .
NumPy	Operaciones numéricas y manipulación eficiente de vectores y matrices.
pandas	Gestión tabular de resultados, métricas y ficheros CSV.
scikit-learn	Cálculo de métricas, evaluación de clasificadores y utilidades de análisis.
imbalanced-learn	Aplicación de estrategias de reducción y reequilibrado del conjunto de entrenamiento.
h5py	Lectura y escritura de <i>embeddings</i> en formato HDF5.
Matplotlib	Generación de figuras, curvas y matrices de confusión.
MLflow	Registro de experimentos, métricas, artefactos y modelos entrenados.
Git / GitHub	Control de versiones, trazabilidad del código y gestión del repositorio.
Overleaf	Redacción, revisión y mantenimiento de la memoria en $\text{\LaTeX}$.
Servidor UGR	Ejecución de experimentos de mayor coste computacional que los viables en local.

De forma complementaria, se utilizaron **herramientas de inteligencia artificial generativa** como apoyo en tareas auxiliares, como la resolución de dudas técnicas y de implementación, la mejora de redacción, la reformulación de textos, la creación de imágenes para la memoria y la organización de contenidos.

Las principales herramientas empleadas fueron *ChatGPT* [29], desarrollado por OpenAI, y *Gemini* [30], desarrollado por Google.

Estas herramientas **no han sustituido la aportación personal del autor en**

los aspectos esenciales del proyecto. Tanto la concepción del problema, como la definición de objetivos, el diseño metodológico, la implementación del sistema, la ejecución de experimentos, la interpretación de resultados y la elaboración de conclusiones corresponden íntegramente al autor de la memoria.

Por tanto, la utilización de inteligencia artificial se ha limitado a un papel de asistencia, manteniéndose en todo momento la responsabilidad plena del autor sobre el contenido, la validez técnica y la autoría final del trabajo presentado.

4.3. Planificación y seguimiento del proyecto

Como se puede observar en la Figura 4.1 y en la Tabla 4.2, la planificación del trabajo se estructuró en distintas fases, cada una orientada a una pregunta concreta del proyecto y condicionando la siguiente.

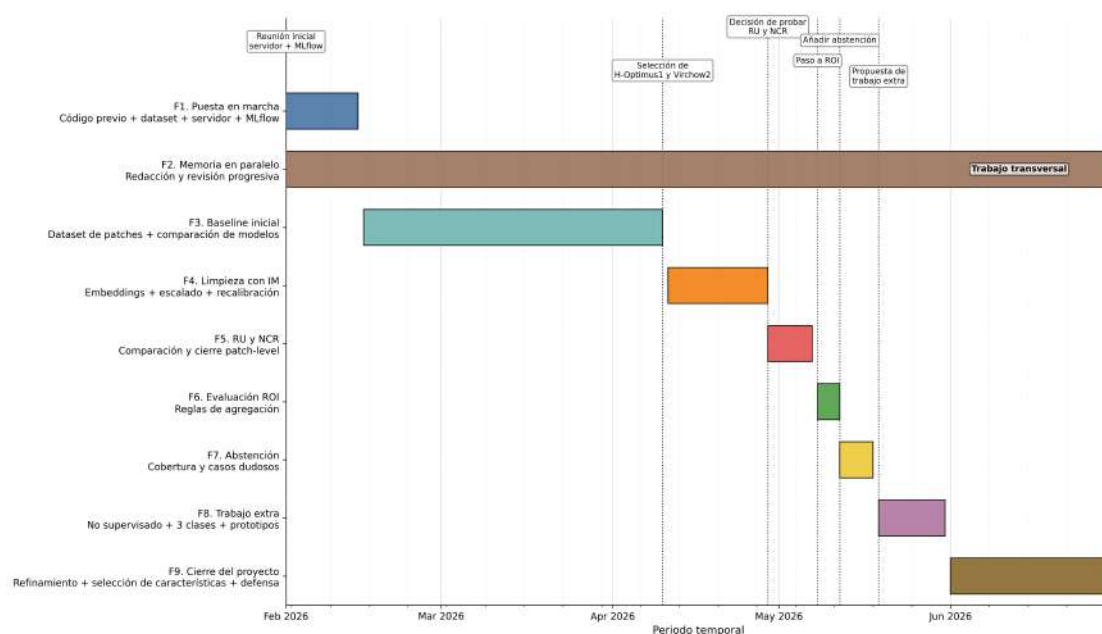


Figura 4.1: Cronograma de desarrollo del TFG por fases e hitos de seguimiento

Tabla 4.2: Planificación por fases de trabajo

Fase	Actividades principales	Resultado / decisión
F1 (OBJ1.2)	Revisión del código previo, comprensión del dataset BRACS, definición del entorno de trabajo y organización inicial del <i>pipeline</i> .	Se usó el TFM de la mentora como base y para extraer el dataset a nivel de <i>patches</i> . Se decidió trabajar en servidor y usar MLflow.
F2 (<i>Trans- versal</i>)	Redacción y revisión progresiva de la memoria, en paralelo al desarrollo experimental.	La memoria se fue construyendo de forma incremental, permitiendo documentar resultados conforme se iban consolidando.
F3 (OBJ1.3, OBJ1.8)	Construcción del dataset de patches, validación del pipeline y baseline inicial con distintas familias de modelos.	Se seleccionaron los modelos fundacionales H-Optimus1 y Virchow2 como línea principal.
F4 (OBJ1.4, OBJ1.8)	Extracción de <i>embeddings</i> y primera fase de limpieza de patches mediante información mutua, recalibración progresiva de parámetros.	Se consolidó un primer pipeline de limpieza y se observó la necesidad de comparar métodos con igual agresividad de reducción.
F5 (OBJ1.4, OBJ1.8)	Comparación entre información mutua, RandomUnderSampler y NCR recalibrado.	Se cerró la etapa a nivel de patches y se decidió avanzar con la evaluación a nivel de ROI.
F6 (OBJ1.5, OBJ1.8)	Predicciones en ROIs y comparación entre reglas de votación.	Se identificó <i>mean_proba</i> como la mejor regla de agregación.
F7 (OBJ1.6, OBJ1.8)	Incorporación de una regla de abstención para ROIs dudosas, análisis de cobertura y estudio de casos representativos.	Se fijó una estrategia de revisión basada en incertidumbre y se confirmó que las dudas se concentraban en clases intermedias.
F8 (OBJ1.7, OBJ1.8)	Trabajo extra: análisis no supervisado del espacio latente, proyección a 3 clases y análisis mediante prototipos claros.	Se reforzó la hipótesis de que parte de la dificultad del problema es estructural y no depende solo del clasificador.
F9 (OBJ1.7, OBJ1.8)	Revisión final de la memoria, refinamiento del trabajo, análisis complementario de selección de características y preparación de la defensa.	Fase de cierre del proyecto, orientada a consolidar resultados, pulir la memoria y dejar definida la versión final del TFG.

4.3.1. Seguimiento y coordinación

Tal y como se muestra en la Tabla 4.3, el desarrollo del proyecto estuvo acompañado de un seguimiento continuo mediante reuniones, correos electrónicos y revisión progresiva tanto del código como de la memoria.

Tabla 4.3: Principales mecanismos de seguimiento y coordinación del proyecto

Medio	Finalidad principal
Reuniones presenciales y videollamadas (<i>Google Meet</i>)	Discusión de resultados, revisión del avance, validación de decisiones metodológicas y definición de siguientes pasos.
Correo electrónico	Resumen de avances, planteamiento de dudas, propuesta de nuevas pruebas y registro de decisiones entre reuniones.
GitHub y Overleaf	Revisión progresiva del código y de la memoria, facilitando el trabajo iterativo y la trazabilidad de cambios.

4.3.2. Dedicación temporal estimada

La dedicación temporal se estimó mediante la herramienta **Clockify**, utilizada como registro orientativo de las horas de trabajo invertidas a lo largo del desarrollo del TFG. A partir de este seguimiento, la carga total acumulada se situó en torno a **400 horas**.

Este número de horas comprende la ejecución de experimentos, la lectura de bibliografía, el análisis de resultados, la redacción de la memoria, la revisión del documento y la preparación final del proyecto. Aunque la distribución del tiempo no fue uniforme entre fases, la mayor dedicación se concentró en el desarrollo experimental y en la documentación y análisis de los resultados.

4.4. Desviaciones, dificultades y reflexión sobre la planificación

Aunque la planificación por fases permitió mantener una estructura clara, el desarrollo real del proyecto estuvo condicionado por varias dificultades propias de un trabajo experimental. Una de las más relevantes fue comprobar que algunos resultados que inicialmente parecían prometedores dejaban de serlo al ampliar el análisis o al imponer comparaciones más justas entre métodos. Esto obligó a rehacer parte del protocolo varias veces y a no dar por buenas conclusiones demasiado pronto.

Otra dificultad importante fue el ajuste de ciertos experimentos especialmente sensibles a escala, como ocurrió en la limpieza de *patches* mediante Información Mutua. En este tipo de pruebas, pequeños cambios en el tamaño del conjunto o en los parámetros cambiaban por completo el comportamiento del método, lo que obligó a dedicar más tiempo del previsto a entender el problema antes de poder extraer conclusiones sólidas.

En conjunto, estas desviaciones no impidieron cumplir los objetivos del proyecto, pero sí confirmaron que, en un TFG de estas características, una planificación útil no consiste en seguir un guion rígido, sino en saber reinterpretar los resultados, corregir decisiones previas y adaptar el trabajo, manteniendo siempre el objetivo principal como referencia.

4.5. Presupuesto estimado del proyecto

Este Trabajo de Fin de Grado se ha desarrollado en un contexto académico y no ha supuesto un coste económico directo. Sin embargo, resulta útil realizar una estimación orientativa del valor de los recursos humanos y técnicos empleados. Por ello este presupuesto no debe entenderse como un gasto real, sino como una aproximación al coste que tendría reproducir un proyecto de características similares en un entorno profesional.

Para esta estimación se han considerado tanto las horas de dedicación personal registradas mediante *Clockify* como el tiempo de seguimiento del profesorado, el uso de equipo personal y la utilización de infraestructura computacional. El resumen del presupuesto estimado se muestra en la Tabla 4.4.

Tabla 4.4: Presupuesto estimado del proyecto.

Concepto	Tiempo	€/unidad	Coste (€)
Trabajo de desarrollo, experimentación y documentación del TFG	400 horas	16,5 €/hora ²	6600 €
Uso de equipo personal ³	5 meses	25 €/mes	125 €
Dedicación de los tutores al seguimiento del TFG	20 h × (2 tutores + 1 mentora)	50 €/hora	3000 €
Uso de infraestructura computacional / servidor	Estimación global	—	300 €
Total estimado			10.025 €

Como puede observarse, la mayor parte del presupuesto estimado corresponde al trabajo de desarrollo, experimentación y documentación, que constituye el núcleo principal del proyecto. El resto es una orientación de los recursos materiales y de supervisión

²Valor orientativo tomado como referencia académica para estimar el coste de dedicación personal en el contexto de un Trabajo de Fin de Grado. No representa una tarifa profesional de mercado, sino una aproximación razonable al valor del trabajo técnico desarrollado en un entorno formativo.

³Equipo personal utilizado principalmente para desarrollo, edición del código, redacción de la memoria y pruebas locales: MacBook Pro de 16,2 pulgadas con chip Apple M4 Pro. Las ejecuciones experimentales de mayor coste computacional se realizaron sobre el servidor de trabajo proporcionado por la Universidad de Granada.

necesarios para llevar a cabo un proyecto de estas características. En consecuencia, el coste total estimado del proyecto es de **10.025 €**.

Capítulo 5

Experimentación y Resultados

5.1. Baseline inicial

5.1.1. Descripción del protocolo experimental

Para abordar los objetivos **OBJ1.3** y **OBJ1.8**, en esta primera fase experimental se definió un protocolo de comparación entre distintas familias de modelos, con el fin de identificar las configuraciones más sólidas sobre el problema de clasificación *patch-level*.

La comparación sirve como base para construir un ranking de modelos y seleccionar posteriormente los modelos más prometedores, sobre los que se realizó la fase de optimización y mejora.

En esta etapa inicial, todos los experimentos se realizaron **exclusivamente a nivel de *patches***. Es decir, cada muestra de entrada al modelo es un parche individual extraído de una ROI del dataset BRACS.

Conjunto de datos

Se utilizó la partición oficial del dataset, distinguiendo entre los subconjuntos de (**train**), (**val**) y (**test**).

Durante esta fase de *baseline*:

- el conjunto de **train** se utiliza para ajustar los parámetros del modelo,
- el conjunto de **val** se utiliza para comparar modelos y estimar su capacidad de generalización,
- el conjunto de **test** permanece completamente aislado y no se utiliza en ningún momento.

De este modo, el conjunto de test se reservó como evaluación final del mejor modelo seleccionado, evitando así cualquier sesgo durante la fase de selección de arquitecturas o ajuste de hiperparámetros.

Estrategia de validación

Para evitar que la comparación entre modelos dependiera de una única ejecución concreta, cada experimento se repitió con cinco semillas aleatorias diferentes: **13, 29, 47, 71 y 101**.

En consecuencia, cada combinación de arquitectura y configuración experimental se entrenó cinco veces, manteniendo constantes todos los hiperparámetros salvo la semilla. Esto permite además estimar su estabilidad frente a variaciones aleatorias en la inicialización, el orden de los *batches* y las transformaciones aplicadas durante el entrenamiento.

La comparación entre modelos se realizó, por tanto, a partir de:

- La media de las métricas obtenidas en validación sobre las cinco semillas.
- Su desviación estándar.

Se abordó una sola configuración del problema:

1. Clasificación en 7 clases, por tratarse del problema más exigente y representativo.

Se compararon tres grandes familias de modelos: Redes convolucionales clásicas, Vision Transformers estándar y modelos fundacionales preentrenados en histopatología.

Redes convolucionales

Se seleccionaron dos arquitecturas convolucionales representativas, priorizando aquellas que han mostrado buen comportamiento previo en trabajos anteriores relacionados con BRACS:

- ResNet18,
- ResNet50,

Vision Transformers

Se seleccionaron dos arquitecturas basadas en *transformers* para visión:

- ViT-B/16,
- DeiT,

Modelos fundacionales

Se evaluaron tres modelos fundacionales especializados en histopatología, obtenidos de la página oficial de *Hugging Face*:

- Phikon (público),
- Virchow2 (público),
- H-Optimus1 (acceso aceptado por la propia organización *Hugging Face*).

Condiciones de comparación

Para garantizar que la comparación entre modelos fuera justa, se mantuvo el mismo protocolo experimental dentro de cada fase.

En particular, se mantuvieron constantes:

- **Dataset utilizado:** Dataset de patches extraídos de las RoI del dataset BRACS, utilizando exclusivamente los conjuntos oficiales de entrenamiento y validación.
- **Número de épocas:** 10.
- **Batch size:** 8.
- **Learning rate inicial:** 3×10^{-4} .
- **Tipo de optimizador:** AdamW.
- **Política de scheduler:** Cosine.
- **Función de pérdida:** Cross-entropy.
- **Transformaciones:** Ninguna.
- **Normalización:** Ninguna.

De este modo, el factor principal que cambió en cada experimento fue la arquitectura evaluada y la semilla.

Métrica principal de comparación

La métrica principal utilizada para construir el ranking de modelos es el **F1-score macro sobre el conjunto de validación**.

El F1-score macro permite asignar el mismo peso a todas las clases, proporcionando una evaluación más equilibrada del modelo.

La elección de esta métrica se debe a que el problema presenta desbalanceo entre clases, por lo que una métrica como la *accuracy* podría sobreestimar el rendimiento en clases mayoritarias y no reflejar adecuadamente el comportamiento sobre clases menos representadas.

Métricas secundarias

Además del F1 macro, se recogieron también otras métricas, para complementar el análisis y obtener una visión más rica de cada modelo:

- *accuracy* en validación,
- F1-score weighted,
- precisión macro

- *recall* macro,
- matriz de confusión,
- métricas por clase,
- tiempo de entrenamiento.

Selección del modelo ganador

Una vez ejecutados todos los experimentos previstos, se construyó un ranking de modelos basado en:

- la media del F1 macro en validación,
- la desviación estándar entre semillas,
- y el análisis cualitativo de matrices de confusión y métricas por clase.

Los modelos ganadores fueron aquellos que presentaron el mejor equilibrio entre rendimiento medio, estabilidad, comportamiento en clases minoritarias y viabilidad computacional.

Dichos candidatos fueron los que avanzaron para las siguientes fases del proyecto.

Trazabilidad y reproducibilidad experimental

Todos los experimentos se registraron mediante **MLflow**, almacenando para cada ejecución:

- la configuración completa del experimento,
- los hiperparámetros de entrenamiento,
- la semilla utilizada,
- las métricas por época,
- la mejor época alcanzada,
- las matrices de confusión,
- los informes de clasificación,
- y los pesos del mejor modelo.

Este sistema de trazabilidad permite comparar experimentos de forma ordenada, reproducir resultados y documentar rigurosamente el proceso de selección del modelo final.



Figura 5.1: Ejemplo de seguimiento experimental mediante MLflow.

La Figura 5.1 muestra una captura de cómo MLflow organiza distintas ejecuciones de un mismo experimento, registrando de forma estructurada tanto los parámetros de entrenamiento como las métricas alcanzadas por cada *run*, facilitando la comparación de distintas configuraciones, la identificación de los mejores resultados y el mantenimiento de una trazabilidad completa del proceso experimental.

5.1.2. Resultados

CNNs clásicas

Tabla 5.1: Resultados de ResNet18 en el baseline

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.5164	0.7030	5
29	0.4962	0.6746	3
47	0.4908	0.6686	6
71	0.4929	0.6865	4
101	0.4811	0.6810	9
Media	0.4955	0.6827	
Desv. típica	0.0130	0.0131	

Los resultados obtenidos con **ResNet18** muestran un comportamiento estable entre semillas, con una media de 0,4955 en F1 macro y 0,6827 en *accuracy* sobre validación. La

Tabla 5.2: Resultados de ResNet50 en el baseline

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.5096	0.6968	8
29	0.4899	0.6728	7
47	0.4891	0.6970	9
71	0.5133	0.6999	10
101	0.4785	0.6757	6
Media	0.4961	0.6884	
Desv. típica	0.0136	0.0116	

desviación típica en ambas métricas es reducida, lo que indica una variabilidad moderada entre ejecuciones y sugiere que la arquitectura ofrece un rendimiento consistente.

No obstante, el análisis de la evolución durante el entrenamiento muestra indicios claros de **sobreajuste**: mientras las métricas de entrenamiento continuaban mejorando de forma pronunciada, las métricas de validación tendían a estabilizarse o incluso empeorar a partir de épocas relativamente tempranas. Este comportamiento resulta coherente con la naturaleza del *baseline* planteado, en el que no se incorporaron ni técnicas de normalización ni estrategias de *data augmentation*. En consecuencia, **ResNet18** constituye una línea base sólida y reproducible, pero todavía susceptible de mejora en fases posteriores del proyecto.

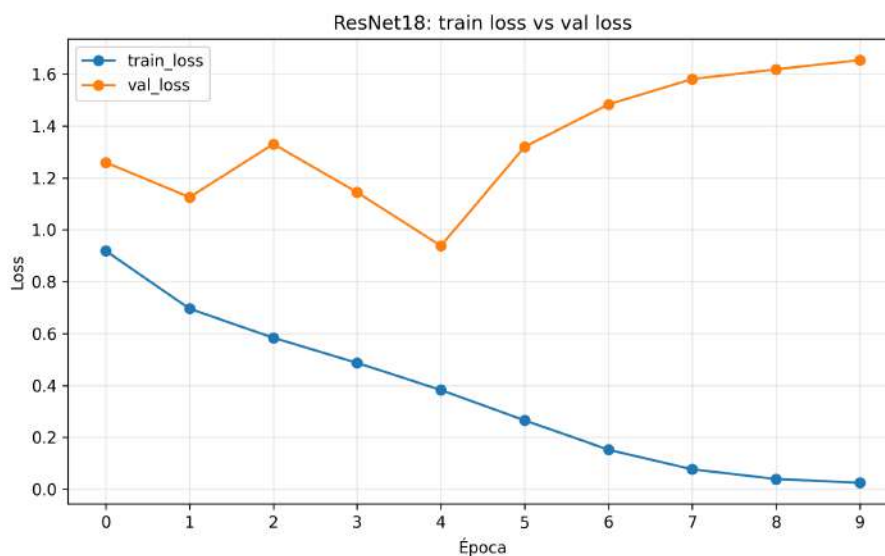


Figura 5.2: Evolución de la pérdida en entrenamiento y validación para ResNet18

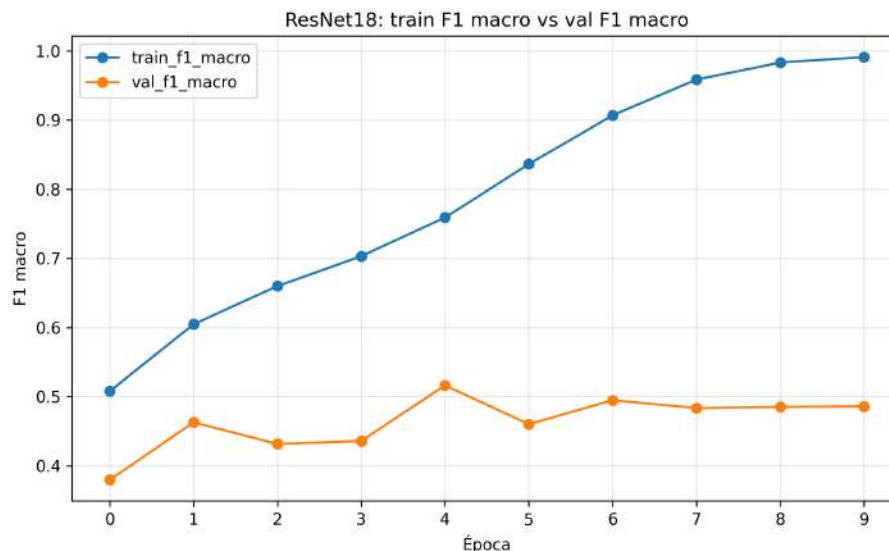


Figura 5.3: Evolución del F1 macro en entrenamiento y validación para ResNet18

Las Figuras 5.2 y 5.3 permiten visualizar de forma clara este fenómeno: la mejora sostenida en entrenamiento no se traduce en una mejora equivalente sobre validación, lo que confirma la existencia de sobreajuste en esta primera configuración *baseline*.

Por otra parte, los resultados obtenidos con **ResNet50** muestran un comportamiento global muy similar al de **ResNet18** en términos de F1 macro medio sobre validación, aunque con una ligera mejora en *accuracy* media. En concreto, ResNet50 alcanzó una media de 0,4961 en F1 macro y 0,6884 en *accuracy*, con desviaciones típicas reducidas entre semillas, lo que indica una estabilidad razonable del modelo bajo el protocolo experimental establecido.

En comparación con ResNet18, la mejora en F1 macro medio es prácticamente despreciable, mientras que la mejora en *accuracy* media resulta ligera. Esto sugiere que el aumento de profundidad y capacidad representacional introducido por ResNet50 no se traduce, en este *baseline* sin normalización ni *data augmentation*, en una mejora clara sobre clases minoritarias. Además, el coste computacional de ResNet50 fue notablemente superior, con tiempos de entrenamiento superiores a los observados en ResNet18.

Por tanto, los resultados apuntan a que, dentro de este *baseline*, **ResNet50** no ofrece una ventaja suficientemente marcada sobre **ResNet18** como para justificar por sí sola el incremento de coste computacional.

Vision Transformers

Los resultados obtenidos con **DeiT-Small** muestran un rendimiento claramente inferior al alcanzado por las arquitecturas convolucionales evaluadas previamente. En concreto, el modelo alcanzó una media de 0,3836 en F1 macro y 0,5342 en *accuracy* sobre validación, con una variabilidad relativamente reducida entre semillas. Este comporta-

Tabla 5.3: Resultados de DeiT-Small en el baseline

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.3783	0.5300	8
29	0.3903	0.5426	9
47	0.3946	0.5487	8
71	0.3758	0.5144	7
101	0.3793	0.5351	5
Media	0.3836	0.5342	
Desv. típica	0.0074	0.0117	

miento indica que, bajo el protocolo *baseline* definido, **DeiT-Small** ofrece una respuesta estable pero claramente menos competitiva que **ResNet18** y **ResNet50**.

Una observación relevante es que el modelo mostró una tendencia a seguir mejorando en validación en varias de las últimas épocas, tal y como puede apreciarse en la Figura 5.4. Esto sugiere que esta familia de arquitecturas podría requerir un entrenamiento más largo para explotar mejor su capacidad representacional. Sin embargo, en esta fase del *baseline* se mantuvo deliberadamente el mismo número de épocas para todos los modelos, con el fin de preservar una comparación homogénea y justa entre arquitecturas.

Como segundo modelo dentro de la familia de *Vision Transformers* se evaluó la arquitectura **ViT-B/16**.

Dado que este tipo de arquitecturas preentrenadas trabaja habitualmente con imágenes de menor tamaño que las CNN utilizadas previamente, los *patches* se redimensionaron a **224 × 224 píxeles** antes de su entrada al modelo.

Tabla 5.4: Resultados de ViT-B/16 en el baseline

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.3632	0.5050	9
29	0.2052	0.3811	9
47	0.3616	0.5122	7
71	0.3588	0.5105	10
101	0.3565	0.4988	10
Media	0.3291	0.4815	
Desv. típica	0.0693	0.0564	

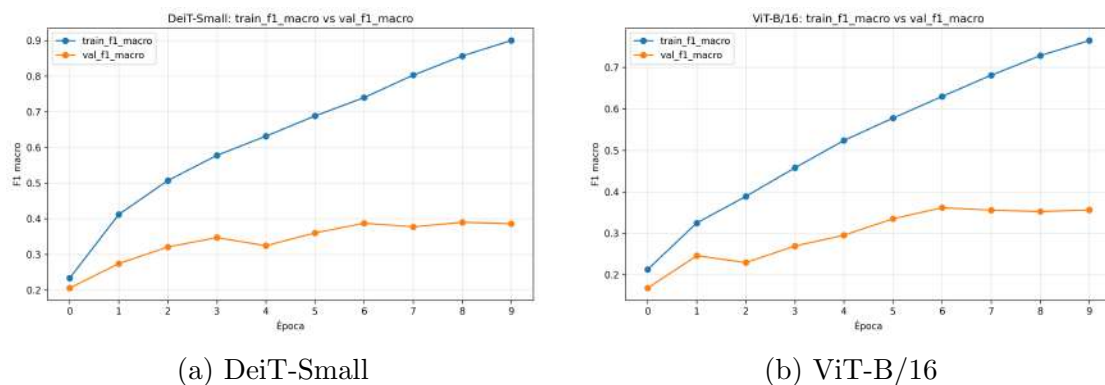


Figura 5.4: Curvas de *train* y *val* para los *ViTs* en el *baseline*

Los resultados obtenidos con **ViT-B/16** muestran un rendimiento inferior al observado tanto en las arquitecturas convolucionales como en **DeiT-Small**. En promedio, el modelo alcanzó un **F1 macro de 0.3291** y una **accuracy de 0.4815** sobre validación, con una desviación típica relativamente elevada en ambas métricas. Esta variabilidad está fuertemente condicionada por la ejecución correspondiente a la semilla 29, cuyo comportamiento fue claramente peor que el del resto de semillas.

Si se observan las demás ejecuciones, **ViT-B/16** presenta un comportamiento bastante consistente alrededor de valores de F1 macro próximos a 0.36 y *accuracies* cercanas a 0.50. Sin embargo, incluso en ese escenario, su rendimiento sigue siendo claramente inferior al de **DeiT-Small** y, más aún, al de las dos arquitecturas CNN evaluadas previamente. Esto sugiere que, bajo el protocolo *baseline* definido —sin normalización, sin *data augmentation* y con un número limitado de épocas—, **ViT-B/16** no consigue aprovechar adecuadamente su mayor capacidad representacional.

Además, las curvas de entrenamiento y validación refuerzan esta lectura, ya que muestran que, aun manteniendo una evolución razonablemente estable, el modelo no llega a consolidar una mejora competitiva dentro del número de épocas fijado para este *baseline* (Figura 5.4).

En consecuencia, este resultado refuerza la idea de que los *Vision Transformers* más grandes pueden requerir condiciones de entrenamiento más favorables para desplegar todo su potencial, tales como mayor número de épocas, estrategias de regularización más adecuadas o configuraciones de preprocesamiento más alineadas con su preentrenamiento. Dentro del *baseline* actual, **ViT-B/16** no se posiciona como una alternativa competitiva frente a los modelos ya evaluados.

Modelos Fundacionales

Como primer modelo dentro del bloque de modelos fundacionales se evaluó **Phikon**. A diferencia de las CNN y de los *Vision Transformers* convencionales, en esta primera fase los modelos fundacionales se emplearon como **extractores de características congelados**, añadiendo únicamente una cabeza lineal entrenable para la clasificación

final. Esta estrategia permite evaluar directamente la calidad de las representaciones aprendidas durante el preentrenamiento, reduciendo al mismo tiempo el coste computacional y la complejidad del ajuste fino completo.

Tabla 5.5: Resultados de Phikon en el baseline

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.5704	0.7597	8
29	0.5865	0.7641	4
47	0.5818	0.7626	7
71	0.5607	0.7622	2
101	0.5669	0.7681	6
Media	0.5733	0.7633	
Desv. típica	0.0095	0.0028	

Los resultados obtenidos con **Phikon** muestran un comportamiento claramente superior al del resto de modelos evaluados hasta este punto del *baseline*. En promedio, el modelo alcanzó un **F1 macro de 0.5733** y una **accuracy de 0.7633** sobre validación, con desviaciones típicas reducidas en ambas métricas. Esto indica no solo un rendimiento elevado, sino también una gran estabilidad entre semillas.

En comparación con las arquitecturas convolucionales y con los *Vision Transformers* estándar, **Phikon** consigue mejorar de manera clara tanto el F1 macro como la *accuracy*. Este resultado sugiere que las representaciones aprendidas por el modelo durante su preentrenamiento en imágenes histopatológicas son altamente transferibles a la tarea planteada en BRACS, incluso sin necesidad de realizar *fine-tuning* completo del *backbone*. De hecho, el modelo ha sido evaluado en esta primera fase únicamente como extractor de características congelado con una cabeza lineal entrenable, lo que refuerza aún más el valor de sus representaciones.

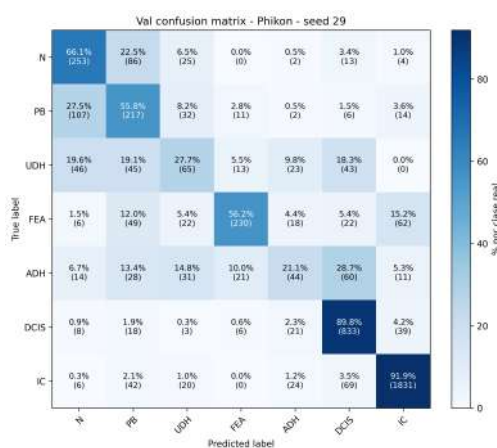


Figura 5.5: Matriz de confusión en validación para Phikon en la semilla 29

La Figura 5.5 permite comprobar que el buen rendimiento global de **Phikon** no se debe únicamente a una mejora en las clases más sencillas, sino a un comportamiento bastante sólido en la mayoría de categorías. Como era esperable, las principales confusiones siguen concentrándose en las clases intermedias, mientras que las clases más extremas muestran diagonales algo más marcadas.

En consecuencia, **Phikon** se posiciona como un candidato muy sólido dentro del *baseline* y constituye, hasta el momento, la referencia más fuerte del estudio. Este comportamiento respalda la hipótesis inicial de que los modelos fundacionales específicamente preentrenados en histopatología pueden superar a las arquitecturas genéricas cuando se aplican a tareas *downstream* del mismo dominio.

Como segundo modelo dentro del bloque de modelos fundacionales se evaluó **Virchow2**. Al igual que en el caso de Phikon, en esta primera fase del *baseline* el modelo se empleó como **extractor de características congelado**, entrenando únicamente una cabeza lineal para la clasificación final.

Tabla 5.6: Resultados de Virchow2 en el baseline

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.5973	0.7864	7
29	0.6118	0.7993	1
47	0.5970	0.7897	5
71	0.5976	0.7804	6
101	0.6019	0.7901	2
Media	0.6011	0.7892	
Desv. típica	0.0056	0.0061	

Los resultados obtenidos con **Virchow2** muestran un rendimiento muy alto y claramente competitivo dentro del *baseline*. En promedio, el modelo alcanzó un **F1 macro de 0.6011** y una **accuracy de 0.7892** sobre validación, con desviaciones típicas reducidas en ambas métricas. Esto indica no solo un rendimiento elevado, sino también una gran estabilidad entre semillas.

En comparación con los modelos evaluados previamente, **Virchow2** se sitúa claramente por encima de las CNN y de los *Vision Transformers* estándar, y supera también a **Phikon**. Esta observación resulta especialmente relevante teniendo en cuenta que, en esta fase, el *backbone* se ha mantenido congelado y únicamente se ha entrenado una cabeza lineal, lo que indica que las representaciones preentrenadas por Virchow2 son altamente informativas para la tarea de clasificación en BRACS.

En consecuencia, **Virchow2** se posiciona como uno de los modelos más sólidos del *baseline* y como uno de los principales candidatos para la siguiente fase del proyecto.

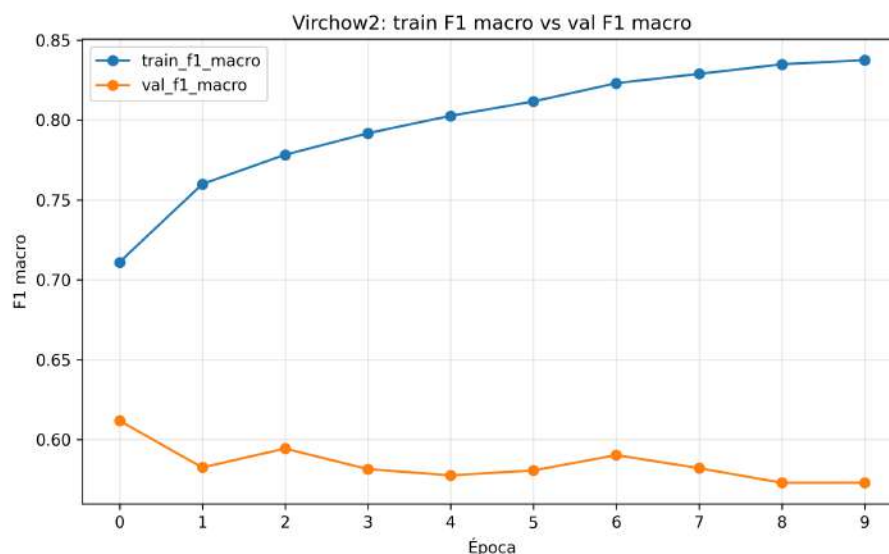


Figura 5.6: Evolución del F1 macro en entrenamiento y validación para Virchow2 en una de las semillas evaluadas

La Figura 5.6 refuerza visualmente la competitividad de Virchow2 dentro del *baseline*, mostrando una evolución del F1 macro en validación claramente superior a la observada en los modelos convencionales evaluados previamente.

Como último modelo dentro del bloque de modelos fundacionales se incorporó **H-Optimus1**.

Tabla 5.7: Resultado de H-Optimus1 en el *baseline*

Semilla	Mejor Val F1 macro	Mejor Val Accuracy	Época
13	0.6191	0.8046	1
29	0.6025	0.7969	5
47	0.6039	0.7941	1
71	0.6153	0.7985	3
101	0.6182	0.8048	1
Media	0.6118	0.7998	
Desv. típica	0.0080	0.0048	

Los resultados obtenidos con **H-Optimus1** muestran el mejor rendimiento global de todo el *baseline*. En promedio, el modelo alcanzó un **F1 macro de 0.6118** y una **accuracy de 0.7998** sobre validación, con desviaciones típicas reducidas en ambas métricas. Esto indica no solo un rendimiento elevado, sino también una estabilidad notable entre semillas.

En consecuencia, H-Optimus1 supera a las CNNs, a los *Vision Transformers* y también a los demás modelos fundacionales considerados.

5.1.3. Ranking

Tabla 5.8: Ranking de modelos del baseline inicial

Puesto	Modelo	Val F1 macro	Val Accuracy
1	H-Optimus1	0.6118 ± 0.0080	0.7998 ± 0.0048
2	Virchow2	0.6011 ± 0.0056	0.7892 ± 0.0061
3	Phikon	0.5733 ± 0.0095	0.7633 ± 0.0028
4	ResNet50	0.4961 ± 0.0136	0.6884 ± 0.0116
5	ResNet18	0.4955 ± 0.0130	0.6827 ± 0.0131
6	DeiT-Small	0.3836 ± 0.0074	0.5342 ± 0.0117
7	ViT-B/16	0.3291 ± 0.0693	0.4815 ± 0.0564

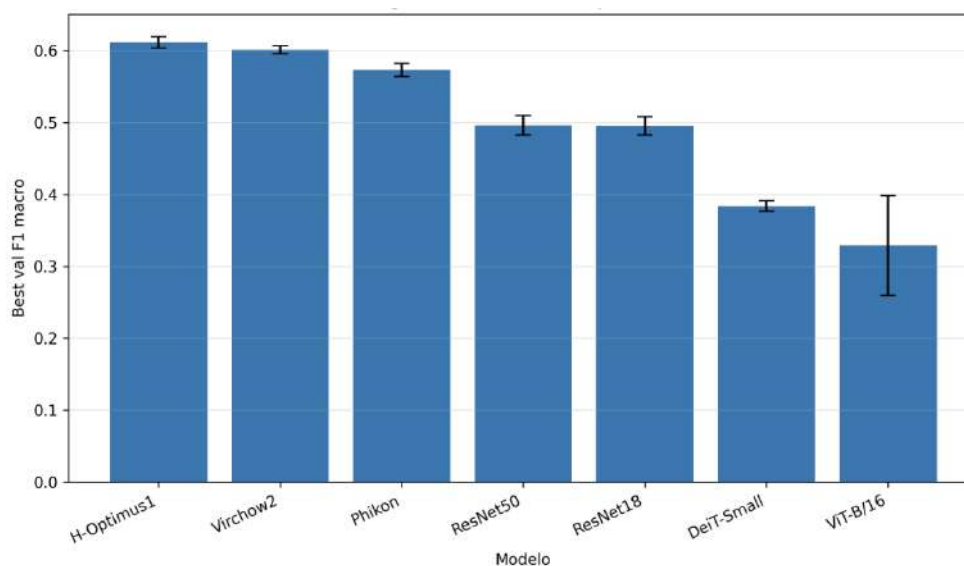


Figura 5.7: Representación del ranking final del baseline inicial según F1 macro sobre validación.

Tanto la tabla 5.8 como la figura 5.7 resumen el resultado final del *baseline* realizado sobre la tarea de clasificación en 7 clases en BRACS. Los resultados muestran de forma clara que, en estas condiciones, los **modelos fundacionales** superan notoriamente tanto a las CNN clásicas como a los *Vision Transformers* estándar evaluados en este trabajo. En particular, **H-Optimus1** se sitúa como el modelo con mejor rendimiento global, seguido muy de cerca por **Virchow2**, mientras que **Phikon** ocupa la tercera posición.

Estos resultados confirman la hipótesis inicial de que los modelos específicamente preentrenados en histopatología ofrecen representaciones mucho más transferibles para tareas dentro del mismo dominio. Además, el hecho de que este comportamiento se haya

obtenido empleando los modelos fundacionales únicamente como extractores de características congelados refuerza aún más el valor de sus representaciones preentrenadas.

A partir de este punto, y una vez completado el *baseline* inicial, se decidió continuar el desarrollo del proyecto únicamente con los dos modelos ganadores: **H-Optimus1** y **Virchow2**.

5.2. Limpieza de patches mediante Información Mutua

5.2.1. Motivación y fundamento metodológico

Una vez establecido el primer *baseline* experimental sobre el conjunto completo de patches, se observó que los modelos fundacionales **H-Optimus-1** y **Virchow2** ofrecían el mejor rendimiento entre todas las arquitecturas evaluadas. A partir de este punto, el trabajo experimental se orienta hacia la mejora de estos dos modelos, manteniendo fijo el esquema de entrenamiento base y actuando sobre la calidad del dataset.

Con el fin de abordar el objetivo **OBJ1.4**, en esta fase se plantea una etapa adicional de **depuración del dataset de patches**, que no se trata de una simple eliminación de ejemplos atípicos, sino como un proceso de identificación de muestras poco informativas o inconsistentes desde el punto de vista de la representación aprendida por cada modelo fundacional.

La referencia metodológica tomada para esta fase es el trabajo de Guillén, Herrera *et al.* [31], donde se propone un método de selección de prototipos basado en **información mutua**. Dicho trabajo toma la selección de muestras como una etapa de preprocesado siguiendo un enfoque decremental: se parte del conjunto completo y se evalúa, para cada prototipo, el efecto que produce su eliminación sobre la dependencia entre entradas y salida. La decisión final no se toma de forma aislada, sino comparando la pérdida de información asociada a cada muestra con la de sus vecinos más próximos.

No obstante, el contexto original de dicho artículo se distingue del problema abordado en este proyecto en varios aspectos relevantes. En primer lugar, el método fue propuesto para problemas de **regresión con salida continua**, mientras que en este trabajo se aplica a una tarea de **clasificación multiclase**. En segundo lugar, el artículo opera directamente sobre vectores de entrada clásicos, mientras que aquí cada patch se representa mediante un *embedding* extraído por un modelo fundacional. Finalmente, el objetivo no es construir desde cero un nuevo subconjunto de entrenamiento para un modelo de regresión, sino **refinar el conjunto de patches con el que se reentrena la cabeza de clasificación** de cada backbone fundacional.

Por tanto, la propuesta adoptada en este trabajo es una **adaptación del criterio de selección por información mutua al espacio de embeddings**. En lugar de medir la relevancia de cada muestra sobre variables de entrada originales, se evalúa su aportación informativa dentro del espacio de representación extraído por cada modelo fundacional. El objetivo del experimento es determinar si parte del ruido, redundancia o

ambigüedad presente en el conjunto de patches puede estar perjudicando el rendimiento final de los modelos.

Dado que H-Optimus-1 y Virchow2 producen embeddings de distinta dimensionalidad y geometría, la limpieza se realiza **por separado para cada modelo**. Así, para cada backbone se construye primero una matriz de embeddings independiente, se aplica sobre ella el criterio de selección y se genera un conjunto de entrenamiento limpio específico.

5.2.2. Decisión metodológica: limpieza exclusiva del conjunto de entrenamiento

En esta primera fase, la depuración se aplica exclusivamente sobre el conjunto de **train**. El conjunto de **validación** se mantiene intacto con el fin de conservar una referencia comparable respecto al *baseline* original. De este modo, cualquier mejora o empeoramiento observados posteriormente en validación podrán atribuirse con mayor claridad al efecto de refinar los datos con los que aprende el modelo, sin introducir por el momento nuevas fuentes de sesgo en la evaluación.

No obstante, los embeddings del conjunto de validación sí se extraen desde esta etapa, dejando desacopladas desde el inicio las fases de extracción de representaciones, depuración y evaluación posterior.

Esta decisión no es la única posible. De hecho, durante la discusión metodológica surgieron otras alternativas, como depurar también validación o introducir mecanismos de rechazo sobre muestras dudosas en test. Sin embargo, en esta primera aproximación se optó por la opción más conservadora: actuar únicamente sobre el conjunto de entrenamiento y mantener inalterado el protocolo de validación.

5.2.3. Pipeline

Una vez fijado el marco metodológico, el procedimiento puede describirse como una secuencia de operaciones sobre el espacio de embeddings generado por cada modelo fundacional. En esencia, se parte de la matriz completa de representaciones, se estima la contribución informativa de cada patch mediante un esquema *leave-one-out* sobre la información mutua global y, finalmente, se decide su conservación o eliminación comparando dicho comportamiento con el de sus vecinos más próximos en el espacio reducido.

1. Extraer los embeddings de todos los patches del split **train** utilizando el backbone fundacional M congelado.
2. Guardar, para cada patch:
 - su embedding correspondiente;
 - su etiqueta de clase y ;
 - un identificador interno `row_id`;

- y sus metadatos externos asociados.
- 3. Reducir la dimensionalidad de X mediante PCA a d' componentes, obteniendo una nueva representación reducida Z .
- 4. Calcular la información mutua global MI_{full} entre Z y y .
- 5. Calcular, para cada patch i , sus K vecinos más cercanos en el espacio reducido Z .
- 6. Para cada patch $i = 1, \dots, N$:
 - a) Construir Z^{-i} e y^{-i} , eliminando temporalmente la muestra i .
 - b) Calcular MI^{-i} , es decir, la información mutua del conjunto tras retirar dicho patch.
 - c) Definir la pérdida de información asociada al patch i como:

$$\Delta_i = MI_{\text{full}} - MI^{-i}.$$
- 7. Para cada patch $i = 1, \dots, N$:
 - a) Inicializar un contador de discrepancias $C_i \leftarrow 0$.
 - b) Para cada vecino $j \in \mathcal{N}_K(i)$:
 - 1) Si $|\Delta_i - \Delta_j| > \alpha$, incrementar el contador: $C_i \leftarrow C_i + 1$.
 - c) Si $C_i \geq T$, marcar el patch i como **eliminado**; en caso contrario, marcarlo como **conservado**.
- 8. Construir el conjunto limpio X_{clean} a partir de los patches conservados.
- 9. Guardar los artefactos resultantes de la depuración:
 - embeddings limpios;
 - metadatos limpios;
 - fichero completo de depuración con los valores Δ_i , el número de discrepancias locales y la decisión final **keep/drop**.
- 10. Reentrenar la cabeza de clasificación del modelo fundacional sobre el conjunto limpio.
- 11. Evaluar el modelo en validación, manteniendo **val** intacto.

5.2.4. Pruebas y ajuste de parámetros para la limpieza con H-Optimus1

Una vez definido el procedimiento, el primer modelo sobre el que se aplicó fue **H-Optimus1**. El objetivo de esta fase fue por un lado, ajustar empíricamente los parámetros más sensibles del método antes de lanzar la limpieza completa sobre el conjunto de entrenamiento. Por otro lado, comprobar si la depuración obtenida resultaba útil en el rendimiento en validación una vez reentrenada la cabeza de clasificación.

Dado el elevado coste computacional del esquema secuencial basado en información mutua *leave-one-out*, no resultaba razonable iniciar directamente una limpieza completa sobre todo el conjunto de entrenamiento sin estudiar antes el comportamiento del criterio sobre subconjuntos representativos. Por este motivo, se planteó una fase previa de calibración sobre subconjuntos aleatorios de distinto tamaño.

Calibración preliminar sobre subconjuntos aleatorios

La calibración inicial del método se realizó sobre subconjuntos **aleatorios** del conjunto de entrenamiento. Este detalle fue importante, ya que tomar las primeras muestras del dataset podría introducir sesgos derivados del orden de almacenamiento de los patches, concentrando el análisis en solo una o dos clases.

En todas las pruebas de esta fase se mantuvieron fijos los siguientes elementos:

- reducción de dimensionalidad mediante PCA a 32 componentes;
- número de vecinos locales $K = 6$;
- umbral de conteo fijado en $T = 3$.

Es por ello que el parámetro que se estudió empíricamente fue el umbral de discrepancia α , entendido como el valor mínimo a partir del cual la diferencia entre la pérdida de información de un patch y la de sus vecinos pasa a considerarse significativa.

Prueba 1: subconjunto aleatorio de 500 patches Los resultados obtenidos se resumen en la Tabla 5.9.

Tabla 5.9: Estudio empírico del parámetro α sobre un subconjunto aleatorio de 500 patches

α	Patches eliminados	Porcentaje eliminado
0.05	1	0.2 %
0.02	123	24.6 %
0.01	365	73.0 %
0.005	489	97.8 %

Como puede observarse, valores altos de α hacen que el método no haga nada, mientras que valores demasiado bajos provocan una eliminación masiva de patches.

A la vista de estos resultados, se descartó $\alpha = 0,05$ por resultar excesivamente conservador, ya que apenas eliminaba ejemplos. En el extremo opuesto, los valores $\alpha = 0,01$ y $\alpha = 0,005$ se consideraron demasiado agresivos, ya que eliminaban un porcentaje muy alto del subconjunto.

En esta primera escala, el valor $\alpha = 0,02$ ofreció el compromiso más razonable entre ambos extremos, eliminando un 24,6 % de los patches sin llegar a vaciar el conjunto. Además, al revisar la distribución por clases antes y después de la depuración con este valor, no se observó un vaciado desproporcionado hacia ninguna clase en concreto, sino una reducción relativamente homogénea del subconjunto.

Prueba 2: subconjunto aleatorio de 5000 patches Una vez realizado el estudio sobre 500 patches, se repitió el mismo análisis sobre un subconjunto aleatorio de mayor tamaño, concretamente de **5000 patches**, con el fin de comprobar si el comportamiento observado para α se mantenía al cambiar el número de muestras.

En una primera comprobación se mantuvieron los valores obtenidos en el estudio de los 500 patches, es decir:

$$K = 6, \quad \alpha = 0,02, \quad \text{PCA} = 32.$$

Sin embargo, al aplicar esta configuración sobre 5000 patches, el algoritmo **no eliminó ninguna muestra**. Este resultado demostró un importante descubrimiento: al aumentar el tamaño del conjunto a limpiar, la variación provocada por la eliminación de un único patch sobre la información mutua global pasa a ser mucho menor. Es por ello que el parámetro α , entendido como umbral absoluto de discrepancia, deja de operar en la misma escala que en el experimento con 500 muestras.

Este fenómeno resulta coherente con la propia naturaleza del criterio:

- cuando el número de muestras es pequeño, la eliminación de un patch puede provocar una caída notoria en la información mutua global, lo que facilita detectar discrepancias significativas entre un patch y sus vecinos;
- cuando el número de muestras es mayor, la eliminación de un único patch tiene un impacto menor sobre la información mutua global, de modo que las diferencias entre patches y vecinos se vuelven más difíciles de detectar con el mismo valor de α .

Tras esta observación, se procedió a recalibrar el parámetro α , manteniendo fijos el resto de parámetros. Los resultados obtenidos se resumen en la Tabla 5.10. En una primera fase de exploración se probaron valores claramente más pequeños que $\alpha = 0,02$, con el objetivo de localizar el rango operativo del parámetro. Posteriormente, una vez acotada la zona útil, se evaluaron valores intermedios para identificar el mejor compromiso entre una limpieza demasiado débil y una excesivamente agresiva.

Tabla 5.10: Estudio empírico del parámetro α sobre un subconjunto aleatorio de 5000 patches

α	Patches eliminados	Porcentaje eliminado
0.0200	0	0.00 %
0.0030	283	5.66 %
0.0020	1173	23.46 %
0.0015	2136	42.72 %
0.0010	3659	73.18 %
0.0005	4805	96.10 %
0.0002	4989	99.78 %

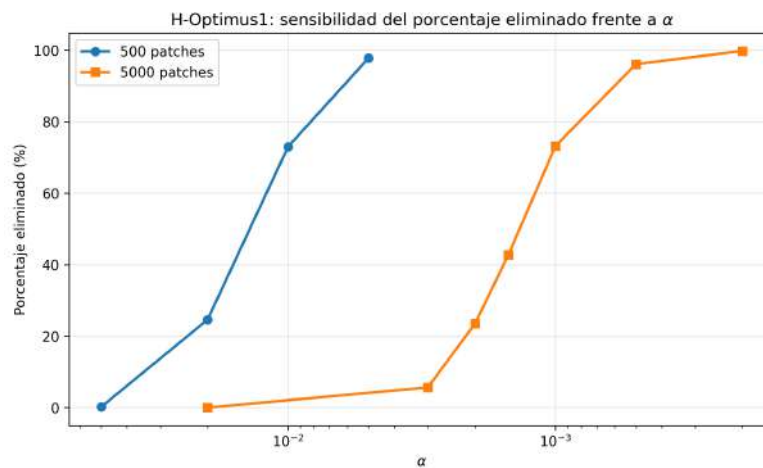


Figura 5.8: Sensibilidad del porcentaje de patches eliminados frente al parámetro α en H-Optimus1

Podemos observar tanto en la tabla como en la figura 5.8 que el valor $\alpha = 0,02$, que en el análisis con 500 patches había eliminado un 24.6 %, pasa aquí a no eliminar ninguna muestra. En el extremo opuesto, los valores $\alpha = 0,001$, $\alpha = 0,0005$ y $\alpha = 0,0002$ provocan una limpieza casi absoluta del subconjunto.

Entre ambos extremos, los valores $\alpha = 0,003$, $\alpha = 0,002$ y $\alpha = 0,0015$ permitieron identificar una zona intermedia útil. De ellos, $\alpha = 0,003$ se consideró todavía demasiado conservador, ya que únicamente eliminaba un 5,66 % del subconjunto, mientras que $\alpha = 0,0015$ se juzgó ya excesivamente agresivo al eliminar un 42,72 %. En consecuencia, el valor $\alpha = 0,002$ se seleccionó inicialmente como el mejor compromiso observado en esta escala, al eliminar un 23,46 % de los patches.

En definitiva, estas pruebas dejaron una conclusión clara: el umbral α no puede considerarse un parámetro absoluto e independiente de la escala del problema, sino que

debe calibrarse teniendo en cuenta el tamaño del subconjunto sobre el que se estima la información mutua.

Limpieza completa del conjunto de entrenamiento

Tras la calibración previa, se lanzó una primera limpieza completa sobre todo el conjunto de entrenamiento de H-Optimus1 empleando la configuración seleccionada en la escala de 5000 patches, es decir:

$$K = 6, \quad \alpha = 0,002, \quad \text{PCA} = 32.$$

Sin embargo, al ejecutar el procedimiento sobre el conjunto completo, formado por 46624 patches, el algoritmo volvió a **no eliminar ninguna muestra**. El análisis posterior mostró que la magnitud de la variable Δ_i , definida como la pérdida de información asociada a la eliminación de un patch, se había reducido todavía más, pasando a situarse en el orden de 10^{-4} . En consecuencia, el valor $\alpha = 0,002$, que había resultado adecuado sobre 5000 patches, volvía a ser demasiado grande para la escala real del problema en el conjunto completo.

Por ello, fue necesario recalibrar nuevamente el umbral, esta vez tomando como referencia la distribución real de Δ_i en la ejecución completa.

A partir de esta nueva observación se probaron dos configuraciones finales sobre todo el conjunto de entrenamiento:

- una primera configuración con $\alpha = 0,0002$, más agresiva;
- y una segunda con $\alpha = 0,00025$, más conservadora.

Los resultados globales de ambas ejecuciones se resumen en la Tabla 5.11.

Tabla 5.11: Resultados de la limpieza completa del conjunto de entrenamiento de H-Optimus1

α	Patches conservados	Patches eliminados	Porcentaje eliminado
0.0002	33058	13566	29.10 %
0.00025	39254	7370	15.81 %

En ambos casos, la distribución por clases tras la depuración se mantuvo razonablemente equilibrada, sin que ninguna clase quedase vaciada de forma extrema. No obstante, la limpieza con $\alpha = 0,0002$ resultó claramente más agresiva que la obtenida con $\alpha = 0,00025$, lo que hacía pensar que ambas configuraciones pudieran comportarse de forma distinta en la fase de reentrenamiento.

Resultados del reentrenamiento con embeddings limpios y comparación con el baseline

Una vez generados los conjuntos limpios, se procedió a reentrenar la cabeza de clasificación de H-Optimus1 sobre embeddings ya extraídos, manteniendo inalterado el conjunto de validación.

En una primera comprobación, la configuración más agresiva ($\alpha = 0,0002$) mostró una degradación clara del rendimiento respecto al *baseline* original, bajando la media de F1 macro hasta 0.5782 puntos. En consecuencia, se consideró más prometedora la variante más conservadora ($\alpha = 0,00025$).

Los resultados obtenidos para H-Optimus1 reentrenado sobre el conjunto limpio con $\alpha = 0,00025$ se recogen en la Tabla 5.12.

Tabla 5.12: Resultados de H-Optimus1 reentrenado sobre el conjunto limpio con $\alpha = 0,00025$

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.6019	0.7947	5
29	0.6021	0.7958	7
47	0.6057	0.7978	5
71	0.5998	0.7938	7
101	0.5964	0.7943	10
Media	0.6012	0.7953	—
Desv. típica	0.0031	0.0014	—

Para interpretar adecuadamente estos resultados, conviene compararlos con el *baseline* original del mismo modelo.

Tabla 5.13: Comparación entre el baseline original de H-Optimus1 y el reentrenamiento sobre el conjunto limpio

Configuración	Val F1 macro medio	Val Accuracy media
Baseline original	0.6118	0.7998
Limpieza ($\alpha = 0,00025$)	0.6012	0.7953
Limpieza ($\alpha = 0,0002$)	0.5782	0.7692

Los resultados muestran que, aunque la limpieza más conservadora ($\alpha = 0,00025$) mejora claramente frente a la más agresiva ($\alpha = 0,0002$), sigue sin superar el rendimiento del baseline original. En promedio, el modelo reentrenado sobre el conjunto limpio pierde aproximadamente 0.0106 puntos de F1 macro y 0.0045 puntos de *accuracy* respecto al entrenamiento sobre el conjunto completo.

Por tanto, en el caso de **H-Optimus1**, la depuración basada en información mutua no produjo una mejora en validación, aunque sí permitió comprobar que el comportamiento

final depende fuertemente del nivel de agresividad impuesto por el parámetro α . Una limpieza más grande degrada claramente el rendimiento, mientras que una limpieza más moderada reduce dicha degradación, aunque sin llegar a superar al modelo sin depuración.

Coste computacional y consideraciones prácticas

Aunque la depuración no produjo una mejora en validación para H-Optimus1, el proceso sí puso de manifiesto una ventaja importante desde el punto de vista computacional. La construcción del conjunto limpio requiere un tiempo inicial elevado, ya que la limpieza completa del conjunto de entrenamiento se ejecuta de forma secuencial y puede prolongarse durante aproximadamente **36 horas**. Sin embargo, esta operación se realiza una única vez por modelo y conjunto de datos.

Una vez generado el conjunto limpio y fijados los embeddings, el reentrenamiento posterior de la cabeza de clasificación resulta muy rápido. En el caso de H-Optimus1, las 10 épocas completas de reentrenamiento sobre embeddings limpios se ejecutaron en aproximadamente **9.6 segundos** por semilla, frente a los 4000 segundos aproximadamente requeridos por el baseline original entrenado directamente sobre imágenes.

Este comportamiento nos deja una conclusión a nuestro favor: aunque no se provoque una mejora en el rendimiento, el desacoplamiento entre extracción de representaciones, depuración y reentrenamiento convierte este procedimiento en una estrategia mucho más **reproducible**, **iterable** y **económica** desde el punto de vista experimental. En otras palabras, una vez pagado el coste inicial de la limpieza, la comparación entre semillas, configuraciones y criterios de selección se vuelve extremadamente rápida.

A la vista de estos resultados, el caso de H-Optimus1 permite extraer una conclusión metodológica provisional: la depuración basada en información mutua, tal y como ha sido formulada en esta primera aproximación, no mejora el rendimiento del modelo en validación, pero sí constituye una herramienta útil para estudiar de forma controlada la relación entre selección de patches, coste computacional y estabilidad experimental.

5.2.5. Pruebas y ajuste de parámetros para la limpieza con Virchow2

Una vez analizado el comportamiento sobre H-Optimus1, se repitió el mismo protocolo sobre **Virchow2**, manteniendo exactamente los mismos experimentos.

Calibración preliminar sobre subconjuntos aleatorios

Al igual que en H-Optimus1, la calibración inicial se realizó sobre subconjuntos aleatorios del conjunto de entrenamiento, manteniendo fijos los mismos parámetros estructurales del procedimiento:

- reducción de dimensionalidad mediante PCA a 32 componentes;

- número de vecinos locales $K = 6$;
- umbral de conteo fijado en $T = 3$.

Por tanto, el único parámetro sometido a ajuste empírico volvió a ser el umbral de discrepancia α .

Prueba 1: subconjunto aleatorio de 500 patches La primera exploración se realizó sobre un subconjunto aleatorio de 500 patches. Los resultados se resumen en la Tabla 5.14.

Tabla 5.14: Estudio empírico del parámetro α sobre un subconjunto aleatorio de 500 patches para Virchow2

α	Patches eliminados	Porcentaje eliminado
0.05	1	0.20 %
0.02	106	21.20 %
0.01	354	70.80 %
0.005	477	95.40 %

El comportamiento observado fue muy similar al de H-Optimus1: valores altos de α vuelven a hacer el criterio prácticamente inactivo, mientras que valores bajos provocan una eliminación demasiado agresiva del subconjunto. En esta primera prueba, $\alpha = 0,02$ volvió a situarse como el más razonable entre ambos extremos.

Prueba 2: subconjunto aleatorio de 5000 patches A continuación, se repitió el análisis sobre un subconjunto aleatorio de 5000 patches, manteniendo inicialmente la configuración de la escala anterior, es decir:

$$K = 6, \quad \alpha = 0,02, \quad \text{PCA} = 32.$$

Sin embargo, al aplicar esta configuración sobre 5000 patches, el algoritmo volvió a no eliminar ninguna muestra, ya que como vimos con H-Optimus1, al crecer el número de muestras, la magnitud de la pérdida de información asociada a la eliminación de un único patch disminuye, de modo que el valor absoluto de α deja de operar en la misma escala.

Por este motivo, se procedió a recalibrar nuevamente el parámetro α . Los resultados obtenidos se muestran en la Tabla 5.15.

Tabla 5.15: Estudio empírico del parámetro α sobre un subconjunto aleatorio de 5000 patches para Virchow2

α	Patches eliminados	Porcentaje eliminado
0.0200	0	0.00 %
0.0030	356	7.12 %
0.0020	1260	25.20 %
0.0015	2251	45.02 %

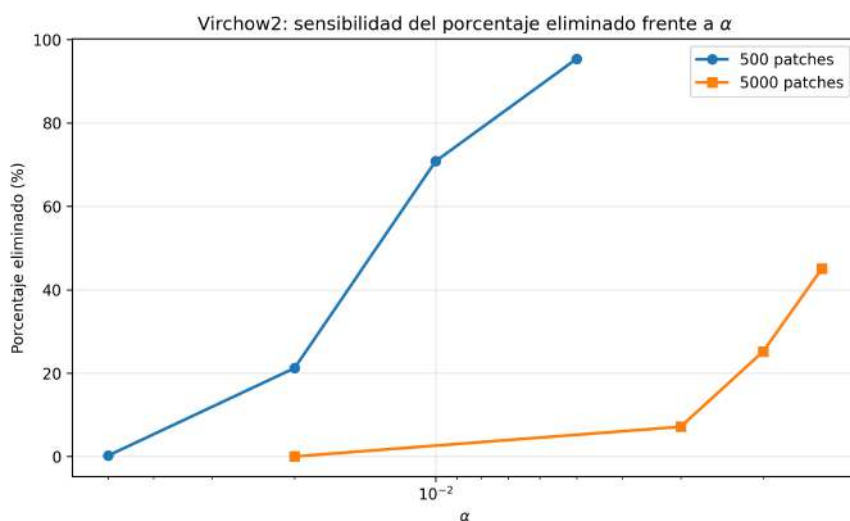


Figura 5.9: Sensibilidad del porcentaje de patches eliminados frente al parámetro α en Virchow2

De nuevo, el comportamiento fue muy similar al observado en H-Optimus1. El valor $\alpha = 0,003$ resultó todavía demasiado conservador, mientras que $\alpha = 0,0015$ ya producía una limpieza demasiado agresiva. En consecuencia, $\alpha = 0,002$ se seleccionó como la configuración inicial más prometedora para la limpieza completa del conjunto de entrenamiento.

Limpieza completa del conjunto de entrenamiento

Una vez fijada la configuración inicial, se lanzó la limpieza completa del conjunto de entrenamiento de Virchow2 con:

$$K = 6, \quad \alpha = 0,002, \quad \text{PCA} = 32.$$

Al igual que había ocurrido en H-Optimus1, esta primera ejecución completa no eliminó ningún patch. El análisis posterior mostró de nuevo que la magnitud de Δ_i en el

conjunto completo se situaba en el orden de 10^{-4} , por lo que el valor $\alpha = 0,002$ volvía a ser demasiado grande para la escala real del problema.

Por ello, se repitió la misma estrategia seguida anteriormente y se ensayaron dos configuraciones finales más adecuadas para el conjunto completo:

- una limpieza más agresiva con $\alpha = 0,0002$;
- y una limpieza más conservadora con $\alpha = 0,00025$.

Los resultados globales obtenidos se resumen en la Tabla 5.16.

Tabla 5.16: Resultados de la limpieza completa del conjunto de entrenamiento de Virchow2

α	Patches conservados	Patches eliminados	Porcentaje eliminado
0.0002	32705	13919	29.85 %
0.00025	39015	7609	16.32 %

En ambos casos, la distribución por clases tras la depuración se mantuvo razonablemente equilibrada y sin vaciar ninguna clase de forma extrema. De nuevo, la configuración con $\alpha = 0,0002$ generó una limpieza claramente más intensa, mientras que $\alpha = 0,00025$ produjo una reducción más moderada del conjunto de entrenamiento.

Resultados del reentrenamiento con embeddings limpios y comparación con el baseline

A diferencia de lo observado en H-Optimus1, en Virchow2 ambas configuraciones finales produjeron resultados relativamente próximos al *baseline* original, especialmente en términos de *accuracy*. Por este motivo, se completaron las cinco semillas para las dos configuraciones finales, $\alpha = 0,0002$ y $\alpha = 0,00025$.

Los resultados correspondientes a la limpieza con $\alpha = 0,0002$ se muestran en la Tabla 5.17.

Tabla 5.17: Resultados de Virchow2 reentrenado sobre el conjunto limpio con $\alpha = 0,0002$

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.5948	0.7954	7
29	0.5872	0.7903	10
47	0.5897	0.7886	7
71	0.5927	0.7923	6
101	0.5894	0.7919	8
Media	0.5908	0.7917	
Desv. típica	0.0027	0.0023	

Por su parte, los resultados de la limpieza más conservadora con $\alpha = 0,00025$ se recogen en la Tabla 5.18.

Tabla 5.18: Resultados de Virchow2 reentrenado sobre el conjunto limpio con $\alpha = 0,00025$

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.5907	0.7914	5
29	0.5874	0.7861	6
47	0.5902	0.7921	4
71	0.5888	0.7870	5
101	0.5902	0.7905	4
Media	0.5895	0.7894	
Desv. típica	0.0012	0.0024	

Para facilitar la comparación, la Tabla 5.19 resume conjuntamente el baseline y las dos configuraciones limpias evaluadas.

Tabla 5.19: Comparación entre el baseline original de Virchow2 y el reentrenamiento sobre conjuntos limpios

Configuración	Val F1 macro medio	Val Accuracy media
Baseline original	0.6011	0.7892
Limpieza ($\alpha = 0,0002$)	0.5908	0.7917
Limpieza ($\alpha = 0,00025$)	0.5895	0.7894

Los resultados muestran que ninguna de las dos configuraciones limpias logra superar al baseline original en términos de F1 macro. No obstante, el comportamiento de Virchow2 difiere del observado en H-Optimus1 en un aspecto relevante: aunque el F1 macro disminuye, la *accuracy* se mantiene muy próxima al baseline e incluso mejora ligeramente en la configuración con $\alpha = 0,0002$.

Entre las dos alternativas evaluadas, $\alpha = 0,0002$ ofrece el mejor compromiso aunque por poco, ya que presenta mejor F1 macro medio y mejor *accuracy* media que $\alpha = 0,00025$. Aun así, la pérdida respecto al baseline original en F1 macro sigue siendo apreciable, de aproximadamente 0.0103 puntos.

Por tanto, en el caso de **Virchow2**, la depuración basada en información mutua **tampoco consiguió mejorar el rendimiento respecto al baseline inicial**.

Coste computacional y consideraciones prácticas

Desde el punto de vista computacional, el comportamiento observado en Virchow2 fue también muy similar al de H-Optimus1. La fase de limpieza completa del conjunto

de entrenamiento vuelve a requerir una inversión inicial muy elevada, ya que el procedimiento secuencial basado en información mutua *leave-one-out* necesita recorrer todas las muestras del conjunto y recalcular sucesivamente el criterio de depuración. En este caso, la limpieza completa de Virchow2 requirió aproximadamente **36.5 horas**.

Sin embargo, una vez generado el conjunto limpio, el reentrenamiento posterior de la cabeza de clasificación vuelve a resultar extremadamente barato. En el caso de Virchow2, las 10 épocas completas de reentrenamiento sobre embeddings limpios se ejecutaron en aproximadamente **10.6 segundos** por semilla con $\alpha = 0,0002$ y en aproximadamente **9.2 segundos** por semilla con $\alpha = 0,00025$, frente a los tiempos del orden de **9 horas** requeridos por el baseline original entrenado directamente sobre imágenes.

Al igual que en H-Optimus1, esto refuerza una idea metodológica importante: aunque la limpieza no haya producido una mejora en F1 macro, el desacoplamiento entre extracción de embeddings, depuración y reentrenamiento sigue siendo una ventaja experimental relevante, ya que facilita la repetibilidad de las pruebas, abarata la exploración de configuraciones y reduce de forma drástica el coste de las iteraciones posteriores.

5.2.6. Discusión conjunta y siguientes alternativas metodológicas

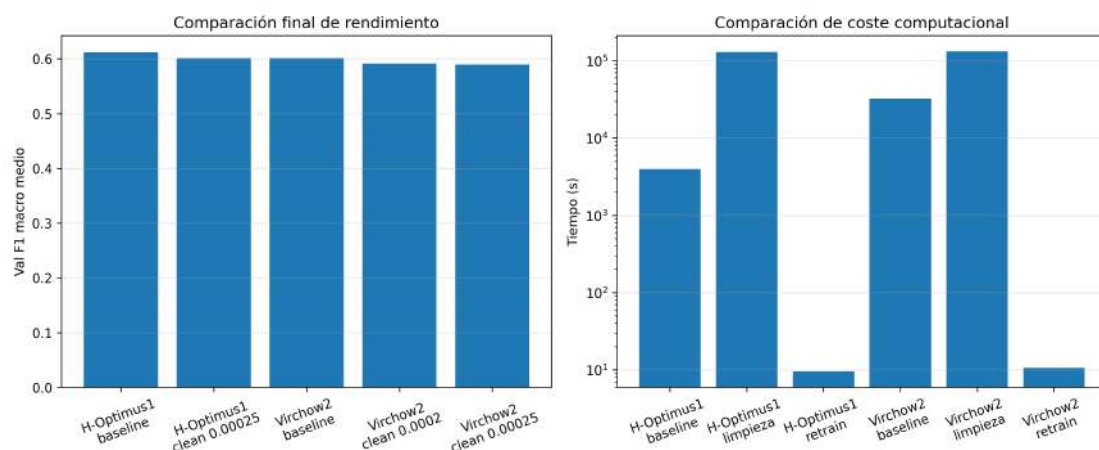


Figura 5.10: Comparación del F1 macro y del coste computacional entre el baseline original y los conjuntos limpios

Los resultados obtenidos en esta fase permiten extraer varias conclusiones relevantes. En primer lugar, aunque la depuración basada en información mutua no ha conseguido mejorar el rendimiento respecto al *baseline* original en términos de F1 macro, sí ha demostrado que una fracción importante del conjunto de entrenamiento podía eliminarse sin provocar una degradación severa del comportamiento de los modelos. En ambos casos, tanto para H-Optimus1 como para Virchow2, fue posible suprimir aproximadamente un

30 % de los patches manteniendo unas métricas de validación relativamente próximas a las del entrenamiento sobre el conjunto completo.

Desde este punto de vista, la limpieza realizada no debe interpretarse como un resultado negativo, sino que pone de manifiesto que gran parte de los patches originales aportaba información poco útil para el modelo.

En segundo lugar, esta fase ha confirmado la utilidad de desacoplar la extracción de embeddings del entrenamiento posterior. Aunque la construcción inicial del conjunto limpio exige un coste elevado, una vez calculadas las representaciones y fijado el subconjunto seleccionado, el reentrenamiento de la cabeza de clasificación pasa a ser extremadamente barato desde el punto de vista computacional. En la práctica, esto transforma el proceso experimental en un flujo mucho más reproducible, iterativo y manejable que el entrenamiento repetido del *baseline* original sobre imágenes.

En definitiva, se da por cerrada esta primera forma de depuración basada en información mutua aplicada exclusivamente sobre el conjunto de entrenamiento. La siguiente fase del trabajo se centrará en comparar este enfoque con métodos de *undersampling* más sencillos, con el objetivo de determinar si existen alternativas más simples y competitivas para refinar el conjunto de patches antes de avanzar hacia experimentos a nivel de ROI.

5.3. Limpieza de patches con métodos de undersampling estándar

Una vez finalizada la etapa de depuración basada en información mutua, se consideró pertinente contrastar dicho enfoque con métodos de *undersampling* más simples y estandarizados.

Esta comparación resulta pertinente por dos motivos:

- En primer lugar, la limpieza mediante IM había mostrado un comportamiento interesante, permitiendo eliminar una fracción importante del conjunto de entrenamiento sin producir una degradación notoria del rendimiento.
- En segundo lugar, su coste de construcción era elevado, por lo que resultaba razonable preguntarse si métodos más sencillos podían ofrecer un rendimiento igual o incluso mejor del modelo.

Es por ello que se planteó una nueva batería de experimentos sobre el mismo escenario de patches, manteniendo fijo todo el protocolo de evaluación ya utilizado en las fases anteriores. De este modo, la comparación entre métodos se realizó en las mismas condiciones: mismos embeddings, mismo conjunto de validación intacto, misma cabeza de clasificación y mismas semillas de entrenamiento.

5.3.1. Métodos evaluados: RandomUnderSampler y NeighbourhoodCleaningRule

Para esta nueva fase de comparación se seleccionaron dos métodos de *undersampling* de la biblioteca imbalanced-learn, con los objetivos de tener:

- un escenario de **reducción aleatoria controlada** del conjunto de entrenamiento, representado por RandomUnderSampler;
- y un escenario de **limpieza estructurada basada en vecindario**, representado por NeighbourhoodCleaningRule.

RandomUnderSampler

Este método se escogió como referencia de submuestreo simple, para analizar que ocurría si se eliminaba aproximadamente la misma fracción de patches de manera aleatoria pero manteniendo la estructura global del conjunto. Esta comparación resulta importante porque permite distinguir si los resultados observados con la información mutua proceden realmente del criterio de selección o, por el contrario, de una simple reducción del tamaño del dataset.

RandomUnderSampler es un método de submuestreo aleatorio que reduce el tamaño del conjunto de entrenamiento seleccionando una fracción de las muestras disponibles. Su versión más habitual se utiliza para rebalancear clases mayoritarias, eliminando ejemplos de dichas clases hasta aproximarlas a las minoritarias. Sin embargo, ese no era exactamente el objetivo perseguido en este trabajo.

En nuestro caso, RandomUnderSampler no se empleó como técnica de rebalanceo estricto, sino como un mecanismo de **reducción aleatoria estratificada del conjunto de entrenamiento**. La idea consistió en conservar aproximadamente el 70 % de las muestras de cada clase, de manera que todas ellas se redujeran de forma proporcional y el tamaño final del dataset resultase comparable al obtenido previamente con la limpieza basada en información mutua. Con ello se evitaba que el experimento se limitase a recortar únicamente las clases mayoritarias, lo que habría introducido un sesgo distinto del que se quería estudiar.

NeighbourhoodCleaningRule

NeighbourhoodCleaningRule (NCR) es un método de limpieza basado en vecindario. Su funcionamiento se apoya en la relación local entre muestras dentro del espacio de representación, eliminándose una muestra si resulta inconsistente con las de su entorno más próximo, provocando así una limpieza de ejemplos conflictivos o ruidosos.

Mientras que la limpieza por información mutua comparaba la contribución informativa de cada patch con la de sus vecinos, NCR actúa también a nivel local, aunque mediante una regla estándar mucho más sencilla y con un coste computacional mucho menor, comparando así una estrategia clásica de limpieza geométrica en el espacio de *embeddings* con un criterio mucho más costoso como la información mutua *leave-one-out*.

5.3.2. Protocolo experimental

Para dar continuidad al objetivo **OBJ1.4** y compararlo de forma rigurosa conforme a **OBJ1.8**, una vez fijados los métodos a evaluar y su adaptación al espacio de embeddings, se definió un protocolo común de evaluación para los métodos de undersampling, de manera que las diferencias observadas pudieran atribuirse al criterio de reducción aplicado y no a cambios en el resto del pipeline.

Para cada modelo fundacional, **H-Optimus1** y **Virchow2**, se partió de la matriz de embeddings del conjunto de entrenamiento y se generaron nuevos subconjuntos mediante las dos estrategias de *undersampling*. En ambos casos, el conjunto de validación se mantuvo intacto, de forma análoga a lo realizado previamente con la limpieza basada en información mutua.

En el caso de **RandomUnderSampler**, el protocolo quedó fijado como una reducción aleatoria del conjunto de entrenamiento, conservando aproximadamente el 70 % de las muestras de cada clase. De este modo, el subconjunto resultante quedaba en una escala de reducción próxima a la utilizada en la limpieza basada en información mutua, lo que permitía una comparación razonablemente homogénea entre métodos.

Por su parte, con **NeighbourhoodCleaningRule** se siguió un procedimiento en dos etapas. En una primera evaluación, el método se aplicó con una configuración inicial estándar, concretamente con $k = 3$, lo que permitió obtener una primera referencia de su comportamiento natural como técnica de limpieza local sobre embeddings. Sin embargo, el análisis posterior mostró que esta primera configuración resultaba claramente más conservadora que el resto de métodos, ya que eliminaba una proporción sensiblemente menor de patches que la observada en la limpieza basada en información mutua y en **RandomUnderSampler**.

No obstante, se decidió incorporar una segunda etapa de **recalibración de NCR**, aumentando progresivamente el número de vecinos k con el objetivo de aproximar su agresividad a la escala de reducción de los demás métodos. Este barrido de valores permitió estudiar cómo variaba el porcentaje de patches eliminados al ampliar el vecindario local. Los resultados de esta recalibración se recogen en la Tabla 5.20.

Tabla 5.20: Recalibración de NCR sobre el conjunto completo de entrenamiento

Modelo	k	Patches conservados	Patches eliminados	% eliminado
H-Optimus1	3	44131	2493	5.35 %
H-Optimus1	15	41482	5142	11.03 %
H-Optimus1	25	39544	7080	15.19 %
H-Optimus1	35	38047	8577	18.40 %
H-Optimus1	45	36838	9786	20.99 %
Virchow2	3	42784	3840	8.24 %
Virchow2	15	39551	7073	15.17 %
Virchow2	25	37607	9017	19.34 %
Virchow2	35	36003	10621	22.78 %
Virchow2	45	34724	11900	25.52 %

Como puede observarse, el incremento de k aumenta progresivamente la agresividad de la limpieza en ambos modelos, aunque Virchow2 mostró una mayor sensibilidad al aumento del vecindario, alcanzando porcentajes de eliminación más elevados para los mismos valores de k .

A la vista de estos resultados, se decidió fijar $k = 45$ como configuración final de NCR para la comparación global. Aunque este valor no iguala exactamente el porcentaje de eliminación de la información mutua o de RandomUnderSampler, sí sitúa a NCR en una escala mucho más comparable y permite realizar una comparación más equilibrada entre estrategias de selección de patches.

Se obtuvieron por tanto dos tipos de resultados con NCR:

- unos primeros resultados con la configuración inicial, que reflejan el comportamiento original del método;
- y unos segundos resultados tras recalibrar el parámetro k , que son los que finalmente se emplean en la comparación global al resultar metodológicamente más justos.

A continuación, la cabeza de clasificación lineal se reentrenó sobre cada subconjunto resultante utilizando exactamente la misma configuración ya empleada en el resto de experimentos: mismas semillas, mismo número de épocas, mismo optimizador, mismo *scheduler* y mismo esquema de evaluación.

5.3.3. Resultados con H-Optimus1

Como se mencionó anteriormente, **RandomUnderSampler** se aplicó como una reducción aleatoria estratificada del conjunto de entrenamiento, conservando aproximadamente el 70 % de las muestras de cada clase. El resultado fue un nuevo conjunto con 32637 patches, lo que equivale a una reducción global del 30,00 %, en una escala prácticamente idéntica a la empleada en la limpieza por información mutua.

Los resultados obtenidos tras el reentrenamiento sobre este conjunto reducido se muestran en la Tabla 5.21.

Tabla 5.21: Resultados de H-Optimus1 reentrenado sobre el conjunto reducido mediante RandomUnderSampler

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.5979	0.7938	10
29	0.6016	0.7947	10
47	0.6017	0.7936	7
71	0.6023	0.7952	7
101	0.5972	0.7938	7
Media	0.6001	0.7942	—
Desv. típica	0.0024	0.0007	—

Estos resultados muestran que la reducción aleatoria del conjunto de entrenamiento permite mantener un rendimiento relativamente competitivo, aunque sin alcanzar al *baseline* original. Además, el comportamiento medio obtenido resulta muy próximo al de la limpieza por información mutua, lo que sugiere que una parte del efecto observado en la fase anterior puede explicarse por la propia reducción del tamaño del conjunto, y no necesariamente por un criterio de selección especialmente informativo.

En segundo lugar, se evaluó **NeighbourhoodCleaningRule** (NCR) en su configuración inicial, es decir, con $k = 3$. En esta primera versión, el método produjo una depuración mucho más conservadora que la del resto de estrategias, eliminando únicamente 2493 patches, lo que equivale a una reducción del 5,35 % del conjunto de entrenamiento.

Los resultados obtenidos tras el reentrenamiento sobre este primer conjunto filtrado mediante NCR se recogen en la Tabla 5.22.

Tabla 5.22: Resultados de H-Optimus1 reentrenado sobre el conjunto filtrado mediante NCR en su configuración inicial ($k = 3$)

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.6083	0.8013	7
29	0.6029	0.7965	9
47	0.6062	0.7980	8
71	0.6091	0.8007	10
101	0.6038	0.7998	9
Media	0.6061	0.7993	—
Desv. típica	0.0027	0.0020	—

En esta configuración inicial, NCR superó tanto a RandomUnderSampler como a la limpieza basada en información mutua y se acercó notablemente al *baseline* original. Sin

embargo, como ya se indicó en el protocolo experimental, esta comparación no resultaba completamente justa, ya que la proporción de patches eliminados era muy inferior a la de los otros métodos.

Por este motivo, se procedió a una **recalibración de NCR** aumentando progresivamente el número de vecinos hasta fijar finalmente $k = 45$, lo que elevó la reducción del conjunto de entrenamiento hasta 9786 patches, es decir, un **20,99%** del total. Los resultados correspondientes a esta versión recalibrada se muestran en la Tabla 5.23.

Tabla 5.23: Resultados de H-Optimus1 reentrenado sobre el conjunto filtrado mediante NCR recalibrado ($k = 45$)

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.5977	0.7982	10
29	0.6014	0.7993	9
47	0.5964	0.7965	8
71	0.5992	0.7991	10
101	0.5952	0.7989	9
Media	0.5980	0.7984	—
Desv. típica	0.0024	0.0011	—

Al aumentar la agresividad de la limpieza, el rendimiento de NCR desciende de forma visible respecto a su configuración inicial. En particular, la media de *F1 macro* pasa de 0.6061 a 0.5980, lo que indica que parte de la ventaja observada inicialmente estaba ligada que era más conservador. Aun así, la *accuracy* media se mantiene relativamente alta y próxima al *baseline*, lo que sugiere que NCR sigue siendo una estrategia razonable incluso cuando se aproxima más a la escala de reducción de los otros métodos.

En definitiva, para H-Optimus1 se observa un comportamiento claro: RandomUnderSampler y la información mutua ofrecen resultados similares cuando se fuerza una reducción grande del conjunto, mientras que NCR se muestra muy competitivo cuando actúa de forma suave, pero pierde parte de esa ventaja al recalibrarse para hacer la comparación más justa. Por ello, en la comparación global final de métodos se reportará la versión recalibrada NCR($k = 45$), al ser la más adecuada desde el punto de vista metodológico.

5.3.4. Resultados con Virchow2

A continuación, se repitió el mismo protocolo experimental sobre **Virchow2**.

En el caso de **RandomUnderSampler**, se aplicó la misma estrategia de reducción aleatoria empleada anteriormente, volviendo a quedar el conjunto de entrenamiento reducido a 32637 patches.

Los resultados obtenidos tras el reentrenamiento de la cabeza de clasificación sobre este subconjunto se muestran en la Tabla 5.24.

Tabla 5.24: Resultados de Virchow2 reentrenado sobre el conjunto reducido mediante RandomUnderSampler

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.5920	0.7921	6
29	0.5897	0.7894	7
47	0.5913	0.7903	7
71	0.5909	0.7901	6
101	0.5961	0.7919	6
Media	0.5920	0.7908	—
Desv. típica	0.0024	0.0012	—

Estos resultados muestran un comportamiento similar al observado en H-Optimus1: el submuestreo aleatorio permite conservar una parte importante del rendimiento, pero no logra alcanzar al *baseline* original. No obstante, en este caso concreto, RandomUnderSampler ofrece un comportamiento ligeramente superior al de la limpieza basada en información mutua en términos de *F1 macro*.

Por su parte, **NeighbourhoodCleaningRule** volvió a comportarse inicialmente como una estrategia de limpieza local conservadora. En su configuración original con $k = 3$, el método eliminó 3840 patches, lo que equivale a una reducción global del 8,24%, menor que la obtenida con información mutua o con RandomUnderSampler, aunque superior a la observada en H-Optimus1.

Los resultados del reentrenamiento sobre el conjunto filtrado mediante esta configuración inicial de NCR se presentan en la Tabla 5.25.

Tabla 5.25: Resultados de Virchow2 reentrenado sobre el conjunto filtrado mediante NCR en su configuración inicial ($k = 3$)

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.6023	0.7989	6
29	0.5981	0.7996	4
47	0.5937	0.7919	8
71	0.5963	0.7954	7
101	0.6048	0.7991	7
Media	0.5990	0.7970	—
Desv. típica	0.0045	0.0033	—

En esta primera configuración, NCR volvió a ser el método alternativo más competitivo. Aunque no superaba al *baseline* original en *F1 macro*, sí se aproximaba mucho a él y obtenía una *accuracy* media claramente superior, superando a la limpieza basada en información mutua y al submuestreo aleatorio.

No obstante, esta ventaja inicial volvió a estar condicionada por su carácter más conservador, por lo que se adoptó también en Virchow2 la versión recalibrada **NCR**($k = 45$), que elevó la reducción del conjunto de entrenamiento hasta 11900 patches, es decir, un **25,52** % del total. Los resultados obtenidos con esta configuración final se recogen en la Tabla 5.26.

Tabla 5.26: Resultados de Virchow2 reentrenado sobre el conjunto filtrado mediante NCR recalibrado ($k = 45$)

Semilla	Best val F1 macro	Best val Accuracy	Época
13	0.5772	0.7897	8
29	0.5832	0.7934	6
47	0.5755	0.7872	9
71	0.5795	0.7901	9
101	0.5773	0.7870	8
Media	0.5785	0.7895	—
Desv. típica	0.0030	0.0026	—

En este caso, el efecto de la recalibración es todavía más acusado que en H-Optimus1. Al aumentar la agresividad de la limpieza, NCR pierde una parte importante de su ventaja inicial y pasa a situarse claramente por debajo tanto del *baseline* como de las otras alternativas evaluadas. Este resultado sugiere que, en Virchow2, la fortaleza de NCR estaba estrechamente ligada a una limpieza local suave, y que forzarlo a una escala de reducción más comparable a la de los otros métodos degrada el rendimiento final.

En consecuencia, los resultados de Virchow2 permiten extraer una conclusión metodológica relevante: aunque NCR puede resultar muy competitivo en configuraciones conservadoras, su comportamiento se deteriora al aumentar su agresividad hasta un poco por debajo de la información mutua y del submuestreo aleatorio.

5.3.5. Comparación global

Dado que en la subsección anterior se ha puesto de manifiesto que la configuración inicial de **NCR** ($k = 3$) producía una limpieza excesivamente conservadora, la comparación global final se realiza utilizando la versión **recalibrada** del método, es decir, **NCR**($k = 45$). De este modo, la comparación entre estrategias resulta metodológicamente más equilibrada, ya que todas ellas operan en una escala de reducción del conjunto de entrenamiento mucho más comparable.

La Tabla 5.27 resume la comparación de los resultados obtenidos para **H-Optimus1**, ordenados de mejor a peor según el *F1 macro* medio en validación.

Tabla 5.27: Ranking global de métodos de selección de patches en H-Optimus1, ordenado por *F1 macro* medio en validación

Puesto	Método	F1 medio	Disp.	Acc. media	Disp.	% elim.
1	Baseline	0.6118	0.0080	0.7998	0.0048	0.00 %
2	IM ($\alpha = 0,00025$)	0.6012	0.0031	0.7953	0.0014	15.81 %
3	RandomUnder (70 %)	0.6001	0.0024	0.7942	0.0007	30.00 %
4	NCR ($k = 45$)	0.5980	0.0024	0.7984	0.0011	20.99 %

En H-Optimus1, el *baseline* original conserva claramente la primera posición. Entre los métodos de selección de patches, la limpieza basada en información mutua y RandomUnderSampler muestran un comportamiento muy próximo entre sí, con una ligera ventaja para la primera en términos de *F1 macro*. Por su parte, NCR recalibrado pierde la ventaja que había mostrado en su configuración inicial y pasa a ocupar la última posición en esta comparación por *F1 macro*. No obstante, su *accuracy* media sigue siendo alta y se mantiene cercana al *baseline*, lo que sugiere que su deterioro afecta más a la sensibilidad global entre clases que a la tasa global de acierto.

La Tabla 5.28 muestra la misma comparación para **Virchow2**.

Tabla 5.28: Ranking global de métodos de selección de patches en Virchow2, ordenado por *F1 macro* medio en validación

Puesto	Método	F1 medio	Disp.	Acc. media	Disp.	% elim.
1	Baseline	0.6011	0.0056	0.7892	0.0061	0.00 %
2	RandomUnder (70 %)	0.5920	0.0024	0.7908	0.0012	30.00 %
3	IM ($\alpha = 0,0002$)	0.5908	0.0027	0.7917	0.0023	29.85 %
4	NCR ($k = 45$)	0.5785	0.0030	0.7895	0.0026	25.52 %

En Virchow2 se observa una tendencia todavía más clara. El *baseline* vuelve a ser la mejor referencia, mientras que RandomUnderSampler se sitúa ligeramente por encima de la información mutua en *F1 macro*. En cambio, NCR recalibrado experimenta una caída mucho más marcada que en H-Optimus1 y pasa a ser la alternativa menos competitiva. Este resultado confirma que, en Virchow2, la aparente fortaleza inicial de NCR estaba muy ligada a su comportamiento conservador cuando eliminaba una proporción mucho menor de patches.

Finalmente, la Tabla 5.29 recoge el comportamiento de los dos modelos y de los cuatro enfoques comparados:

Tabla 5.29: Ranking final de métodos de selección de patches.

Modelo	Método	F1 macro	Accuracy	% elim.
H-Optimus1	Baseline	0.6118 ± 0.0080	0.7998 ± 0.048	0.00 %
H-Optimus1	IM ($\alpha = 0,00025$)	0.6012 ± 0.0031	0.7953 ± 0.0014	15.81 %
H-Optimus1	RandomUnder (70 %)	0.6001 ± 0.0024	0.7942 ± 0.0007	30.00 %
H-Optimus1	NCR ($k = 45$)	0.5980 ± 0.0024	0.7984 ± 0.0011	20.99 %
Virchow2	Baseline	0.6011 ± 0.0056	0.7892 ± 0.0061	0.00 %
Virchow2	IM ($\alpha = 0,0002$)	0.5908 ± 0.0027	0.7917 ± 0.0023	29.85 %
Virchow2	RandomUnder (70 %)	0.5920 ± 0.0024	0.7908 ± 0.0012	30.00 %
Virchow2	NCR ($k = 45$)	0.5785 ± 0.0030	0.7895 ± 0.0026	25.52 %

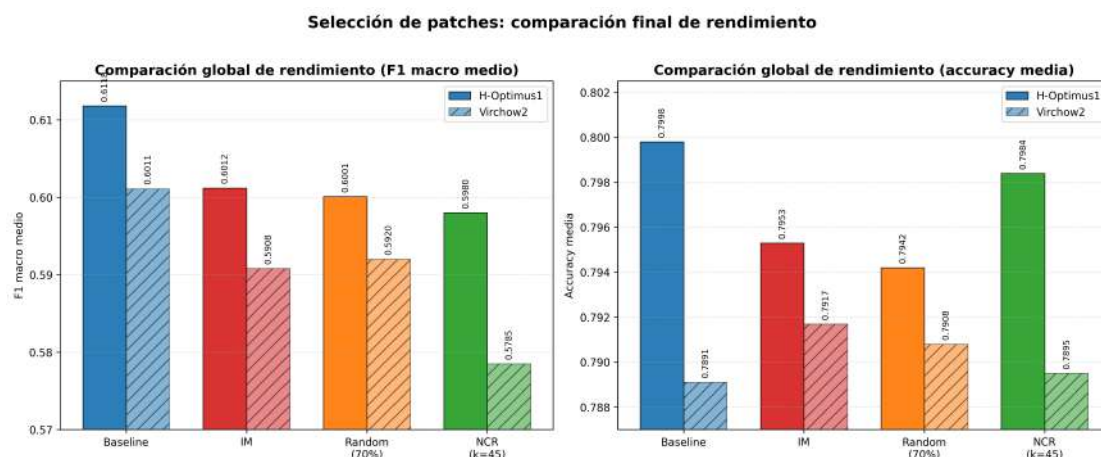


Figura 5.11: Comparación global del rendimiento medio en validación para los métodos evaluados sobre patches

Desde el punto de vista del rendimiento, la conclusión principal es clara: **ninguno de los métodos de selección de patches supera al baseline original en términos de $F1$ macro**. No obstante, la *accuracy* aporta un matiz complementario relevante. En particular, la limpieza basada en información mutua ofrece en algunos casos valores de *accuracy* superiores a los de otras alternativas, aunque no llegue a liderar la comparación en *F1 macro*. Esto sugiere que la selección de patches puede afectar de forma distinta al equilibrio entre clases y al porcentaje global de aciertos.

Además, cuando NCR se recalibra para aproximarlos a la escala de reducción de la información mutua y de RandomUnderSampler, pierde buena parte de la ventaja que había mostrado en su configuración inicial. Esto indica que su buen comportamiento previo estaba condicionado, al menos en parte, por una limpieza mucho más suave.

Junto a esta comparación de rendimiento, también resulta relevante incorporar una visión computacional del problema. La Tabla 5.30 resume el coste experimental asociado a cada estrategia, distinguiendo entre el coste de construcción del subconjunto y el tiempo de reentrenamiento posterior de la cabeza de clasificación.

Tabla 5.30: Comparación computacional de las estrategias evaluadas sobre patches

Modelo	Método	Constr. subconjunto	Reentrenamiento
H-Optimus1	Baseline	—	~1 h
H-Optimus1	IM ($\alpha = 0,00025$)	~36 h	9.6 s
H-Optimus1	RandomUnder (70 %)	~10 s	7.8 s
H-Optimus1	NCR ($k = 45$)	~26 s	8.8 s
Virchow2	Baseline	—	~1 h
Virchow2	IM ($\alpha = 0,0002$)	~36.5 h	8.4 s
Virchow2	RandomUnder (70 %)	~10 s	8.6 s
Virchow2	NCR ($k = 45$)	~42 s	8.9 s

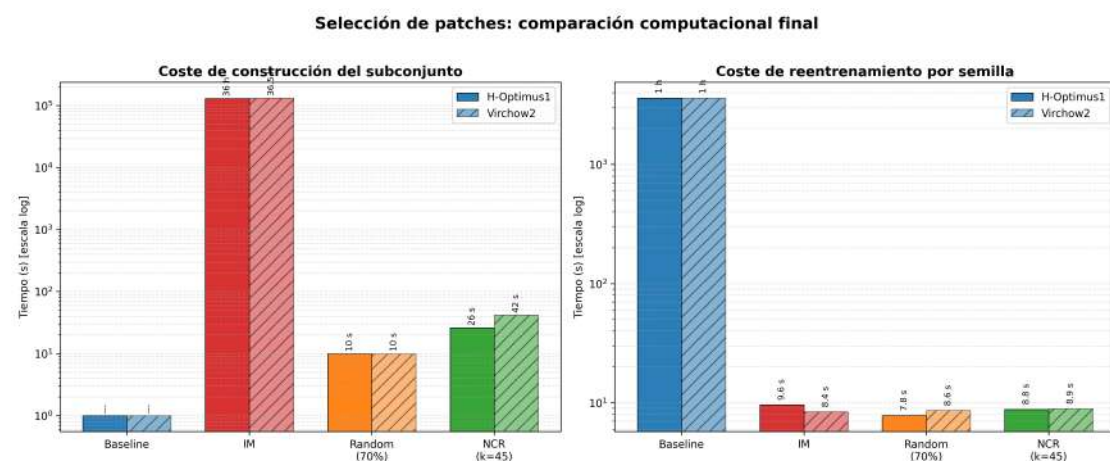


Figura 5.12: Comparación del coste de construcción del subconjunto y del tiempo de reentrenamiento por semilla

Aunque las distintas limpiezas no hayan sido la estrategia con mejor rendimiento final, nos han hecho sacar varias conclusiones:

- Puede reducirse el conjunto de entrenamiento en torno a un 30 % mediante selección de patches **casi manteniendo el rendimiento de los modelos**, aunque el criterio elegido influye de forma decisiva en el resultado final.
- Cuando la estrategia de reducción de NCR se fuerza para aproximarla a la escala de reducción de un 30 %, su rendimiento se degrada de forma apreciable.

- Los métodos estándar de *undersampling* no solo permiten reentrenar en apenas unos segundos, sino que también construyen el subconjunto reducido en tiempos muy inferiores a los requeridos por la información mutua. En particular, `RandomUnderSampler` es prácticamente inmediato, mientras que `NCR`, incluso tras su recalibración a $k = 45$, sigue siendo computacionalmente muy barato frente al coste de construcción de la limpieza informativa.
- La discusión conjunta de *F1 macro* y *accuracy* permite matizar mejor las conclusiones. Mientras que el *F1 macro* penaliza con mayor sensibilidad los desequilibrios entre clases y se mantiene como la referencia principal del estudio, la *accuracy* refleja que algunos métodos, en particular el de información mutua, ofrecen porcentajes de acierto competitivos. Por ello, las conclusiones de esta fase deben entenderse a la luz de ambas medidas, aunque priorizando el *F1 macro* como criterio de decisión principal.

A la vista de estas conclusiones, la etapa de experimentación sobre patches se da por cerrada. El trabajo realizado en esta fase ha permitido caracterizar con detalle la relación entre agresividad de la limpieza, rendimiento en validación y coste computacional, dejando así una base sólida para orientar la siguiente etapa del proyecto, centrada en el análisis a nivel de regiones de interés (*ROIs*).

5.4. Evaluación a nivel de ROIs a partir de predicciones sobre patches

Una vez completada la fase experimental a nivel de patches, acercamos nuestro análisis al problema real de clasificación histopatológica: las **regiones de interés** (*ROIs*). Hasta este punto, todos los modelos se han entrenado y validado tratando cada patch como una muestra independiente. Sin embargo, desde el punto de vista clínico y visual, la decisión final no se toma sobre un patch aislado, sino sobre una región completa que contiene múltiples patches relacionados entre sí.

Por ello, en esta nueva fase se plantea una evaluación a nivel de **test-ROI**, reutilizando los modelos ya entrenados sobre patches y agregando posteriormente sus predicciones para obtener una única etiqueta final por ROI.

Cada ROI del conjunto de test puede entenderse como una **bolsa de patches**, y cada uno de esos patches recibe una predicción por parte del modelo previamente entrenado. A partir de ese conjunto de predicciones locales, se construye una predicción global para la ROI mediante una regla de consenso o votación.

Este planteamiento nos permite obtener métricas a un nivel de análisis más representativo, al mismo tiempo que aprovecha directamente todo el trabajo ya realizado, sin necesidad de reentrenar los modelos para ROIs. En otras palabras, el aprendizaje sigue produciéndose a nivel de patch, pero la evaluación final puede trasladarse a nivel de ROI agregando las salidas ya disponibles del clasificador.

Además, esta transición de patches a ROIs resulta especialmente útil en el contexto de este trabajo, ya que permite estudiar si las distintas estrategias de limpieza del conjunto de entrenamiento a nivel de patch —*baseline*, información mutua, *RandomUnderSampler* y *NeighbourhoodCleaningRule*— tienen o no un efecto relevante cuando la decisión se formula sobre ROIs.

Por tanto, el objetivo de esta sección es **reutilizar los clasificadores ya ajustados a nivel de patch para obtener predicciones a nivel de ROI** y comparar su comportamiento sobre el conjunto de test.

5.4.1. Metodología de agregación patch-ROI

Esta fase se planteó para abordar específicamente el objetivo **OBJ1.5**, trasladando la decisión desde el nivel de patch al nivel de ROI mediante distintas reglas de agregación y analizando cuál de ellas representaba mejor la salida final del sistema en un contexto más próximo al uso real.

Para ello, se siguió un esquema de agregación sencillo pero completamente trazable. En primer lugar, para cada configuración experimental se obtuvieron las predicciones a nivel de patch sobre el conjunto de test. En el caso del *baseline*, dichas predicciones se generaron aplicando directamente el modelo fundacional entrenado sobre imágenes. En cambio, para las configuraciones derivadas de limpieza (*IM*, *RandomUnderSampler* y *NCR*), las predicciones se obtuvieron a partir de los embeddings de test y de la cabeza lineal correspondiente previamente entrenada.

En todos los casos, para cada patch se almacenaron:

- El identificador interno.
- La ruta y nombre de fichero.
- La etiqueta real.
- La etiqueta predicha.
- El vector completo de probabilidades por clase.

A partir del nombre del fichero de cada patch se reconstruyó el identificador de la ROI de origen, lo que permitió agrupar automáticamente todos los patches pertenecientes a una misma región. Una vez realizado ese agrupamiento, cada ROI quedó representada por:

- su identificador `roi_id`;
- su etiqueta real única y_{true} ;
- el número total de patches que la componen;
- y el conjunto de predicciones *patch-level* asociadas.

Sobre cada ROI se evaluaron tres reglas de agregación distintas, para así evaluar distintas estrategias de votación y analizar cuál resultaba más adecuada al trasladar la predicción desde patch hasta la ROI.

Media de probabilidades + *argmax*

La primera regla consiste en promediar, para cada clase, las probabilidades predichas en todos los patches de una misma ROI. Si una ROI contiene n patches y cada uno de ellos produce un vector de probabilidades $p_i \in \mathbb{R}^C$, se calcula primero el vector medio:

$$\bar{p} = \frac{1}{n} \sum_{i=1}^n p_i.$$

La etiqueta final de la ROI se obtiene entonces como la clase con mayor probabilidad media:

$$\hat{y}_{\text{ROI}} = \arg \max_c \bar{p}_c.$$

Esta estrategia aprovecha toda la información probabilística producida por el modelo sin reducir cada patch a una decisión dura demasiado pronto.

Votación mayoritaria

La segunda regla transforma primero cada patch en una clase predicha, y posteriormente asigna a la ROI la clase que aparece con mayor frecuencia entre todos sus patches. Formalmente, si $\hat{y}_1, \dots, \hat{y}_n$ son las predicciones *patch-level* de una ROI, la clase final es la moda de dicho conjunto.

Esta estrategia resulta especialmente interpretable, ya que la decisión final se basa en la clase predominante dentro de la ROI. Sin embargo, al convertir cada patch a una etiqueta discreta antes de la agregación, pierde la información asociada al nivel de confianza de cada predicción.

Regla de la clase más maligna

La tercera regla asigna a la ROI la clase más maligna presente entre las predicciones de sus patches. Dado que en este problema las clases están ordenadas desde la más benigna hasta la más maligna,

$$0 < 1 < 2 < 3 < 4 < 5 < 6,$$

la predicción final de la ROI se define simplemente como el máximo de las etiquetas predichas *patch-level*:

$$\hat{y}_{\text{ROI}} = \max_i \hat{y}_i.$$

Con esta estrategia basta con que aparezca evidencia local de una clase más severa para que la ROI completa herede dicha etiqueta. Desde un punto de vista clínico, esta aproximación puede resultar interesante como mecanismo de sensibilidad ante lesiones más agresivas. No obstante, también puede amplificar errores locales aislados y producir una sobreasignación hacia clases de mayor malignidad.

Estructura final de los artefactos generados

Como resultado de este proceso, para cada combinación de modelo y método se generaron dos niveles de artefactos para el conjunto de test:

- Un fichero de predicciones a nivel de patch, con probabilidades y etiquetas por patch;
- Un fichero de predicciones a nivel de ROI, con las tres decisiones finales obtenidas por cada regla de votación.

Sobre estos ficheros agregados se calcularon posteriormente las métricas finales a nivel de ROI, incluyendo *accuracy*, *F1 macro*, *classification report* y matrices de confusión normalizadas por fila.

5.4.2. Resultados globales de las reglas de votación

Una vez definidas las tres estrategias de agregación, se comparó su comportamiento sobre el conjunto de test-ROI para todas las configuraciones evaluadas. La Tabla 5.31 resume los resultados obtenidos en **H-Optimus1**.

Tabla 5.31: Comparación de reglas de votación a nivel de test-ROI en H-Optimus1

Método	Votación	Accuracy	F1 macro
Baseline	mean_proba	0.5895	0.5731
Baseline	majority_vote	0.5842	0.5660
Baseline	most_malignant	0.5246	0.5102
IM ($\alpha = 0,00025$)	mean_proba	0.5842	0.5554
IM ($\alpha = 0,00025$)	majority_vote	0.5684	0.5421
IM ($\alpha = 0,00025$)	most_malignant	0.5228	0.4953
RandomUnder	mean_proba	0.5719	0.5484
RandomUnder	majority_vote	0.5632	0.5386
RandomUnder	most_malignant	0.5158	0.4925
NCR ($k = 45$)	mean_proba	0.5614	0.5122
NCR ($k = 45$)	majority_vote	0.5561	0.5066
NCR ($k = 45$)	most_malignant	0.5158	0.4658

La misma comparación para **Virchow2** se recoge en la Tabla 5.32.

Tabla 5.32: Comparación de reglas de votación a nivel de test-ROI en Virchow2.

Método	Votación	Accuracy	F1 macro
Baseline	mean_proba	0.6088	0.5666
Baseline	majority_vote	0.6053	0.5623
Baseline	most_malignant	0.5386	0.4994
IM ($\alpha = 0,0002$)	mean_proba	0.5912	0.5615
IM ($\alpha = 0,0002$)	majority_vote	0.5842	0.5538
IM ($\alpha = 0,0002$)	most_malignant	0.5333	0.5066
RandomUnder	mean_proba	0.6018	0.5750
RandomUnder	majority_vote	0.5947	0.5635
RandomUnder	most_malignant	0.5386	0.5156
NCR ($k = 45$)	mean_proba	0.5789	0.5210
NCR ($k = 45$)	majority_vote	0.5754	0.5184
NCR ($k = 45$)	most_malignant	0.5193	0.4643

Los resultados muestran una tendencia muy clara en ambos modelos: la regla **mean_proba** proporciona los mejores valores de *accuracy* y de *F1 macro*. La votación mayoritaria queda normalmente en segunda posición, con resultados próximos pero inferiores, mientras que la regla de la clase más maligna resulta ser la más agresiva y la que peor se comporta.

Por tanto, para la comparación principal entre métodos a nivel de ROI, se toma **mean_proba** como la estrategia de agregación de referencia, al ser la que ofrece el mejor compromiso global entre estabilidad y rendimiento.

5.4.3. Comparación final entre métodos a nivel de ROI

Una vez comprobado que la regla de agregación **mean_proba** es la más sólida de las tres alternativas evaluadas, la comparación principal entre métodos a nivel de ROI se realizó exclusivamente con dicha estrategia. De este modo, la discusión final se centra en el comportamiento de cada configuración cuando las probabilidades *patch-level* se agregan mediante promedio y posterior *argmax*.

La Tabla 5.33 resume los resultados obtenidos para **H-Optimus1**.

Tabla 5.33: Comparación final a nivel de ROI en H-Optimus1.

Método	Accuracy	F1 macro
Baseline	0.5895	0.5731
IM ($\alpha = 0,00025$)	0.5842	0.5554
RandomUnder	0.5719	0.5484
NCR ($k = 45$)	0.5614	0.5122

En **H-Optimus1**, el *baseline* sigue siendo la mejor configuración tanto en *accuracy* como en *F1 macro*. No obstante, la limpieza basada en información mutua se mantiene

relativamente próxima a dicha referencia, mientras que *RandomUnderSampler* queda ligeramente por debajo y *NCR* recalibrado resulta claramente la alternativa más débil.

La misma comparación para **Virchow2** se muestra en la Tabla 5.34.

Tabla 5.34: Comparación final a nivel de ROI en Virchow2.

Método	Accuracy	F1 macro
RandomUnder	0.6018	0.5750
Baseline	0.6088	0.5666
IM ($\alpha = 0,0002$)	0.5912	0.5615
NCR ($k = 45$)	0.5789	0.5210

En **Virchow2** aparece el resultado más interesante de esta fase. El *baseline* mantiene la mejor *accuracy*, pero *RandomUnderSampler* alcanza el mejor valor de *F1 macro*, superando ligeramente a la referencia original. Este comportamiento sugiere que, en esta configuración concreta, la reducción aleatoria del conjunto de entrenamiento podría estar favoreciendo una decisión más equilibrada entre clases cuando la predicción final se formula a nivel de ROI. Por su parte, la limpieza por información mutua ofrece un comportamiento competitivo, aunque algo inferior, mientras que *NCR* vuelve a situarse como la alternativa más débil.

Para recoger conjuntamente ambos resultados, la Tabla 5.35 resume la comparación global final entre modelos y métodos a nivel de ROI.

Tabla 5.35: Comparación final entre métodos a nivel de ROI usando `mean_proba`

Modelo	Método	Accuracy	F1 macro
H-Optimus1	Baseline	0.5895	0.5731
H-Optimus1	IM ($\alpha = 0,00025$)	0.5842	0.5554
H-Optimus1	RandomUnder	0.5719	0.5484
H-Optimus1	NCR ($k = 45$)	0.5614	0.5122
Virchow2	RandomUnder	0.6018	0.5750
Virchow2	Baseline	0.6088	0.5666
Virchow2	IM ($\alpha = 0,0002$)	0.5912	0.5615
Virchow2	NCR ($k = 45$)	0.5789	0.5210

De esta comparación global pueden extraerse dos conclusiones principales. En primer lugar, el *baseline* sigue siendo una referencia muy sólida, especialmente en términos de *accuracy*. En segundo lugar, el paso a evaluación a nivel de ROI modifica parcialmente la lectura obtenida a nivel de patch: aunque algunas estrategias de limpieza no mejoraban claramente en métricas *patch-level*, al agregarse sobre regiones completas pueden dar a lugar a decisiones más equilibradas. Esto se observa de forma especialmente clara en **Virchow2 + RandomUnder**, que alcanza el mejor *F1 macro* a nivel de ROI entre

todas las configuraciones estudiadas, mientras que en **H-Optimus1** el *baseline* continúa siendo superior en ambas métricas.

Para analizar mejor los errores entre clases se muestran cuatro matrices de confusión, correspondientes al *baseline* y la mejor alternativa en cada modelo.

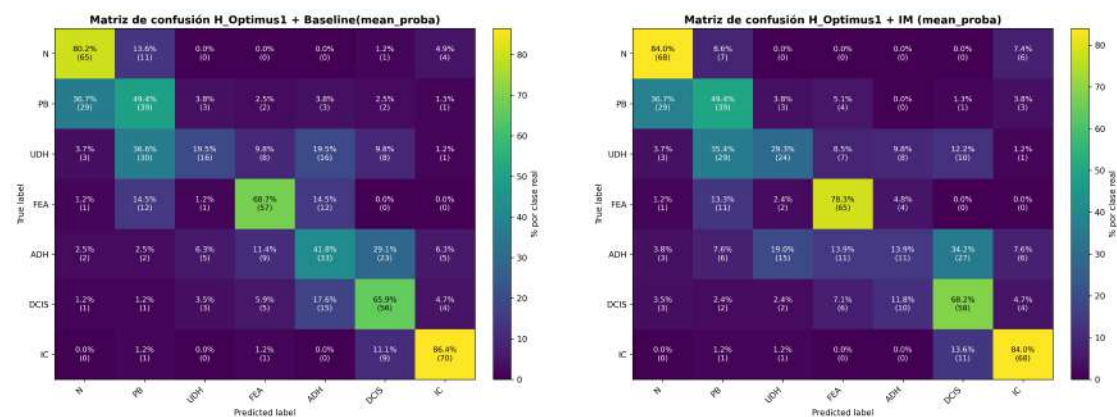


Figura 5.13: Matrices de confusión a nivel de ROI para H-Optimus1

En **H-Optimus1**, el *baseline* muestra un comportamiento especialmente sólido en las clases extremas o morfológicamente más estables. Las mejores tasas de acierto aparecen en **IC** (86,42 %), **N** (80,25 %), **FEA** (68,67 %) y **DCIS** (65,88 %). Esto indica que el modelo reconoce bien la frontera entre ROIs benignas de las malignas.

Las principales dificultades se concentran en las clases intermedias, especialmente en **PB**, **UDH** y **ADH**. En **PB**, aunque el acierto en diagonal alcanza un 49,37 %, aparece una confusión muy fuerte hacia **N** (36,71 %). En **UDH**, la diagonal cae hasta 19,51 %, y la clase se reparte entre **PB** (36,59 %) y **ADH** (19,51 %), además de mostrar errores hacia **FEA** y **DCIS**. La clase **ADH** vuelve a ser una de las más problemáticas: solo se acierta en un 41,77 % de los casos y se confunde muy frecuentemente con **DCIS** (29,11 %) y **FEA** (11,39 %) y otras clases intermedias. Esto resulta coherente con la cercanía entre estas categorías, ya que el modelo tiene la capacidad de distinguir razonablemente bien los extremos (benignas y malignas), pero tiene muchas dificultades para separar las lesiones atípicas.

La versión con **información mutua** conserva parte de esta estructura del *baseline*, aunque con cambios relevantes. Por un lado, mejora algunas diagonales concretas: **N** pasa de 80,25 % a 83,95 %, **FEA** mejora de 68,67 % a 78,31 %, y **DCIS** sube de 65,88 % a 68,24 %.

Sin embargo, el comportamiento global empeora por el deterioro de las clases más ambiguas. **UDH** mejora algo respecto al *baseline* (29,27 % frente a 19,51 %), pero sigue manteniendo una confusión muy fuerte con **PB** (35,37 %). Lo más importante aquí es lo ocurrido con **ADH**: su diagonal cae desde 41,77 % hasta 13,92 %, dispersándose sobre todo hacia **DCIS** (34,18 %), **UDH** (18,99 %) y **FEA** (13,92 %). Por tanto, esta limpieza no parece estar provocando pérdida de capacidad para reconocer las clases extremas,

sino que se degrada más aún la separación entre clases intermedias muy próximas entre sí. Esa pérdida de equilibrio en categorías de transición explica en gran medida la caída del *F1 macro* global.

En conjunto, la comparación en H-Optimus1 confirma que la limpieza por información mutua mantiene bastante bien el comportamiento general del modelo en clases extremas e incluso mejora algunas de ellas, pero no consigue mantener con solidez la separación entre clases intermedias, especialmente en el caso de **ADH**. Por ello, aunque el rendimiento global se mantiene razonablemente próximo al *baseline*, la mejora local en algunas diagonales no compensa el empeoramiento en las clases más próximas y ambiguas.

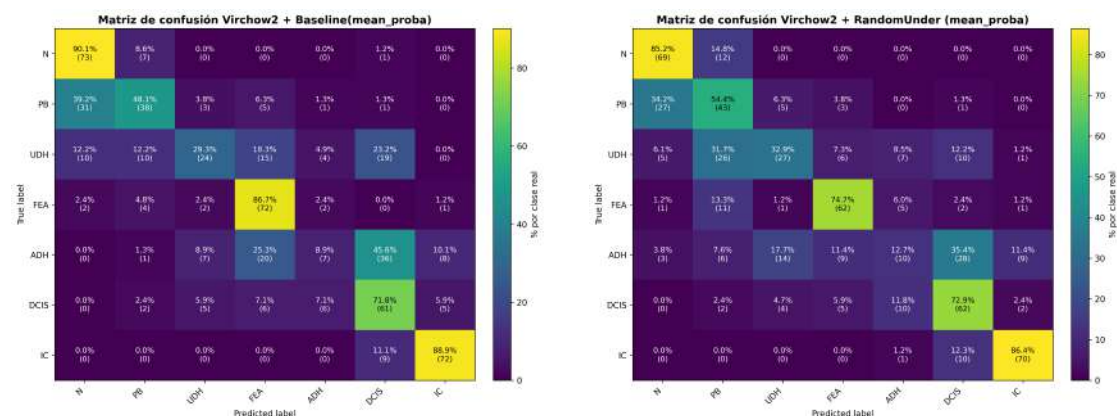


Figura 5.14: Matrices de confusión a nivel de ROI para Virchow2

En **Virchow2**, el *baseline* presenta un patrón muy fuerte en algunas clases concretas, destacando especialmente **N** (90,12%), **IC** (88,89%) y **FEA** (86,75%), con diagonales muy elevadas. También **DCIS** mantiene un comportamiento bueno (71,76%). Esto explica que el modelo alcance la mejor *accuracy* global, ya que acierta con mucha claridad en clases bien definidas.

Aun así, vuelven a aparecer dificultades importantes en las clases intermedias. **PB** solo alcanza un 48,10% de acierto y vuelve a confundirse fuertemente con **N** (39,24%). **UDH** presenta una diagonal modesta (29,27%) y reparte bastantes errores entre **DCIS** (23,17%), **FEA** (18,29%) y tanto **N** como **PB** (12,20% en ambos casos). De nuevo, la situación crítica vuelve a encontrarse en **ADH**: su diagonal es solo del 8,86%, con una fuerte desviación hacia **DCIS** (45,57%) y **FEA** (25,32%). De nuevo vemos cómo el comportamiento es muy fiable en clases extremas y bien definidas, sufriendo claramente en la frontera entre lesiones proliferativas e intraductales de complejidad intermedia.

La versión con **RandomUnderSampler** modifica este equilibrio de una forma bastante interesante. En algunas clases fuertes el comportamiento empeora ligeramente: **N** baja de 90,12% a 85,19%, **FEA** cae de 86,75% a 74,70%, **IC** de 88,89% a 86,42%, y **DCIS** se mantiene prácticamente en el mismo rango (72,94%). Es decir, el modelo pierde algo de precisión en clases donde el *baseline* era dominante.

Sin embargo, esa pérdida se compensa con una mejora en varias clases más difíciles. **PB** sube de 48,10 % a 54,43 %, y **UDH** mejora de 29,27 % a 32,93 %. Aunque **ADH** sigue siendo la clase más complicada, también muestra una ligera mejora en diagonal, pasando de 8,86 % a 12,66 %, aunque aún muy confundida con **DCIS** (35,44 %), **UDH** (17,72 %) y **FEA** (11,39 %). En consecuencia, *RandomUnderSampler* no elimina el problema estructural de **ADH**, pero redistribuye de otra forma parte de los errores del modelo y mejora el equilibrio general en varias clases medias.

Este comportamiento permite interpretar mejor el resultado cuantitativo observado en **Virchow2**: el *baseline* conserva una *accuracy* superior porque sigue siendo más fuerte en varias clases más fáciles de separar, mientras que *RandomUnderSampler* alcanza un mejor *F1 macro* al mejorar relativamente el tratamiento de clases más difíciles como **PB** y **UDH**, sin degradar de forma extrema el resto. Conviene subrayar que este fenómeno se observa específicamente en **Virchow2**, ya que en **H-Optimus1** el *baseline* se mantiene por delante tanto en *accuracy* como en *F1 macro*.

Una posible explicación de este comportamiento podría estar relacionada con el propio **desbalance del conjunto de entrenamiento**. Aunque el conjunto de test-ROI utilizado para la evaluación final presenta una distribución bastante equilibrada entre clases, el entrenamiento a nivel de *patch* se realiza sobre un conjunto desbalanceado (véase Tabla 3.4). Desde este punto de vista, *RandomUnderSampler* podría estar contribuyendo a una cierta homogeneización del conjunto de entrenamiento, reduciendo la sobrerrepresentación de algunas clases y favoreciendo así una decisión más equilibrada a nivel de ROI. En cualquier caso, esta interpretación debe entenderse como una hipótesis razonable apoyada en los resultados observados, y no como una demostración causal cerrada.

En conjunto, el análisis de las matrices de confusión refuerza la lectura de las métricas globales. En **H-Optimus1**, el *baseline* sigue siendo la opción más robusta, ya que la limpieza por información mutua no consigue preservar adecuadamente algunas clases intermedias críticas. En **Virchow2**, en cambio, *RandomUnderSampler* parece producir una redistribución de errores más favorable desde el punto de vista multiclase, lo que justifica que alcance el mejor *F1 macro* a nivel de ROI aun sin superar al *baseline* en *accuracy*. Por último, en ambos backbones se observa de manera consistente que las clases más conflictivas son las intermedias, especialmente **PB**, **UDH** y, sobre todo, **ADH**, mientras que **N**, **FEA**, **DCIS** e **IC** suelen presentar diagonales considerablemente más altas.

5.4.4. Discusión

La evaluación a nivel de ROI permite dar respuesta a los objetivos **OBJ1.5** y **OBJ1.8**, extrayendo una lectura más rica del problema real que la obtenida únicamente a nivel de patch. A partir de los resultados, pueden sacarse cinco conclusiones principales.

- **La mejor regla de agregación es mean_proba.** Entre las tres estrategias evaluadas, la media de probabilidades seguida de *argmax* es la que ofrece el mejor

comportamiento global.

- **La limpieza de patches no destruye el rendimiento al pasar a ROIs.** En **H-Optimus1**, el *baseline* sigue siendo la configuración más robusta, aunque la limpieza por información mutua mantiene un comportamiento relativamente próximo. En **Virchow2**, el resultado más destacado es que *RandomUnderSampler* alcanza el mejor *F1 macro* a nivel de ROI, superando ligeramente al *baseline*, aunque este conserva la mejor *accuracy*. Por tanto, las estrategias de reducción o limpieza aplicadas a nivel de patch no provocan una degradación drástica al pasar a la escala regional, e incluso pueden resultar competitivas en determinadas condiciones.
- **La principal limitación metodológica está en la asignación de etiquetas a nivel de patch.** Este punto es el más importante de este análisis: en esta fase, cada patch hereda la etiqueta global de la ROI a la que pertenece, pero eso no garantiza que todos los patches de esa ROI representen realmente la misma categoría histológica. En consecuencia, pueden existir patches sanos, poco representativos o ambiguos dentro de una ROI patológica, lo que introduce **ruido de etiquetado** en el problema a nivel de patches. Esto ayuda a explicar por qué algunas estrategias de limpieza o reducción no empeoran el rendimiento e incluso pueden mejorar métricas como el *F1 macro* a nivel de ROI, ya que solo están eliminando redundancia.
- **Desde el punto de vista del dominio, la principal dificultad no está en los extremos, sino en las clases intermedias.** El análisis de las matrices de confusión muestra que las clases más estables suelen ser **N** e **IC**, junto con otras relativamente bien definidas como **FEA** y **DCIS**. En cambio, las mayores dificultades aparecen de forma sistemática en clases intermedias como **PB**, **UDH** y, sobre todo, **ADH**. Por tanto, el problema no está en distinguir regiones benignas de regiones malignas, sino en separar correctamente categorías próximas o de transición.
- **El buen comportamiento de *RandomUnderSampler* en Virchow2 no tiene una explicación completamente cerrada.** La mejora observada en *F1 macro* no implica necesariamente que el método esté seleccionando patches “mejores”. Una interpretación lógica es que, al haber ruido en el etiquetado a nivel de patches y una gran redundancia, una reducción aleatoria pueda disminuir parte de esa redundancia y amortiguar el efecto de ejemplos poco informativos o mal alineados con la etiqueta heredada de la ROI. En este sentido, su rendimiento puede entenderse no solo como un efecto del submuestreo en sí, sino también como una consecuencia de la limitación del problema a nivel de etiquetado de patches.

En conjunto, esta fase confirma que la evaluación a nivel de ROI aporta una visión más informativa que la obtenida a nivel de patch. Además, permite observar que las estrategias de limpieza o reducción no deben juzgarse solo por su efecto sobre métricas

globales, sino también por cómo redistribuyen el error entre clases y por cómo interactúan con el ruido de la asignación de etiquetas a nivel de patches.

5.5. Clasificación a nivel de ROI con abstención y revisión de casos dudosos

Aunque las tres reglas de votación consideradas en la sección anterior permiten asignar una etiqueta final a cada ROI, todas ellas **obligan al sistema a tomar una decisión incluso cuando es ambigua**. En un problema histopatológico esta situación es muy importante, ya que existen clases y regiones de transición en las biopsias que incluso un especialista puede requerir una valoración más cuidadosa.

Es por ello que para abordar el objetivo **OBJ1.6**, se introdujo una regla de decisión con abstención que permitiera al sistema no forzar una clasificación cerrada en aquellas ROIs donde el modelo dudara.

Como ya comentamos a raíz de los resultados obtenidos, se comprobó que gran parte de los errores a nivel de ROI se concentraban en las clases intermedias, especialmente **PB**, **UDH**, **FEA** y, sobre todo, **ADH**. Además, el análisis de las matrices de confusión mostró que muchas de las confusiones no eran arbitrarias, sino que aparecían entre pares de clases muy cercanas, lo que muestra que el modelo no solo se equivoca, sino que en muchos casos se enfrenta a muestras muy difíciles de evaluar.

Por este motivo, se plantea una cuarta estrategia de decisión a nivel de ROI basada en **abstención**. La idea consiste en no forzar al modelo a una predicción en todos los casos, sino en permitirle que identifique aquellas ROIs en las que no tiene suficiente confianza para emitir una decisión fiable. En un contexto real, estas muestras serían derivadas a revisión por parte de uno o varios patólogos, en lugar de dar automáticamente una etiqueta ambigua.

Este planteamiento nos permite evaluar si el modelo es capaz de reconocer cuándo está ante un caso dudoso, además de ofrecernos una visión más rica del problema no solo midiendo rendimiento global, sino también **qué tipo de muestras generan incertidumbre y a qué clases pertenecen**.

5.5.1. Regla de decisión con abstención

La nueva estrategia parte de la regla de votación con mejor comportamiento global de la sección anterior, la media de probabilidades a nivel de ROI (**mean_proba**). En consecuencia, para cada ROI se calcula en primer lugar el vector medio de probabilidades de clase a partir de todos los patches que la componen:

$$\bar{\mathbf{p}} = (\bar{p}_0, \bar{p}_1, \dots, \bar{p}_6),$$

donde $\bar{p}_c$ representa la probabilidad media asignada a la clase c en dicha ROI. A partir de este vector, la predicción estándar se obtiene tomando la clase con mayor probabilidad media:

$$\hat{y}_{ROI} = \arg \max_c \bar{p}_c.$$

Sin embargo, en lugar de aceptar siempre esta decisión, se introduce un mecanismo de control. Para ello, se ordenan las probabilidades medias de mayor a menor y se identifican las dos clases más probables, denotadas por $p_{(1)}$ y $p_{(2)}$, con $p_{(1)} \geq p_{(2)}$. A continuación, se define el **margen de confianza** como:

$$\Delta = p_{(1)} - p_{(2)}.$$

La interpretación de este margen es:

- Si Δ es grande, el modelo tiene una clara preferencia por una clase sobre las demás, lo que indica una predicción fiable.
- Si Δ es pequeño, el modelo está dudando entre al menos dos clases, lo que indica que la decisión es más incierta.

A partir de este valor, se definió una regla de abstención parametrizada por un umbral τ :

$$\begin{aligned} &\text{si } \Delta \geq \tau, \text{ la ROI se acepta y se asigna la clase } \arg \max_c \bar{p}_c, \\ &\text{si } \Delta < \tau, \text{ la ROI se marca como } \textit{requiere revisión}. \end{aligned}$$

Con el fin de mantener una trazabilidad completa, para cada ROI se almacena no solo la clase predicha principal, sino también las dos siguientes clases más probables, sus probabilidades asociadas y el margen Δ . De este modo, es posible analizar no solo cuántas muestras quedan fuera del cómputo automático, sino también **entre qué clases duda el modelo y qué tipos de regiones generan mayor incertidumbre**.

5.5.2. Selección del umbral de abstención

Para fijar el valor del umbral τ que determina cuándo una ROI se acepta automáticamente y cuándo se deriva a revisión, se realizó un barrido sobre varios valores de τ , concretamente $\{0,05, 0,10, 0,15, 0,20, 0,25\}$, analizando su efecto sobre la **coverage** (cobertura) del sistema, entendida como la fracción de ROIs aceptadas automáticamente, y el **F1 macro** obtenido sobre dichas ROIs aceptadas, para así tener un balance entre rendimiento y cantidad de muestras en las que el modelo muestra incertidumbre.

Con el fin de no introducir un ajuste específico para cada método, se optó por fijar un **umbral común** para todas las configuraciones evaluadas.

La Figura 5.15 resume visualmente este compromiso entre cobertura y rendimiento para las cuatro configuraciones más representativas de la sección anterior: **H-Optimus1 + Baseline**, **H-Optimus1 + Información Mutua**, **Virchow2 + Baseline** y **Virchow2 + RandomUnderSampler**. Cada punto representa un valor distinto de τ , de

forma que el desplazamiento hacia la izquierda indica una peor cobertura, mientras que el desplazamiento hacia arriba refleja una mejora del $F1$ macro sobre las ROIs aceptadas.

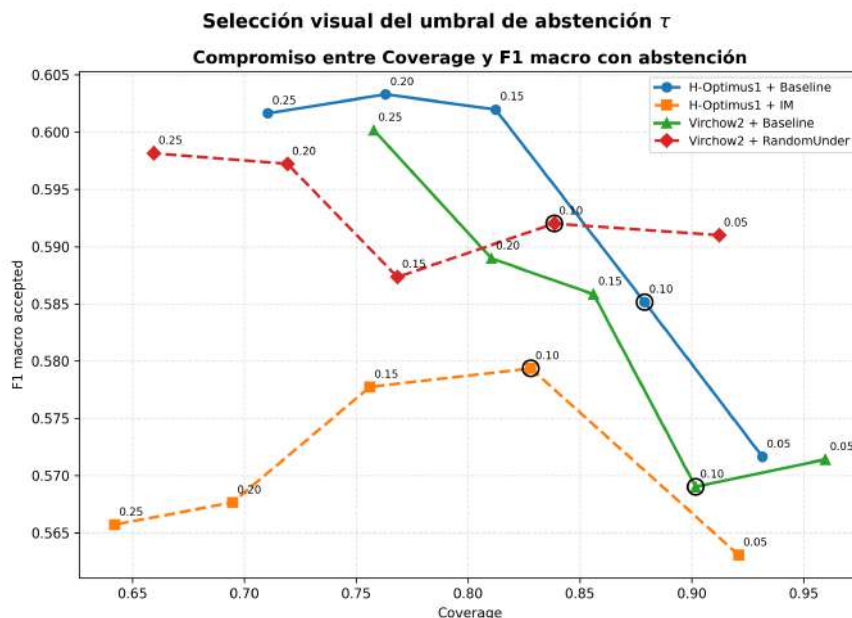


Figura 5.15: Compromiso entre *coverage* y $F1$ macro sobre las ROIs aceptadas al variar el umbral de abstención τ .

Como puede observarse, valores altos de τ permiten incrementar en algunos casos el $F1$ macro sobre las muestras aceptadas, pero a costa de reducir de forma agresiva la cobertura del sistema, es decir, aumentando el número de ROIs enviadas a revisión. En cambio, valores muy bajos apenas introducen abstención y, por tanto, apenas aprovechan la posibilidad de filtrar los casos más ambiguos. En este contexto, $\tau = 0,10$ se identificó como el valor que ofrecía el **mejor compromiso**, mejorando de forma consistente el rendimiento sobre las ROIs aceptadas, pero manteniendo aún una cobertura elevada, generalmente en el rango aproximado del 83 % al 90 %.

Por este motivo, se fijó $\tau = 0,10$ como umbral global de abstención para todas las configuraciones estudiadas.

5.5.3. Resultados de la abstención

Una vez fijado el umbral global de abstención en $\tau = 0,10$, se evaluó de nuevo el comportamiento de las ocho configuraciones, distinguiendo entre ROIs aceptadas por el sistema y ROIs derivadas a revisión.

La Tabla 5.36 y la gráfica 5.16 resumen los resultados obtenidos. Como era de esperar, en todos los casos **la abstención produce una mejora de las métricas sobre las ROIs aceptadas**, lo que indica que el criterio usado entre las dos clases más probables

filtra una parte relevante de los casos ambiguos manteniendo además una cobertura elevada, situada aproximadamente entre el 83 % y el 90 %, lo que refuerza la idea de que la mejora no es a costa de rechazar una fracción excesiva del conjunto de test.

Tabla 5.36: Resultados globales a nivel de ROI con abstención ($\tau = 0,10$)

Modelo	Método	ROIs aceptadas	ROIs revisión	Coverage	Accuracy accepted	F1 macro accepted
H-Optimus1	Baseline	501	69	0.8789	0.6108	0.5852
H-Optimus1	IM	472	98	0.8281	0.6292	0.5794
H-Optimus1	RandomUnder	471	99	0.8263	0.6030	0.5632
H-Optimus1	NCR ($k = 45$)	500	70	0.8772	0.6040	0.5324
Virchow2	Baseline	514	56	0.9018	0.6284	0.5690
Virchow2	IM	486	84	0.8526	0.6358	0.5803
Virchow2	RandomUnder	478	92	0.8386	0.6381	0.5920
Virchow2	NCR ($k = 45$)	509	61	0.8930	0.6169	0.5305

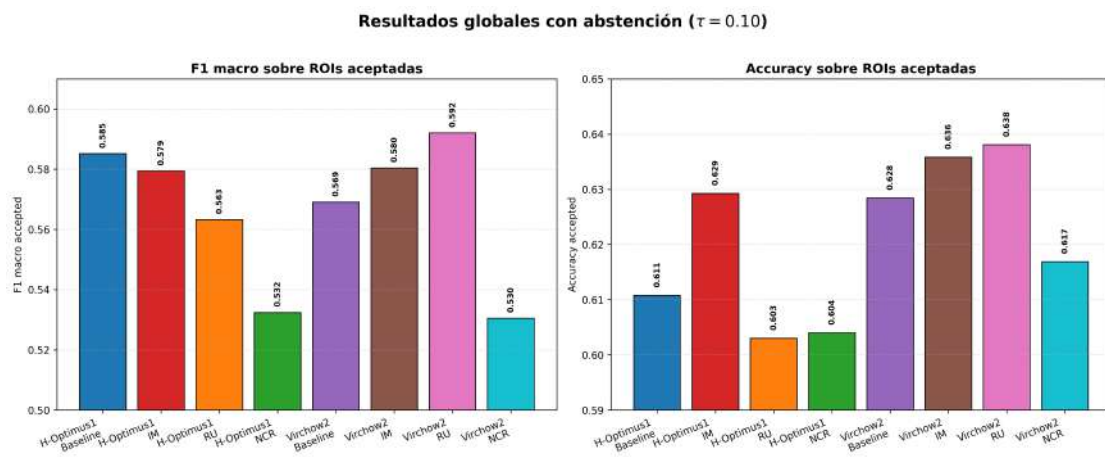


Figura 5.16: Gráfico de barras de los resultados con abstención ($\tau = 0,10$).

Por un lado, a nivel de **H-Optimus1**, el *baseline* continúa siendo la configuración más sólida en términos de *F1* macro sobre las ROIs aceptadas(0,5852), aunque la versión limpia con información mutua presenta la mejor *accuracy* (0,6292). Esto sugiere que, bajo un criterio de abstención, la configuración con información mutua resulta algo más conservadora y selectiva, mientras que el *baseline* mantiene un equilibrio más favorable en una métrica balanceada. *RandomUnderSampler* y *NCR* quedan claramente por debajo, incluso tras excluir las ROIs más inciertas.

Por otro lado, en **Virchow2**, la situación resulta más interesante aún. La configuración con *RandomUnderSampler* alcanza la mejor *accuracy* sobre las ROIs aceptadas (0,6381) y el mejor *F1* macro (0,5920), superando tanto al *baseline* como al resto de alternativas. De este modo, el resultado que ya se había observado previamente a nivel de ROI se refuerza también bajo esta regla de abstención.

En conjunto, estos resultados muestran que la abstención ofrece una mejora bastante notable del rendimiento sin comprometer de forma excesiva la cobertura. Además, la

comparación entre métodos no cambia respecto a la evaluación sin abstención, sino que la refuerza aún más: en **H-Optimus1** el *baseline* sigue apareciendo como la opción más robusta, mientras que en **Virchow2** la alternativa con *RandomUnderSampler* se consolida como la configuración más prometedora.

5.5.4. Análisis de casos dudosos y claros

Para llevar a cabo un análisis mucho más rico de las ROIs enviadas a revisión y como complemento a los objetivos **OBJ1.6** y **OBJ1.7**, se seleccionaron tres casos que permiten ilustrar visualmente distintos comportamientos de los modelos. En concreto, se escogió un caso de confusión entre clases benignas próximas (PB, N), un caso de transición entre lesiones intermedias de mayor complejidad (FEA, ADH) y un caso claro en el que el modelo realiza una predicción con alta confianza.



Figura 5.17: Confusión benigna entre N y PB

La Figura 5.17 muestra un escenario de incertidumbre entre clases benignas. En este caso, la etiqueta real de la ROI es tejido **Normal (N)**, pero el modelo asigna probabilidades idénticas a la clase **N** y a la clase **PB**.

Este comportamiento se debe a la heterogeneidad del tejido de la ROI, la cual está compuesta por unidades lobulillares de conductos terminales dentro de una matriz de tejido conectivo y un abundante estroma graso (caracterizado por las grandes vacuolas ópticamente blancas de los adipocitos). Al analizar los dos parches visualmente se puede entender la ambigüedad del modelo:

- **Patch 1 (Sesgo hacia N):** En este parche se aprecian las glándulas mamarias con una forma circular muy limpia. Se puede ver que mantienen sus centros completamente huecos o blancos y sin acumulaciones raras en su interior. Además, estas estructuras redondas conectan directamente con las grandes zonas blancas de la imagen (esquina superior izquierda), que son el tejido graso natural de una mama sana. Este aspecto tan “limpio” empuja al modelo a asignar probabilidad a la clase Normal (**N**).
- **Patch 2 (Sesgo hacia PB):** Por el contrario, este parche muestra un grupo de glándulas mucho más apretadas, oscuras y juntas entre sí. Aquí se nota que casi no hay espacio de fondo rosado (el tejido conectivo, estroma) separando los círculos morados debido a este apiñamiento. Esta aglomeración de estructuras es un rasgo visual clásico que puede entenderse como alteraciones benignas leves, donde el tejido cambia ligeramente pero sigue sin ser cancerígeno. Al ver esta concentración tan alta de estructuras moradas, el modelo duda y se inclina con la misma fuerza hacia la clase Benigna Patológica (**PB**).

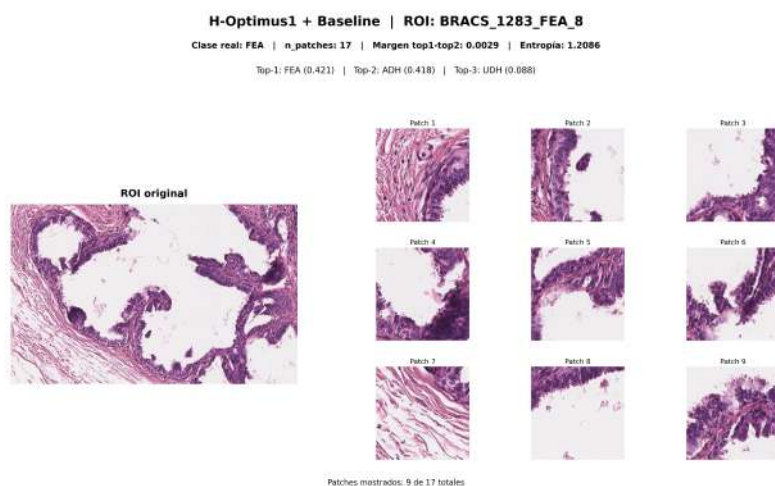


Figura 5.18: Confusión entre FEA y ADH

Si el caso anterior mostraba una frontera difusa dentro de lo benigno, la Figura 5.18 ilustra el escenario más complejo que venimos discutiendo durante todo el proyecto: la transición entre lesiones intermedias o precancerosas. En esta muestra, la etiqueta real de la ROI es **FEA**, pero el modelo entra en un empate técnico entre **FEA** y **ADH**. Un margen de diferencia insignificante ($\Delta = 0,0029$) que refleja que el clasificador se encuentra ante una muestra que en la práctica real obligaría a solicitar una segunda opinión médica.

Para entender este comportamiento sin necesidad de ser un experto clínico, basta con observar la estructura geométrica de la ROI original. Se aprecia un gran conducto mamario que se ha ensanchado o dilatado, dejando un enorme espacio blanco en el centro.

La clave del problema está en cómo se comportan las células moradas de la pared de ese hueco. Conceptualmente, si las células se limitan a forrar la pared de forma plana, la lesión es **FEA**; pero si empiezan a multiplicarse demasiado y a crecer hacia dentro del hueco formando bultos o puentes, la lesión podría llegar a **ADH**.

Visualmente se observa que existen a la vez ambos patrones visuales en la misma biopsia:

- **Patches con sesgo hacia FEA (ej. Patch 4 y Patch 8):** Estos recortes muestran la esencia de la Atipia Epitelial Plana. En ellos se ve una frontera limpia entre la zona blanca (el interior del conducto) y una banda delgada de células moradas en el borde sin llegar a invadir el espacio vacío. Al procesar estos parches, el modelo extrae características que lo llevan hacia la clase **FEA**.
- **Patches con sesgo hacia ADH (ej. Patch 6 y Patch 9):** Por el contrario, en estos parches esa arquitectura ordenada se rompe. En el Patch 6 se aprecia claramente cómo un grupo de células moradas empieza a sobresalir de la pared, formando una especie de “bulto” que invade el espacio blanco. En el Patch 9 se observa un apiñamiento mucho más grueso y desorganizado de células que altera el contorno plano del conducto. Estas estructuras más “raras” son marcas visuales típicas de la clase **ADH**.
- **Patches con tejido de ruido (ej. Patch 7):** Este recorte actúa prácticamente como ruido visual, ya que captura únicamente el fondo rosado y fibroso que rodea a las glándulas (el estroma). Al no contener células, no aporta información útil para desempatar la predicción, lo que contribuye a mantener esa incertidumbre del modelo.

Este análisis demuestra que el modelo H-Optimus1 no tiene un error de cálculo, sino que demuestra una sensibilidad muy buena para detectar que en una misma biopsia conviven regiones donde las células aún se mantienen planas en la pared (FEA) junto a regiones donde ya han empezado a proliferar hacia el interior del conducto (ADH). La necesidad de revisar este tipo de empates morfológicos es imprescindible para simular el comportamiento prudente que adoptaría un patólogo humano ante una lesión de este tipo.

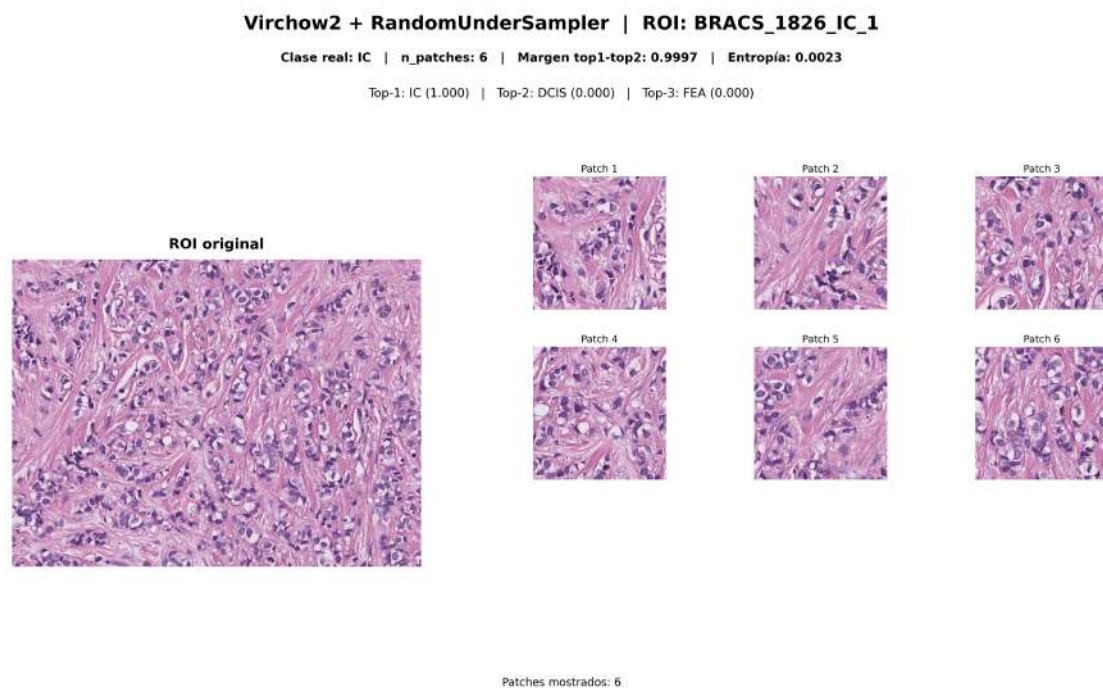


Figura 5.19: Caso de éxito y alta confianza en IC

Finalmente, la Figura 5.19 ilustra un caso de éxito rotundo donde el modelo realiza una predicción con una confianza del 100 % a favor de la clase real, **Carcinoma Invasivo (IC)**. En este ejemplo, el margen de diferencia entre el top 1 y 2 alcanza su valor casi máximo (0,9997) y la entropía cae a niveles prácticamente nulos (0,0023), mostrando matemáticamente que el clasificador no tiene ninguna duda, lo que permite validar la muestra automáticamente sin necesidad de derivarla a revisión.

La diferencia visual de esta ROI frente a las anteriores es evidente a primera vista. En los casos benignos o intermedios se distinguían estructuras circulares (los conductos) con centros huecos y blancos. Aquí, esa arquitectura está totalmente destruida. El tejido ha sido invadido por un caos de células cancerígenas (los puntos de color morado oscuro) que ya no respetan ninguna pared y se desparpaman sin control sobre el fondo rosado (el estroma fibroso).

Al analizar los parches individuales utilizados para la votación, se comprende de inmediato el porqué de la alta confianza del modelo:

- **Homogeneidad absoluta en la malignidad (todos los patches):** A diferencia de los casos dudosos, donde un parche mostraba una tendencia y otro la contraria, aquí todos los recortes muestran exactamente el mismo patrón. En los seis parches se puede observar cómo las células moradas se agrupan desorganizadamente, formando filas que rompen y “se comen” el tejido rosado.
- **Características morfológicas extremas:** Las células individuales muestran rasgos claros de malignidad que el modelo identifica con facilidad al ser mucho más

grandes de lo normal, con formas muy irregulares y asimétricas, y con un color morado muy intenso.

En conjunto, estos tres casos de estudio refuerzan la interpretación global obtenida en esta fase. Por un lado, muestran que las muestras enviadas a revisión no son aleatorias, sino que muestran fronteras diagnósticas interesantes. Por otro, confirman que cuando la arquitectura tisular presenta patrones claramente malignos y homogéneos, el modelo responde con una seguridad muy alta. En consecuencia, la regla de abstención no solo mejora el rendimiento sobre las ROIs aceptadas, sino que además permite identificar de forma explícita aquellas muestras en las que la propia dificultad morfológica del problema justifica una revisión experta.

Capítulo 6

Trabajo Extra

6.1. Análisis no supervisado de la estructura del dataset

Hasta este punto del proyecto, los resultados ya han mostrado un patrón bastante repetitivo y claro: las clases extremas se diferencian de forma más clara, mientras que las clases intermedias concentran la mayor parte de las confusiones, tanto en las matrices de confusión como en los casos enviados a revisión.

A partir de esta observación, surge una pregunta natural: **¿hasta qué punto estas dificultades proceden de los modelos y hasta qué punto proceden de una estructura intrínsecamente compleja del propio dataset?** Para abordar esta cuestión, se decidió incorporar una fase adicional apoyada en técnicas de aprendizaje no supervisado sobre el conjunto de entrenamiento.

6.1.1. Motivación de la fase no supervisada

La motivación principal de esta fase es analizar cómo se organiza el dataset en el espacio latente aprendido por los modelos fundacionales, sin imponer las etiquetas durante el proceso de agrupamiento. Así, en lugar de preguntarnos únicamente si un clasificador es capaz de separar las siete subclases del problema, estudiamos si esas subclases aparecen de forma natural como grupos bien diferenciados o si existen regiones del espacio en las que varias categorías quedan mezcladas.

Esto nos permite estudiar la calidad estructural del dataset desde un punto de vista distinto al aprendizaje supervisado. Si las clases reales se separan de forma clara en el espacio de embeddings, cabría esperar que los métodos no supervisados recuperasen agrupaciones relativamente limpias. En cambio, si incluso en ese espacio latente siguen apareciendo mezclas importantes entre subclases, constituye una evidencia clara de que la dificultad del problema no depende solo del clasificador final, sino de la complejidad intrínseca del propio dataset.

Por tanto, con esta fase buscamos enriquecer el estudio previamente hecho, estudiando si las subclases histológicas más conflictivas permanecen solapadas incluso desde una perspectiva no supervisada.

6.1.2. Metodología

El análisis no supervisado se realizó sobre los *embeddings* del conjunto de entrenamiento de patches extraídos con los dos modelos fundacionales analizados en el trabajo: **Virchow2** y **H-Optimus1**.

Como primer paso, se aplicó **PCA** (*Principal Component Analysis*), una técnica de reducción de dimensionalidad que transforma un espacio de muchas características en otro más pequeño conservando, en la medida de lo posible, la mayor parte de la variabilidad de los datos. En este trabajo se utilizó, por un lado, para facilitar la visualización del espacio latente en dos dimensiones y, por otro lado, para obtener una representación más manejable sobre la que aplicar posteriormente los algoritmos de agrupamiento.

Una vez obtenida esta representación, se aplicó **K-Means**, un algoritmo no supervisado que intenta dividir las muestras en k grupos, asignando cada punto al centroide más cercano y reajustando dichos centroides de manera iterativa hasta estabilizar la partición. Es decir, K-Means busca formar grupos compactos internamente pero lo más separados posible entre sí. En este estudio se evaluaron distintos valores, concretamente $k \in \{3, 5, 7, 10\}$, probando valores bajos como $k = 3$ para comprobar si el dataset se organiza en 3 grandes regiones (benigna, maligna e intermedia) y valores más altos para analizar si las subclases histológicas se distinguen como grupos más específicos o si permanecen mezcladas.

La calidad de los agrupamientos se evaluó mediante tres métricas:

- **Silhouette score:** mide la calidad geométrica del agrupamiento, evaluando si las muestras quedan próximas a las de su propio cluster y alejadas de los clusters vecinos. Los valores van de -1 a 1, siendo valores cercanos a 1 indicativos de clusters bien separados, mientras que valores cercanos a 0 sugieren que las muestras están en la frontera entre dos grupos y valores negativos indican que las muestras podrían estar asignadas al cluster equivocado.
- **ARI (*Adjusted Rand Index*):** cuantifica el grado de coincidencia entre los clusters obtenidos y las etiquetas reales, corrigiendo además el efecto del azar. Un valor de ARI cercano a 1 indica una alta coincidencia con las etiquetas reales, mientras que un valor cercano a 0 sugiere que la asignación es similar a una aleatoria y valores negativos indican que la asignación es peor incluso que una aleatoria.
- **NMI (*Normalized Mutual Information*):** mide la cantidad de información compartida entre la partición generada por el clustering y las etiquetas reales. Un valor de NMI cercano a 1 indica que las agrupaciones creadas coinciden exactamente con las etiquetas reales y un valor cercano a 0 sugiere que no hay relación entre ambas particiones.

Finalmente, para interpretar los resultados de forma visual y no solo numérica, se generaron representaciones bidimensionales del espacio latente coloreadas tanto por clase real como por cluster asignado, además de tablas de contingencia y mapas de calor que muestran la distribución porcentual de clases reales dentro de cada cluster.

6.1.3. Resultados cuantitativos

Tabla 6.1: Resultados globales del análisis no supervisado con K-Means sobre embeddings de entrenamiento.

Modelo	k	<i>Silhouette</i>	ARI	NMI
Virchow2	3	0.1241	0.2731	0.3089
Virchow2	5	0.1410	0.3753	0.3901
Virchow2	7	0.1488	0.2465	0.3564
Virchow2	10	0.1388	0.2270	0.3636
H-Optimus1	3	0.1096	0.3040	0.3838
H-Optimus1	5	0.1162	0.4487	0.4969
H-Optimus1	7	0.1302	0.3737	0.4663
H-Optimus1	10	0.1375	0.3144	0.4383

La primera observación importante es que los valores de *silhouette* son en general modestos en ambos modelos, indicando que el espacio latente no presenta una separación geométrica completamente limpia entre grupos, sino una estructura algo solapada. Es decir, las clases no forman bolsas perfectamente compactas. Esto coincide con la dificultad que ya se había observado en el análisis supervisado.

Aun así, aparecen diferencias relevantes entre ambos backbones. En términos de **separación geométrica**, **Virchow2** obtiene los mejores valores de *silhouette*, alcanzando su máximo en $k = 7$ con un valor de 0,1488. Esto sugiere que el espacio latente de Virchow2 tiende a organizarse de forma algo más compacta y separable desde un punto de vista geométrico.

Sin embargo, cuando se analiza la **coincidencia con las etiquetas reales**, la conclusión cambia. Tanto el **ARI** como el **NMI** son claramente superiores en **H-Optimus1** para todos los valores de k . El mejor resultado global de toda la comparación aparece en **H-Optimus1 con $k = 5$** , donde se alcanzan $ARI = 0,4487$ y $NMI = 0,4969$. Por tanto, aunque Virchow2 genere una estructura algo más limpia en términos geométricos, H-Optimus1 genera agrupamientos que se parecen más a la organización real del problema.

Esto no implica que uno de los modelos sea globalmente “mejor” que el otro, sino que ambos capturan aspectos distintos de la estructura del dataset. **Virchow2** parece construir un espacio latente más ordenado geométricamente, mientras que **H-Optimus1** consigue que esa organización esté más alineada con las subclases reales del dominio.

Respecto al número de clusters, el valor más informativo es $k = 5$, ya que proporciona la mejor correspondencia global con las etiquetas reales en ambos modelos, especialmente en H-Optimus1. A partir de $k = 7$ y $k = 10$, aunque la geometría pueda mejorar ligeramente en algunos casos, la correspondencia con las clases reales disminuye, lo que demuestra que el clustering empieza a subdividir regiones sin llegar a recuperar de forma limpia las siete subclases histológicas.

En consecuencia, este análisis demuestra que el dataset **sí contiene estructura** (ni el ARI ni el NMI son cercanos a cero) y, al mismo tiempo, que dicha estructura **no reproduce de forma natural las siete subclases como grupos perfectamente separados**. Esto refuerza la hipótesis central de esta fase extra: la dificultad del problema no depende únicamente del clasificador supervisado utilizado, sino también de la propia organización intrínseca del dataset en el espacio latente.

6.1.4. Análisis visual de la estructura latente

Para complementar los resultados cuantitativos, se analizó visualmente la organización del espacio latente mediante proyecciones 2D obtenidas con PCA. Esta representación permite observar si las clases reales aparecen agrupadas en regiones relativamente diferenciadas o si, por el contrario, se solapan ampliamente entre sí.

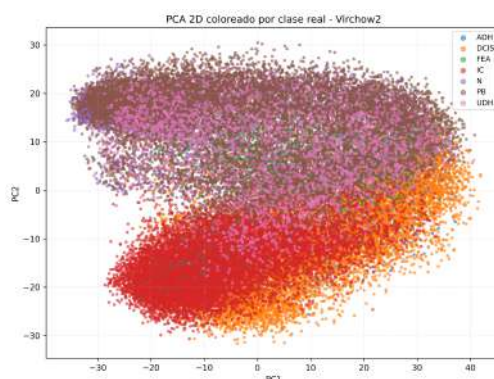


Figura 6.1: Proyección PCA 2D de los embeddings de entrenamiento de Virchow2.

En el caso de **Virchow2**, se muestra una organización geométrica relativamente compacta, aunque con una mezcla notable entre clases. No se distinguen siete regiones aisladas, sino más bien una estructura continua en la que algunas categorías ocupan áreas preferentes, pero sin fronteras claras entre todas ellas.

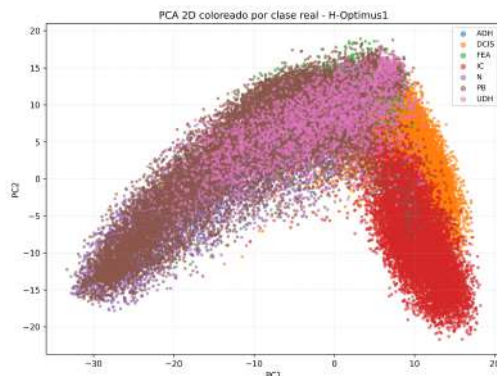


Figura 6.2: Proyección PCA 2D de los embeddings de entrenamiento de H-Optimus1.

En **H-Optimus1**, la estructura visual resulta algo más interpretable desde el punto de vista de las clases reales. Se aprecia una organización más clara en grandes regiones del espacio, con una zona asociada a clases benignas, otra más próxima a lesiones malignas y una región intermedia con un claro solapamiento entre subclases. Esta observación es coherente con los resultados previos, donde H-Optimus1 mostraba una mejor alineación entre clusters y etiquetas reales.

Para comprobar si existe una estructura más gruesa del problema, se aplicó K-Means con $k = 3$. Las Figuras 6.3 y 6.4 muestran el resultado de este agrupamiento sobre las proyecciones PCA.

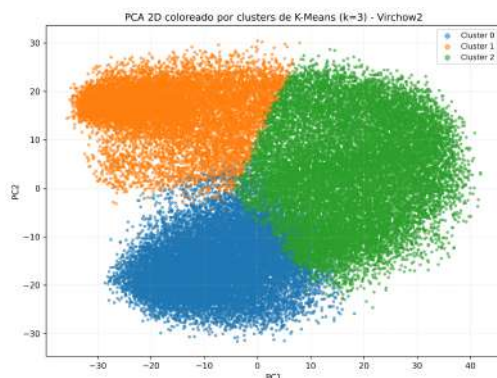


Figura 6.3: Agrupamiento K-Means con $k = 3$ sobre los embeddings de entrenamiento de Virchow2.

En **Virchow2**, el agrupamiento con $k = 3$ muestra tres grandes regiones del espacio latente. Aunque las fronteras no son completamente nítidas, la partición es compatible con una separación en tres zonas: benigna, maligna y una de transición.

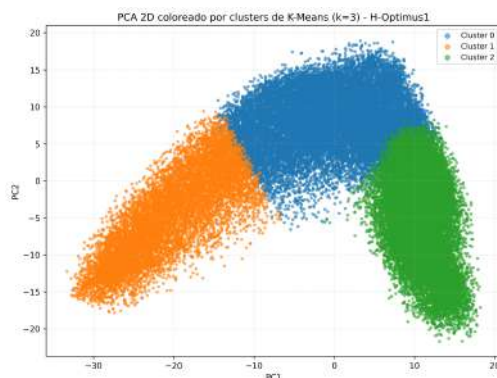


Figura 6.4: Agrupamiento K-Means con $k = 3$ sobre los embeddings de entrenamiento de H-Optimus1.

En **H-Optimus1** se observa un patrón similar, reflejando con bastante naturalidad una organización donde un grupo recoge regiones asociadas a clases benignas, otro se aproxima a las clases malignas y el tercero actúa como zona intermedia.

En conjunto, el análisis visual deja una conclusión clara: el espacio de embeddings no se organiza espontáneamente en siete bolsas limpias, pero sí aparece una estructura global en tres regiones.

6.1.5. Análisis fino con $k = 5$

Una vez vemos que con $k = 3$ hay una estructura compatible con tres grandes regiones benignas, malignas e intermedias, el siguiente paso consiste en analizarlo más a fondo. Para ello, se estudió con especial atención el caso $k = 5$, ya que fue el valor que proporcionó la mejor correspondencia global con las etiquetas reales en ambos modelos.

Las Figuras 6.5 y 6.6 muestran la distribución porcentual de clases reales dentro de cada cluster obtenido con K-Means. Esta representación permite interpretar qué grupos aparecen más puros y cuáles siguen actuando como mezcla de varias subclases.

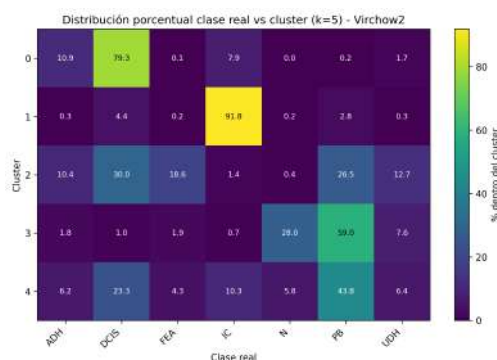


Figura 6.5: Mapa de calor de clases reales para Virchow2 con $k = 5$.

En **Virchow2**, el patrón general muestra varias cosas:

- Por un lado, aparecen 2 clusters relativamente puros para las clases malignas **IC** y **DCIS**, en los clusters 1 y 0 respectivamente.
- Por otro lado, el cluster 3 es de carácter claramente benigno, formado principalmente por **PB** y **N**.
- Sin embargo, los clusters 2 y 4 tienen una composición mucho más heterogénea. El cluster 2 mezcla de forma notable **PB**, **DCIS**, **FEA**, **UDH** y **ADH**, mientras que el cluster 4 combina **PB** con porcentajes bastante notables de **DCIS** e **IC**, además de otras clases.

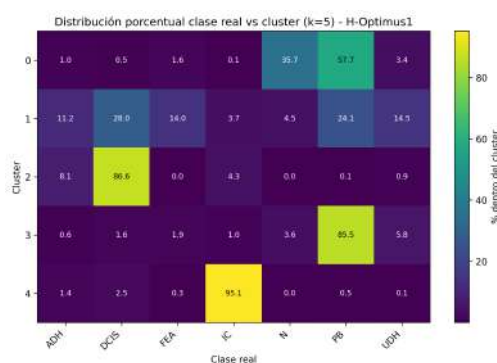


Figura 6.6: Mapa de calor de clases reales dentro de cada cluster para H-Optimus1 con $k = 5$.

En **H-Optimus1**, la estructura resulta aún más expresiva:

- Vuelven a aparecer dos clusters muy puros para las clases malignas extremas: el cluster 2 está dominado por **DCIS** con un 86,64 % de sus muestras, mientras que el cluster 4 es prácticamente exclusivo de **IC** con un 95,15 % de pureza.
- En los benignos, el cluster 0 se compone principalmente de **PB** y **N**, mientras que el cluster 3 es casi exclusivamente de **PB**.
- El cluster 1 actúa como la principal región de mezcla del problema, reuniendo clases intermedias como **FEA**, **ADH** y **UDH**, pero también porcentajes relevantes de **PB** y **DCIS**.

En conjunto, el análisis con $k = 5$ refuerza de forma muy clara la hipótesis central de esta fase extra. El dataset BRACS no tiene las siete subclases totalmente separadas, sino que parece organizarse bien en los extremos benigno y maligno, mientras que las clases intermedias permanecen como el núcleo principal de mezcla y ambigüedad.

Virchow2 sigue mostrando una estructura geométrica razonable, pero con dos clases de mezcla. **H-Optimus1**, en cambio, consigue una organización más alineada con las etiquetas reales, separando mejor los grupos extremos y dejando más localizada en un solo cluster la región intermedia de transición.

6.1.6. Conclusiones

A partir del análisis no supervisado sobre los embeddings de entrenamiento, pueden extraerse las siguientes conclusiones principales:

- El dataset **sí presenta estructura latente**, ya que el algoritmo de clustering empleado no produce agrupamientos aleatorios, sino regiones con una correspondencia apreciable con parte de las clases reales.
- Sin embargo, dicha estructura **no reproduce de forma natural las siete subclases como grupos completamente separados**. Esto confirma que la dificultad del problema no depende únicamente del clasificador supervisado, sino también de la propia organización intrínseca del dataset.
- Los resultados muestran de forma consistente que las clases más extremas y mejor definidas, en particular **IC** y **DCIS**, tienden a concentrarse en clusters mucho más puros.
- En el extremo benigno, **N** y **PB** aparecen en regiones próximas o compartidas, lo que sugiere que ambas clases mantienen una gran similitud.
- Las clases **UDH**, **FEA** y **ADH** son el núcleo principal de mezcla del problema, ya que tienden a repartirse entre varios clusters o a convivir dentro de regiones compartidas.
- Desde el punto de vista cuantitativo, **k = 5** es el mejor valor, al proporcionar la mejor alineación global con las etiquetas reales.
- La comparación entre modelos muestra que **Virchow2** genera una estructura algo más limpia en términos geométricos, mientras que **H-Optimus1** produce agrupamientos más alineados con las clases histológicas reales.

En conjunto, esta fase extra refuerza la interpretación obtenida en el resto del trabajo: el problema planteado por BRACS no solo es exigente desde el punto de vista supervisado, sino que presenta una complejidad estructural intrínseca en el propio espacio latente, especialmente en las clases intermedias.

6.2. Proyección clínica en 3 clases

Como último análisis, se estudió el comportamiento de las dos mejores configuraciones obtenidas previamente en el problema de 7 clases, pero reinterpretando sus salidas

en la formulación clínica de **3 clases**. En lugar de entrenar nuevos modelos desde cero, se reutilizaron directamente las probabilidades a nivel de ROI ya obtenidas con la regla **mean_proba**, agrupando las siete subclases originales según el esquema oficial en 3 clases: **AT** (FEA, ADH), **BT** (N, PB, UDH) y **MT** (DCIS, IC).

Se evaluaron en esta proyección clínica las configuraciones principales consideradas a lo largo del estudio en 7 clases: **baseline**, **Información Mutua (IM)**, **RandomUnderSampler** y **NCR** para **H-Optimus1** y **Virchow2**.

Tabla 6.2: Resultados a nivel de ROI tras la proyección clínica de 7 clases a 3 clases.

Modelo	Método	Accuracy	F1 macro
Virchow2	Baseline	0.7789	0.7673
Virchow2	NCR	0.7772	0.7631
Virchow2	RandomUnderSampler	0.7754	0.7534
Virchow2	IM	0.7649	0.7404
H-Optimus1	Baseline	0.7684	0.7578
H-Optimus1	NCR	0.7667	0.7532
H-Optimus1	RandomUnderSampler	0.7596	0.7388
H-Optimus1	IM	0.7649	0.7461

Los resultados muestran que, al reinterpretar clínicamente en 3 clases, alcanzan en general un rendimiento muy alto. La mejor combinación global pasa a ser **Virchow2 + baseline**, que obtiene la mayor *accuracy* (0,7789) y también el mejor *F1 macro* (0,7673). Muy próxima queda **Virchow2 + NCR**, mientras que **H-Optimus1 + baseline** sigue manteniendo un comportamiento competitivo y cercano al grupo de cabeza.

La conclusión vuelve a ser clara: al reducir la granularidad y al no intentar hilar tan fino, el rendimiento es muy notable respecto al escenario de 7 clases y las dos mejores configuraciones convergen hacia un comportamiento muy próximo. Esto vuelve a reafirmar que parte importante de la dificultad observada anteriormente estaba asociada a la separación fina entre subclases (especialmente entre las atípicas), mientras que la estructura clínica más gruesa queda ya razonablemente bien capturada por ambos modelos.

A continuación se examinan con más detalle las dos configuraciones más competitivas de esta comparación: **Virchow2 + baseline** y **Virchow2 + NCR**, permitiendo ilustrar dónde se siguen concentrando los errores una vez reducida la granularidad diagnóstica.

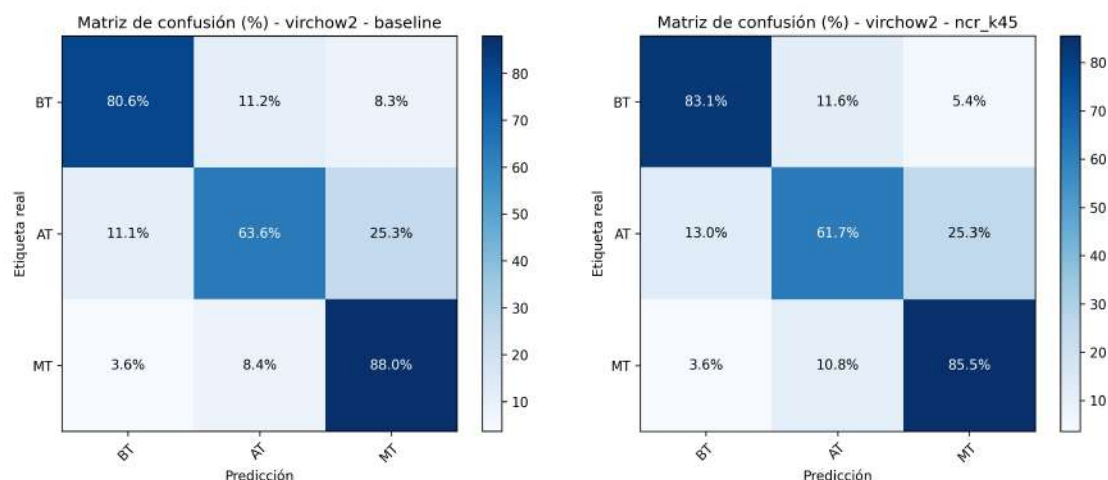


Figura 6.7: Matrices de confusión a nivel de ROI en 3 clases de las dos configuraciones más competitivas.

La Figura 6.7 permite de nuevo comprobar dónde se concentran los errores en esta formulación de 3 clases. En ambos casos, las clases **BT** y **MT** presentan diagonales altas, lo que indica que la separación entre tejido benigno y lesiones malignas queda razonablemente bien capturada.

En ambas configuraciones, el principal foco de dificultad sigue estando en la categoría intermedia:

- **Virchow2+Baseline:** La clase **BT** alcanza un 80,6% de acierto y la clase **MT** un 88%, mientras que la clase **AT** baja hasta 63,6%. Esta categoría intermedia se confunde más con **MT** (25,3%) que con **BT** (11,1%).
- **Virchow2+NCR:** La clase **BT** alcanza un 83,1% y la clase **MT** hasta 85,5%, pero la clase **AT** desciende hasta 61,7%. Es decir, esta configuración penaliza algo más la categoría intermedia, que vuelve a repartirse más entre **MT** (25,3%) que entre **BT** (13%).

La lectura de ambas matrices resulta bastante interesante porque, al pasar del problema de 7 clases al de 3, **cambian las dos mejores configuraciones**. En el escenario original, las combinaciones que destacaban eran **H-Optimus1 + baseline** y **Virchow2 + RandomUnderSampler**. Sin embargo, tras reagrupar las clases en **AT**, **BT** y **MT**, las dos configuraciones más competitivas pasan a ser **Virchow2 + baseline** y **Virchow2 + NCR**. Este cambio sugiere que el orden relativo entre métodos no es completamente rígido cuando se modifica la granularidad diagnóstica.

En conclusión, este cambio de los dos mejores sugiere que, al simplificar el problema, muchas confusiones finas entre subclases desaparecen y no penalizan tanto, pasando a capturar mejor la estructura clínica gruesa del problema, aunque la dificultad sigue residiendo en la clase intermedia **AT**, que continúa actuando como zona de transición entre benignas y malignas.

6.3. Análisis de ROIs dudosas mediante prototipos claros

Como análisis complementario, se estudió si las *ROIs* enviadas a revisión por la regla de abstención (véase apartado 5.5) podían explicarse a partir de su similitud con ejemplos especialmente claros del conjunto de test. La idea es: si una ROI dudosa se parece más a ejemplos prototípicos (ideales) de otra clase que a ejemplos prototípicos de su propia clase, entonces la incertidumbre del modelo no debe interpretarse como un fallo del mismo, sino como una señal de ambigüedad en el espacio latente que puede ser causa de una ambigüedad a su vez en la etiqueta.

Para ello, se extrajo un conjunto de **prototipos claros** a partir de las predicciones a nivel de ROI, exigiendo que dichas muestras fueran correctas, no enviadas a revisión con $\tau = 0,10$, con al menos tres patches y con alta confianza. En primer lugar, se impuso un umbral estricto de `top1_prob` $\geq 0,90$. Sin embargo, dado que las clases intermedias apenas generaban ejemplos con ese nivel de seguridad, se adoptó una estrategia jerárquica: únicamente en aquellas clases con representación insuficiente, se relajó el criterio hasta `top1_prob` $\geq 0,80$, manteniendo el resto de condiciones.

Finalmente, cada ROI dudosa se representó mediante la media de los embeddings patch-level de sus patches, y se buscaron sus tres prototipos claros más cercanos mediante similitud coseno. Este procedimiento permite estudiar si la duda del modelo se mantiene anclada en la propia clase real o si, por el contrario, la ROI se parece a prototipos claros de otras clases.

Aunque inicialmente se planteó este análisis sobre las dos configuraciones principales del estudio, finalmente se continuó únicamente con **H-Optimus1 + baseline** debido a que **Virchow2 + RandomUnderSampler** no generó ejemplos claros en las clases intermedias, especialmente en **FEA** y **ADH**. Por el contrario, **H-Optimus1 + baseline** sí conservó un conjunto mínimo de prototipos intermedios, lo que permitió realizar un estudio más equilibrado y, sobre todo, más informativo sobre las zonas de transición del problema.

Resultados globales del análisis por prototipos

Una vez construido el banco de prototipos claros, se analizó la relación entre cada ROI dudosa y su prototipo más cercano en el espacio latente. Los resultados muestran que, de las 69 ROIs enviadas a revisión, en un 47,83 % de los casos el prototipo más cercano pertenece a la **misma clase real**, mientras que en un 30,43 % de los casos dicho prototipo coincide con la **clase competidora** situada en segunda posición (*top-2*) por el modelo.

Es decir, por un lado, existen ROIs dudosas que aun generando una predicción insegura, permanecen ancladas en la región latente de su propia clase. En estos casos, la duda del modelo parece responder más a una frontera difusa dentro de la misma categoría que a un desplazamiento real hacia otra clase. Por otro lado, también aparecen casos en los que la ROI dudosa se aproxima más a un prototipo claro de la clase rival,

lo que sugiere una ambigüedad estructural más profunda y no un simple error arbitrario del clasificador.

Las ROIs dudosas de **FEA** se mantienen completamente ancladas en prototipos de su propia clase, mientras que **ADH** vuelve a aparecer como la categoría más inestable: solo un 8,33 % de sus ROIs dudosas encuentran como vecino más cercano un prototipo claro de ADH, mientras que una fracción importante se desplaza hacia **FEA** o **DCIS**. De forma similar, en **PB** y **N** se observa una proximidad mutua coherente con la frontera benigna ya identificada en análisis anteriores.

En conjunto, estos resultados indican que la duda del modelo no se distribuye al azar. En una parte importante de los casos, la incertidumbre sigue siendo compatible con la propia clase real. En las clases más conflictivas, especialmente las intermedias, muchas ROIs dudosas se aproximan a prototipos claros de clases vecinas.

Análisis visual de casos representativos

Para estudiarlo más a fondo, se seleccionaron tres casos representativos:

Caso 1. ROI dudosa de FEA anclada en un prototipo claro de FEA

```
ROI dudosa: BRACS_1283_FEA_8
Clase real: 3
Top1: 3 (p=0.421)
Top2: 4 (p=0.418)
Margen top1-top2: 0.003
Entropía: 1.209
Nº patches ROI dudosa: 17

Prototipos más cercanos:
#1 BRACS_1943_FEA_18 | clase=FEA | sim=0.809 | nivel=relaxed
#2 BRACS_1940_PB_1 | clase=PB | sim=0.696 | nivel=strict
#3 BRACS_1589_ADH_10 | clase=ADH | sim=0.692 | nivel=relaxed
```

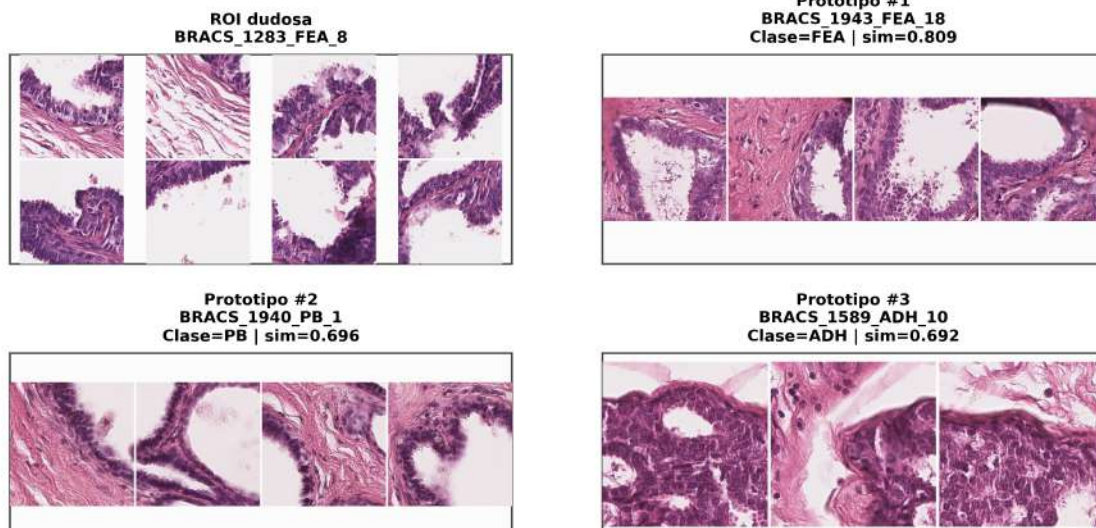


Figura 6.8: ROI dudosa de FEA con prototipo más cercano FEA.

La Figura 6.8 ilustra un caso de duda que, sin embargo, permanece anclado en la propia clase real. Aunque la ROI presenta una competencia muy estrecha entre **FEA** y **ADH**, su prototipo más cercano es una ROI clara de **FEA**, mostrando que la duda no implica necesariamente un desplazamiento hacia otra clase, sino que en algunos casos la muestra sigue ocupando una región coherente con su propia categoría.

Caso 2. ROI dudosa de ADH próxima a un prototipo claro de FEA

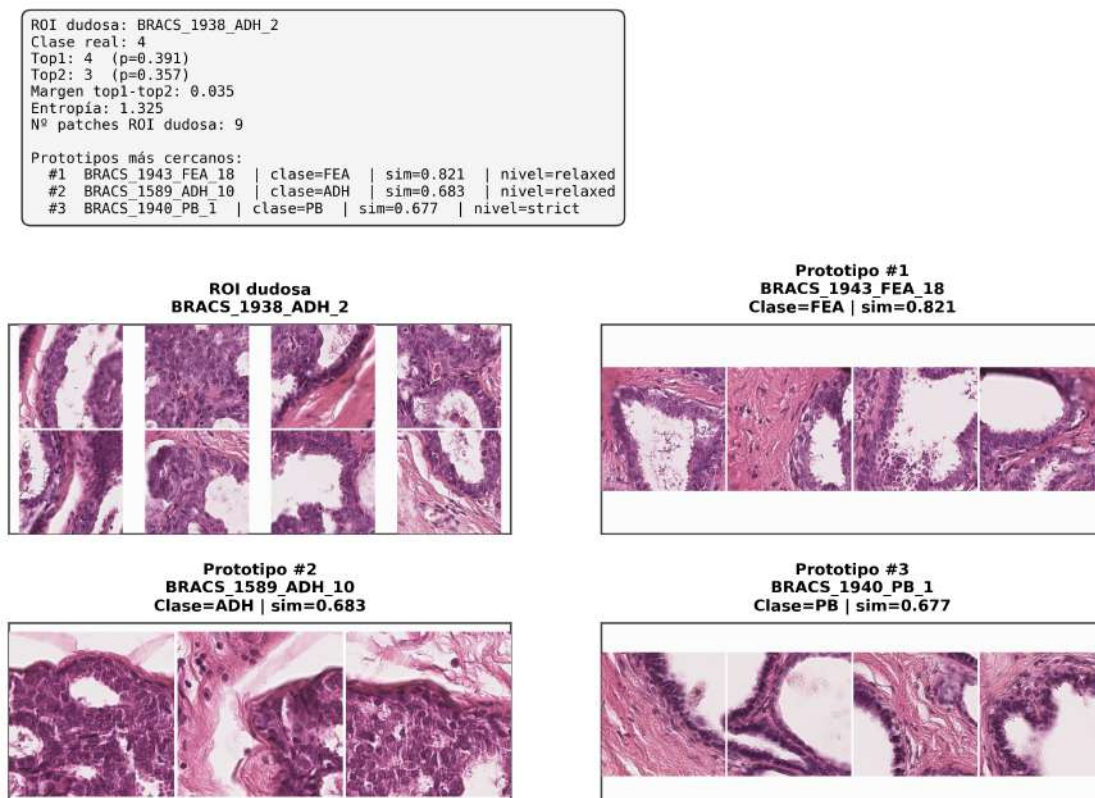


Figura 6.9: ROI dudosa de ADH próxima a FEA.

La Figura 6.9 muestra el caso más representativo de esta fase. La ROI pertenece realmente a **ADH**, pero su prototipo más cercano es una ROI clara de **FEA**, con una similitud incluso superior a la observada en algunos casos anclados en su propia clase. Esta situación refuerza la idea de que **ADH** constituye una región especialmente inestable del problema, ya que sus muestras dudosas no siempre encuentran apoyo en prototipos claros de la propia clase, sino que se redistribuyen hacia categorías vecinas.

Caso 3. ROI dudosa de PB próxima a un prototipo claro de N

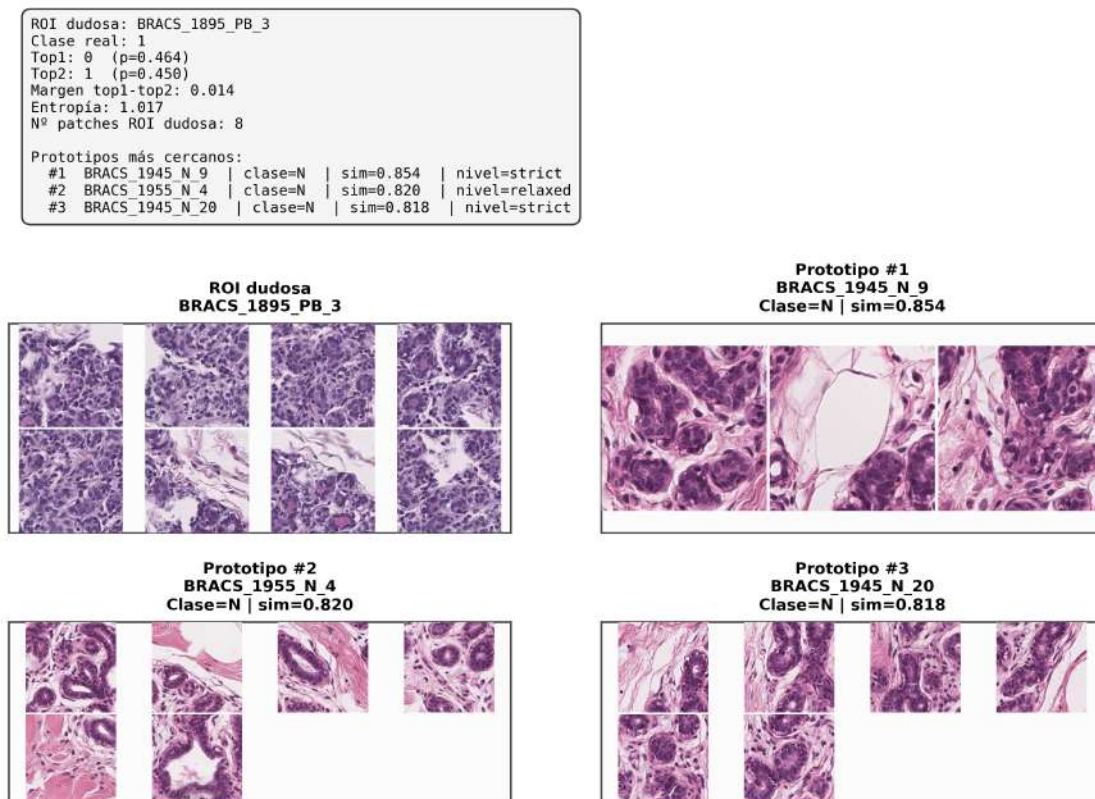


Figura 6.10: ROI dudosa de PB próxima a N.

Por último, la Figura 6.10 recoge un ejemplo dentro del extremo benigno. En este caso, una ROI real de **PB** aparece más próxima a un prototipo claro de **N**, lo que encaja con la proximidad estructural ya observada entre ambas clases en los análisis previos. Este ejemplo resulta útil porque muestra que este análisis por prototipos no solo explica transiciones entre clases intermedias, sino también fronteras difusas dentro de regiones benignas.

En definitiva, el análisis por prototipos confirma que la incertidumbre del modelo no es arbitraria. Una parte de las ROIs dudosas sigue anclada en prototipos claros de su propia clase, mientras que otra parte, especialmente en clases intermedias como **ADH**, se aproxima a prototipos de clases vecinas como **FEA** o **DCIS**. Por tanto, este último bloque refuerza la misma conclusión obtenida a lo largo del trabajo: una parte importante de la dificultad de BRACS no proviene solo del clasificador, sino de la propia ambigüedad estructural del problema.

6.4. Selección de características sobre embeddings

Como último experimento, se estudió si una reducción de las características de los embeddings fundacionales podía mantener, o incluso mejorar, el rendimiento del sistema. La motivación de este bloque fue comprobar si todas las dimensiones del embedding eran realmente necesarias o si, por el contrario, una representación más compacta podía conservar la información más relevante para la clasificación.

Para ello, se tomó como referencia la configuración basada en **Virchow2** y se planteó una comparación en la que únicamente se modificaba el subconjunto de características de entrada, manteniendo fijo el resto del pipeline: mismo backbone, misma cabeza lineal, mismo protocolo de entrenamiento y misma evaluación final a nivel de ROI con la regla *mean_proba*.

Se evaluaron dos métodos distintos:

- **Hy-index:** método de selección de características basado en la evaluación de subconjuntos de variables, combinando relevancia respecto a la clase y criterio de selección global.
- **mRMR (*minimum Redundancy Maximum Relevance*):** método clásico de selección de características que busca un equilibrio entre conservar variables con alta relevancia respecto a la clase y evitar seleccionar dimensiones redundantes entre sí.

En ambos casos, la selección se aplicó sobre los embeddings de entrenamiento y, a partir de las dimensiones retenidas, se reconstruyeron los embeddings reducidos de *train*, *val* y *test*. Posteriormente, se reentrenó la misma cabeza lineal utilizada en el resto del proyecto y se evaluó el rendimiento final a nivel de ROI.

Tabla 6.3: Resultados de selección de características sobre embeddings de Virchow2 a nivel de ROI.

Método	Dimensiones retenidas	Accuracy ROI	F1 macro ROI
Baseline	2560 (100.00 %)	0.6088	0.5666
Hy-index	652 (25.47 %)	0.5596	0.5323
mRMR	652 (25.47 %)	0.5684	0.5463
mRMR	1280 (50.00 %)	0.5667	0.5452

Los resultados muestran que **mRMR** se comporta mejor que **Hy-index** cuando ambos se comparan con el mismo grado de reducción del embedding, ya que mejora tanto la *accuracy* como el *F1 macro* a nivel de ROI. Sin embargo, ninguna de las configuraciones con selección de características logra igualar el rendimiento del **baseline** original.

La caída es especialmente acusada en el caso de **Hy-index**, mientras que **mRMR** consigue resultados algo más competitivos, aunque insuficientes para justificar la reducción.

Además, aumentar la retención con mRMR desde el 25 % hasta el 50 % no aporta una mejora real, ya que los resultados de ambos son prácticamente equivalentes. Esto sugiere que el problema no reside únicamente en el método de selección empleado, sino en que la reducción explícita del embedding elimina información relevante para la clasificación final a nivel de ROI.

Por tanto, esta fase se interpretó como un experimento exploratorio final. Aunque permitió comparar dos estrategias distintas de selección de características, la conclusión obtenida fue clara: **en este problema, mantener el embedding fundacional completo resulta mejor que forzar una reducción dimensional explícita.**

Capítulo 7

Conclusiones y Trabajos futuros

7.1. Conclusiones

Este Trabajo de Fin de Grado ha abordado el problema de la clasificación automática de lesiones mamarias histopatológicas sobre el dataset BRACS, siguiendo un enfoque progresivo desde el nivel de *patch* hasta el nivel de ROI, y analizando distintas estrategias de reducción de datos, mecanismos de abstención y bloques complementarios de interpretación de resultados. Más allá de la mera comparación de modelos, el trabajo no se ha limitado únicamente a ello, sino que se ha orientado a comprender qué configuraciones resultan más sólidas, en qué situaciones aparecen las principales confusiones y hasta qué punto dichas confusiones dependen del clasificador o de la propia estructura del dataset utilizado.

En conjunto, los resultados obtenidos permiten afirmar que el **objetivo general OBJ1**, junto con los **objetivos específicos principales (OBJ1.1, OBJ1.2, OBJ1.3, OBJ1.4, OBJ1.5 y OBJ1.6)** establecidos al inicio del trabajo, se han alcanzado y satisfecho en su totalidad. Asimismo, los **objetivos complementarios (OBJ1.7 y OBJ1.8)**, incorporados posteriormente, también pudieron abordarse de forma satisfactoria, ampliando el proyecto con análisis adicionales orientados a profundizar en la interpretación del problema y a reforzar la solidez de las conclusiones extraídas.

- En primer lugar, el trabajo confirma de forma consistente que los **modelos fundacionales** son la mejor alternativa actual para este problema, superando con claridad a las arquitecturas CNN y *Vision Transformers*. En particular, y como ya se esperaba en la revisión del estado del arte, **H-Optimus1** y **Virchow2** se consolidaron como las configuraciones más competitivas y sirvieron como base para el resto del desarrollo experimental.
- En segundo lugar, los resultados muestran que el problema no debe interpretarse desde el rendimiento a nivel de *patch*, ya que no es representativo. El paso a evaluación a nivel de **ROI** es fundamental para que el escenario sea más realista,

y permitió comprobar que la regla de agregación basada en **media de probabilidades** representa la decisión final de forma más robusta que otras alternativas más simples.

- Asimismo, las distintas estrategias de reducción del conjunto de entrenamiento permitieron extraer una conclusión metodológica relevante: **reducir el número de patches no implica necesariamente mejorar el sistema, pero sí puede servir para mejorar la eficiencia computacional manteniendo el rendimiento**. Aunque algunos métodos ofrecieron resultados competitivos o relativamente próximos al *baseline*, en conjunto no se observó una mejora estable que justificara sustituir el *pipeline* original. Aun así, estas estrategias sí podrían resultar valiosas en contextos donde interese reducir el tiempo de entrenamiento manteniendo un rendimiento próximo con menos datos. Esto sugiere que, en este problema, la aparente redundancia convive con una variabilidad útil que no conviene eliminar de forma agresiva.
- Por otra parte, la incorporación de mecanismos de **abstención** permitió dar un paso más allá de la clasificación obligada. En lugar de forzar siempre una decisión, el sistema pudo identificar de manera razonable aquellas ROIs más conflictivas, concentrando la incertidumbre en pares de clases histológicamente próximas. Este resultado es especialmente valioso, ya que aproxima el comportamiento del modelo a un esquema más prudente y compatible con contextos reales de apoyo al diagnóstico.
- Los análisis complementarios realizados en el bloque de trabajo extra reforzaron una idea central del proyecto: una parte importante de las confusiones observadas no parece deberse únicamente a limitaciones del clasificador, sino a la propia **estructura del problema**. Tanto el análisis no supervisado del espacio latente como el estudio por prototipos claros y casos dudosos apuntan a que existen zonas de transición y ambigüedad diagnóstica intrínseca, especialmente en las clases intermedias. En este sentido, el trabajo no solo compara modelos, sino que contribuye también a interpretar mejor la dificultad real del dataset BRACS.
- Finalmente, los experimentos exploratorios de selección de características permitieron cerrar el estudio con una conclusión adicional: forzar una reducción explícita de la dimensionalidad de los *embeddings* fundacionales no es beneficioso en este problema. Incluso cuando se emplearon métodos más competitivos como **mRMR**, el rendimiento final quedó por debajo del *baseline* original, lo que sugiere que estas representaciones ya contienen una organización suficientemente rica y que una eliminación agresiva priva al modelo de información relevante para la decisión final.

Desde esta perspectiva, puede considerarse que tanto el **objetivo general** del TFG como los **objetivos específicos principales** se cubrieron con el desarrollo experimental

central, mientras que los **objetivos complementarios** se materializaron en los bloques de trabajo extra y en la interpretación avanzada de los resultados.

En definitiva, este TFG no solo ha permitido construir y evaluar un sistema completo de clasificación histopatológica sobre BRACS, sino también comprender mejor la naturaleza del problema. El trabajo realizado muestra que los mejores resultados no dependen únicamente de elegir un modelo potente sin estudiar el problema, sino de combinarlo con una metodología rigurosa, una evaluación realista y un análisis cuidadoso de la incertidumbre. Desde esta perspectiva, la principal aportación del proyecto no reside únicamente en las métricas alcanzadas, sino en haber construido una visión más completa, crítica y fundamentada del comportamiento del sistema, identificando tanto sus virtudes como sus defectos en un escenario histopatológico muy complejo.

7.2. Trabajos futuros

Como líneas naturales de continuación, resultaría especialmente interesante abordar el problema desde su origen y reconstruir el dataset con una anotación más precisa a nivel de *patch*, con el fin de reducir el ruido de heredar directamente la etiqueta de la ROI completa, donde pueden habitar distintas clases histológicas. Del mismo modo, sería relevante estudiar enfoques de agregación e interpretación a nivel de muestra, como estrategias basadas en *multiple instance learning*. También sería una práctica valiosa validar el pipeline sobre conjuntos externos o escenarios más cercanos a la práctica clínica real, incorporando además un análisis más profundo de incertidumbre y calibración. Finalmente, la continuidad del trabajo podría consistir en integrar estos resultados en una herramienta más completa de apoyo al diagnóstico, donde el sistema no solo clasifique, sino que también priorice revisión, aporte evidencia visual y facilite la interacción con el especialista.

Bibliografía

- [1] Asociación Española Contra el Cáncer. «¿Qué es el cáncer de mama?» Dirección: <https://www.contraelcancer.es/es/todo-sobre-cancer/tipos-cancer/cancer-mama/que-es-cancer-mama>.
- [2] International Agency for Research on Cancer. «Global Cancer Observatory: Cancer Today.» Data visualization tools for estimated cancer incidence, mortality and prevalence. dirección: <https://gco.iarc.fr/today/en>.
- [3] Mayo Clinic Staff. «Biopsia de mama: Procedimientos y riesgos,» Mayo Foundation for Medical Education y Research (MFMER). dirección: <https://www.mayoclinic.org/es/tests-procedures/breast-biopsy/about/pac-20384812>.
- [4] N. Brancati, A. M. Anniciello, P. Pati et al., «BRACS: A Dataset for BReAst Carcinoma Subtyping in H&E Histology Images,» *Database*, vol. 2022, 2022. DOI: 10.1093/database/baac093. dirección: <https://academic.oup.com/database/article/doi/10.1093/database/baac093/6762351>.
- [5] WHO Classification of Tumours Editorial Board, *Breast Tumours* (WHO Classification of Tumours), 5.^a ed. Lyon (France): IARC Press, 2019, vol. 2, ISBN: 978-92-832-4502-5.
- [6] Fundación QUAES. «La patología digital: el gran abanico multicolor abierto a investigadores del área de la salud.» dirección: https://www.fundacionquaes.org/retos_biomedicina/la-patologia-digital-el-gran-abanico-multicolor-abierto-a-investigadores-del-area-de-la-salud/.
- [7] Bradley. «Whole Slide Imaging: Digital Pathology Revolution,» Targeted Bioscience. dirección: <https://www.targetedbioscience.com/post/whole-slide-imaging>.
- [8] C. Sampias y G. Rolls. «H&E Staining Overview: A Guide to Best Practices,» Leica Biosystems. dirección: <https://www.leicabiosystems.com/es/knowledge-pathway/he-staining-overview-a-guide-to-best-practices/>.

- [9] S. K. Suvarna, J. D. Layton y J. D. Bancroft, *Bancroft's theory and practice of histological techniques*, 8th. London, UK: Elsevier Health Sciences, 2018.
- [10] J. A. Kiernan, *Histological and histochemical methods: theory and practice*, 5th. Oxford, UK: Scion Publishing Ltd, 2015.
- [11] J. K. Chan, «The wonderful world of H&E,» *International Journal of Surgical Pathology*, vol. 22, n.º 4, págs. 292-307, 2014.
- [12] BRACS. «BRACS Dataset,» Institute for High Performance Computing y Networking (ICAR-CNR). dirección: <https://www.bracs.icar.cnr.it>.
- [13] M. Veta, P. J. van Diest, S. M. Willems et al., «Assessment of algorithms for mitosis detection in breast cancer histopathology images,» *Diagnostic Pathology*, vol. 10, n.º 1, págs. 1-7, 2015. DOI: 10.1186/1746-1596-9-121.
- [14] G. Litjens, T. Kooi, B. E. Bejnordi et al., «A survey on deep learning in medical image analysis,» *Medical Image Analysis*, vol. 42, págs. 60-88, 2017. DOI: 10.1016/j.media.2017.07.005.
- [15] elenabsl, *Artificial intelligence, machine learning and deep learning development infographic with icons and timeline*, ID del elemento: 1628424196. Consultado el 9 de junio de 2026, 2020. dirección: <https://www.shutterstock.com/es/image-vector/artificial-intelligence-machine-learning-deep-development-1628424196>.
- [16] A. Karpathy, J. Johnson y L. Fei-Fei, *CS231n Convolutional Neural Networks for Visual Recognition: Neural Networks Part 1*, <https://cs231n.github.io/neural-networks-1/>, Universidad de Stanford. [Accedido el 09-junio-2026], 2015.
- [17] A. Richaud, *Redes perceptrón multicapa (MLP): ¿Qué son, cómo funcionan y cuándo utilizarlas? (o no)*, Publicado el 18 de octubre de 2024. Consultado el 9 de junio de 2026, oct. de 2024. dirección: <https://antonio-richaud.com/blog/archivo/publicaciones/41-redes-perceptron-multicapa.html>.
- [18] M. M. López, *¿Qué es una función de activación en Deep Learning?* Última modificación: 22 de enero de 2025. Consultado el 9 de junio de 2026, ene. de 2025. dirección: <https://keepcoding.io/blog/funcion-de-activacion-en-deep-learning/>.
- [19] The MathWorks, Inc., *¿Qué son las redes neuronales convolucionales?* Consultado el 9 de junio de 2026, 2024. dirección: <https://la.mathworks.com/discovery/convolutional-neural-network.html>.

- [20] V. Rajput, *Pooling layers in Neural nets and their variants*, Publicado en AI Guys el 10 de enero de 2022. Consultado el 9 de junio de 2026, ene. de 2022. dirección: <https://medium.com/aiguys/pooling-layers-in-neural-nets-and-their-variants-f6129fc4628b>.
- [21] P. Prajapati, *Residual blocks — Building blocks of ResNet*, Publicado en Toward Data Science el 25 de febrero de 2019. Consultado el 9 de junio de 2026, feb. de 2019. dirección: <https://medium.com/data-science/residual-blocks-building-blocks-of-resnet-fd90ca15d6ec>.
- [22] A. Vaswani et al., «Attention is all you need,» en *Advances in neural information processing systems*, vol. 30, 2017.
- [23] A. Dosovitskiy et al., «An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,» en *International Conference on Learning Representations (ICLR)*, 2021.
- [24] R. J. Chen, M. Y. Lu, D. F. K. Williamson, T. Y. Chen, J. Lipkova y F. Mahmood, «Towards a general-purpose foundation model for computational pathology,» *Nature Medicine*, 2024.
- [25] E. Vorontsov, A. Bozkurt, A. Casson, G. Shaikovski, M. Zelechowski, S. Liu et al., «Virchow: A Million-Slide Digital Pathology Foundation Model,» *arXiv preprint arXiv:2309.07778*, 2023.
- [26] Owkin, «Phikon: A large-scale foundation model for histopathology,» *arXiv preprint arXiv:2307.12914*, 2023.
- [27] A. et al., «PathBench: A Comprehensive Benchmark for Pathology Foundation Models,» *arXiv preprint arXiv:2505.20202*, 2025.
- [28] N. Brancati et al., «BRACS: A dataset for BReAst carcinoma subtyping in H&E histology images,» *Database*, vol. 2022, baac093, 2022.
- [29] OpenAI, *ChatGPT*, <https://openai.com/chatgpt/>, Accedido: 15 de junio de 2026, 2026.
- [30] Google, *Google Gemini*, <https://gemini.google.com/>, Accedido: 15 de junio de 2026, 2026.
- [31] A. Guillén, L. J. Herrera, G. Rubio, H. Pomares, A. Lendasse e I. Rojas, «New method for instance or prototype selection using mutual information in time series prediction,» *Neurocomputing*, vol. 73, n.º 10–12, págs. 2030–2038, 2010. DOI: 10.1016/j.neucom.2009.11.031.

Apéndice A

Repositorio del proyecto

A.1. Enlace al repositorio

Enlace: <https://github.com/juancarlosmoradiaz/TFG-BRACS>

A.2. Estructura general del repositorio

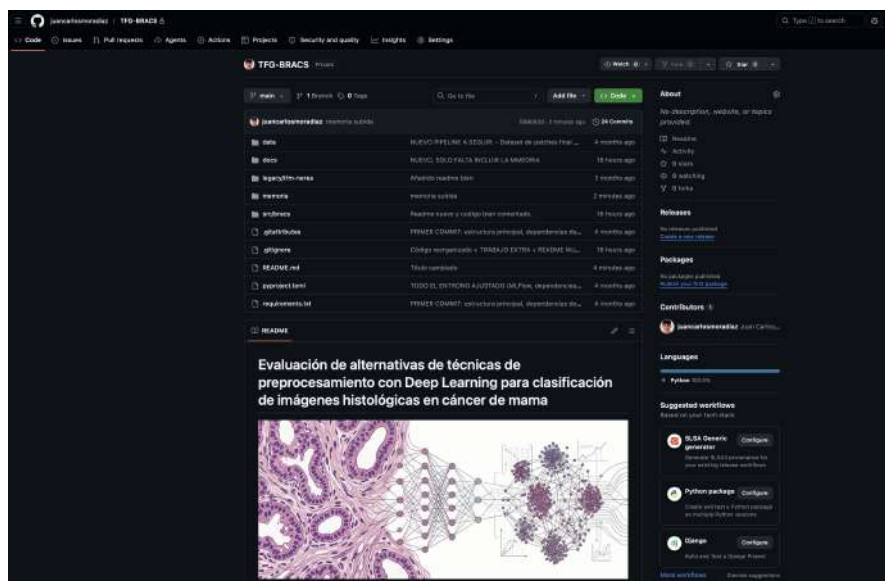


Figura A.1: Vista general del repositorio del TFG en GitHub.

La Figura A.1 muestra la vista principal del repositorio utilizado durante el desarrollo del proyecto. En él se centralizan tanto el código fuente como la memoria.

A.3. Organización del código experimental

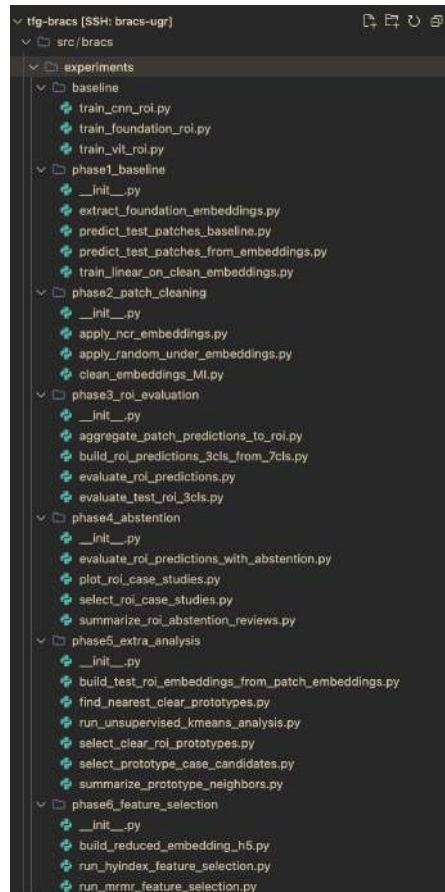


Figura A.2: Carpeta de scripts experimentales del proyecto.

La carpeta de experimentación concentra los scripts utilizados en las distintas fases del TFG. De este modo, la estructura del código refleja la evolución metodológica seguida durante el proyecto.

A.4. Control de versiones y trazabilidad

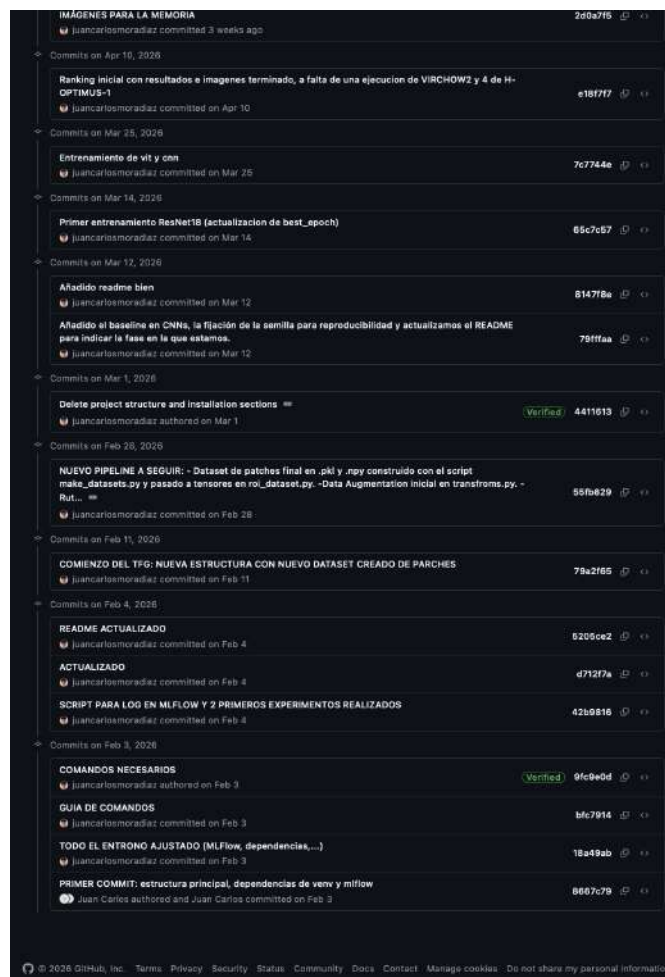


Figura A.3: Historial de commits del repositorio durante el desarrollo del TFG.

El uso de GitHub permitió mantener un historial de cambios continuo tanto sobre el código como sobre la memoria, facilitando la revisión del trabajo, la recuperación de versiones anteriores y la trazabilidad del desarrollo. Este aspecto complementa lo indicado en el Capítulo de Metodología y Planificación, donde se describió el uso de control de versiones como parte de la estrategia de reproducibilidad del proyecto.